

1.项目负责人

主要的工作是参与 前端工程化 , 基础设施建设 , 比如, 三库一架、多包仓库、私有仓库、文档系统、监控统、CI/CD 的开发, 三库一架, 也就是公司内部组件库 Hooks库 插件库 自定义脚手架, 除此之外, 也负责了 5个0到1的项目开发 以及现有项目多条业务线的二次开发迭代和维护, 参与落地20多个项目开发, 涉及医疗、物流、人工智能、金融、新能源的ToB ToC项目, 其中也会参与项目需求评审, 项目评估, 风险规避 跨部门合作沟通, 项目整体把控, 需要说明的是, 我作为前端项目负责人, 主要工作还是负责公司业务线的项目开发落地工作, 项目的基建, 项目0到1的开发, 带过3个前端开发, 主要负责项目的开发, 落地, 技术攻关, 并没有参与团队的绩效管理。

1.行业业务

行业	业务场景
金融行业	1. 资产管理平台：用于投资组合管理、自动化风险评估、收益分析、智能化的财务报表生成。 2. 支付清算系统：支持实时支付、跨国支付、交易清算和自动对账功能。 3. 移动支付应用：包括用户账户管理、转账、支付、消费记录管理等，提供便捷的移动端支付体验。 4. 保险服务平台：保险产品推荐、保单管理、在线保费计算、理赔服务，支持用户个性化定制保险方案。 5. 金融风控系统：整合大数据和 AI 技术，实时监控和预测金融风险，进行贷款风控、信用评估。
医疗行业	1. 电子病历系统（EMR）：涵盖患者健康档案管理、病历存储、历史治疗记录，支持跨院数据互通与医生间的协作。 2. 远程医疗平台：医生可通过该平台提供在线诊疗、健康监测、视频会诊、处方开具等服务。 3. 医疗设备管理系统：用于追踪设备状态、维护历史、使用效率，确保设备的正常运作并降低医疗事故发生率。 4. 预约挂号与排队管理系统：患者可以在线预约医生，查看候诊时间，支持智能排队提醒，提升患者就医体验。 5. 医疗保险平台：整合医保报销流程，支持患者在线查询报销进度，核算药品费用。
物流行业	1. 运输管理系统（TMS）：提供物流调度、运输路线规划、车辆管理、货物追踪，优化运输效率，减少成本。 2. 仓储管理系统（WMS）：自动化库存管理、订单处理、商品分拣、出入库优化，确保仓库运转高效。 3. 供应链管理系统（SCM）：涵盖从采购、库存到物流配送的全链路管理，确保供应链流畅运行，降低供应链风险。 4. 快递物流管理平台：提供包裹追踪、配送状态更新、签收确认、运输路径优化等功能，提升用户体验。 5. 智能货运匹配平台：基于 AI 和大数据进行智能匹配，帮助货主与承运商高效对接，实现智能调度与运输优化。
新能源行业	1. 智能电网管理系统：用于电力资源分配、电力调度、负荷预测、电力使用的实时监控与优化，保障电网运行的稳定性和可持续性。 2. 充电桩管理平台：管理充电桩的分布、使用情况，支持充电记录查询、实时电价显示，优化电动车用户充电体验。 3. 光伏发电管理平台：监控太阳能光伏设备的发电情况，分析发电效率，支持设备状态监控与异常告警，推动绿色能源的发展。 4. 能源交易平台：支持企业和个人参与能源交易，提供绿色电力交易、碳排放配额交易，助力新能源市场化发展。 5. 新能源车队管理系统：包括新能源车辆的调度、路线规划、实时监控、充电站信息查询，帮助企业降低能耗和运营成本。
	1. 商品管理系统：支持商品上架、分类管理、定价、库存更新、销售分析，帮助商家管理商品的全生命周期。

电商行业	2. 订单管理系统：包括订单生成、支付处理、发货管理、售后服务，提供一站式订单管理服务。 3. 客户关系管理（CRM）系统：分析用户行为，提供个性化营销策略、忠诚度计划、积分管理等，帮助电商平台提升用户粘性。 4. 营销推广系统：通过数据分析，进行精准广告投放，支持优惠券发放、折扣促销、限时抢购等营销活动，提升平台的营销效果。 5. 多渠道分销系统：支持 B2B、B2C 业务，打通线上线下销售渠道，实现多平台商品分发与库存管理。
教育行业	1. 在线教育平台：提供课程学习、考试系统、作业提交、视频直播、录播课程等功能，帮助学生随时随地进行学习。 2. 校园管理系统：包括学生信息管理、排课系统、考勤记录、成绩管理，优化学校日常运营效率。 3. 远程教育系统：支持教师与学生进行实时互动授课，提供多种互动方式如答疑、打分、作业批改等。 4. 在线考试系统：提供考试创建、自动阅卷、成绩分析、排名统计等功能，实现高效的考试管理。 5. 培训机构管理系统：帮助机构管理学员报名、课程安排、收费处理、培训进度跟踪，提高管理效率。
零售行业	1. 会员管理系统：通过积分系统、会员等级、消费记录管理、个性化推荐，帮助零售商提高会员粘性和复购率。 2. 库存管理系统：支持实时库存更新、商品入库出库管理、库存预警，帮助商家优化补货策略。 3. 收银系统：支持多种支付方式的 POS 收银系统，自动生成销售报表和财务报表，提升门店运营效率。 4. 促销活动管理系统：支持优惠券发放、折扣活动、会员特惠、满减促销等多种促销活动，结合用户行为数据进行精准推送。 5. 店铺管理系统：支持多门店的商品、订单、会员、销售数据的统一管理，实时监控每个店铺的经营情况。
制造行业	1. 生产管理系统（MES）：帮助制造企业管理生产计划、工艺流程、产能分析，实时跟踪生产进度，确保生产效率。 2. 供应链管理系统：实现供应商管理、物料采购、库存管理、物流调度，优化供应链环节，确保物料及时供给。 3. 质量管理系统（QMS）：帮助企业管理产品质检流程、缺陷检测、合规性检查，生成质检报告，确保产品质量达标。 4. 设备管理系统：对生产设备进行实时监控，管理设备的运行状态、保养计划、维修记录，降低设备故障率。 5. 成本控制系统：跟踪每个生产环节的成本，帮助企业优化成本控制策略，提高生产效率和利润率。
房地产行业	1. 房产交易平台：提供房源发布、在线看房、价格评估、合同签署、交易撮合等功能，提升房产交易的便捷性与透明度。 2. 租赁管理平台：支持租赁合同签订、租金支付、租期管理、租金催缴提醒，提升房东和租客的租赁体验。 3. 物业管理系统：提供物业费缴纳、维修申请、社区公告、住户服务等功能，提升社区物业管理的智能化水平。 4. 楼盘销售管理系统：支持楼盘信息展示、销售数据统计、客户预约、意向跟踪等功能，帮助房产公司提升销售业绩。 5. 房地产金融平台：提供房贷申请、审批、还款管理、利率计算等功能，打通购房与金融服务链条。

2. 项目0到1

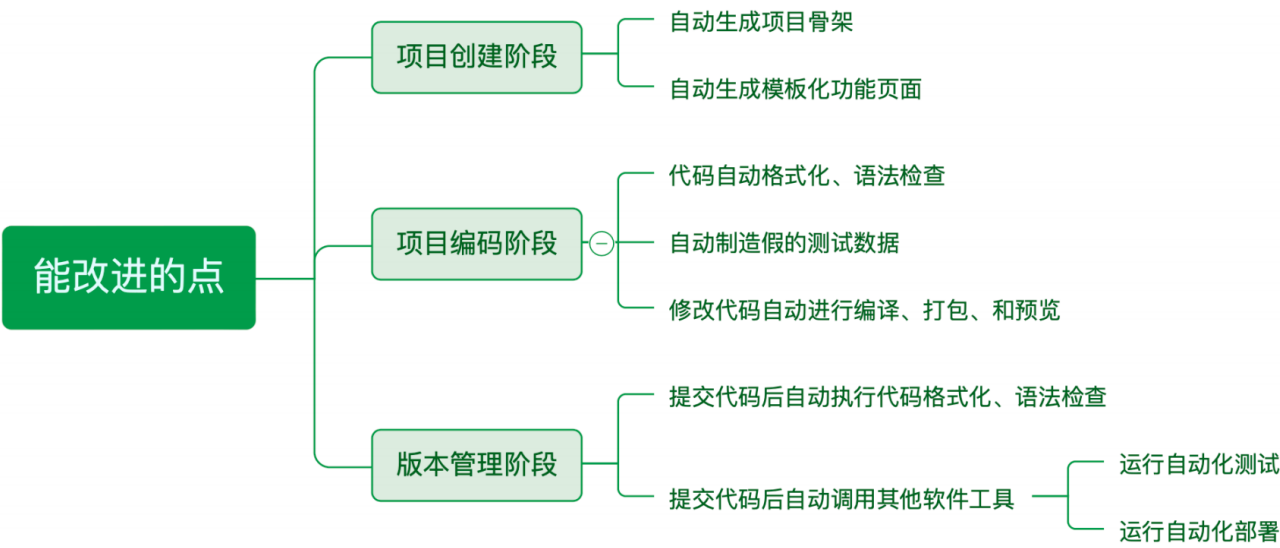
阶段	优化方案与深入总结	权威性与备注
1. 路由设计	1. 拆分路由： 一级路由包括 注册、登录、layout 布局、404 页面，确保页面结构清晰可维护； 二级路由为 layout 下的所有功能模块，减少路由嵌套，提升用户体验。 2. 权限设计： Vue 采用 RBAC 权限控制，通过用户角色动态加载路由和权限；	路由设计与权限管理 是确保前端应用结构合理与访问安全的核心，避免了不必要的权限泄露与性能损耗。

	React 通过封装高阶组件 AuthRouter ，判断 token 进行鉴权，未登录用户自动重定向至登录页。	
2. 状态设计	Vuex/Pinia：拆分模块化管理状态，将 mutations、actions 和 getters 按照业务逻辑独立拆分，确保状态管理清晰、可维护。 Redux：拆分 store、action、reducer，结合 Redux-thunk 处理异步状态，提升数据流的灵活性和可追踪性。	状态管理 是项目的核心之一，通过模块化拆分和异步状态管理，确保应用的灵活性与可扩展性。
3. axios 二次封装	请求拦截：在请求头中自动添加 token，确保每次请求都带有身份验证信息，防止未授权请求。 响应拦截：简化响应数据，统一处理 401 错误（未授权），自动清除用户信息和 token，并重定向到登录页面。 请求重连机制：为网络请求添加 重连机制，设置最大重连次数（如 3 次），在请求失败时自动重试，增强请求的健壮性。 - 设置统一 baseUrL 和请求超时机制，确保请求时长受控，避免长时间卡顿。	axios 重连机制 增强了网络请求的稳定性，特别适用于网络不稳定或 API 请求频繁的场景。
4. 区分环境与全局变量	环境区分：使用 .env.development 和 .env.production 文件定义不同环境变量，确保开发和生产环境中使用的变量各自独立，避免误用。 全局变量：通过 process.env 定义全局 baseUrL 和其他配置，保证项目能够根据不同环境加载正确的资源。 - 根据 环境变量 动态调整日志输出，开发环境打印详细调试日志，生产环境仅记录错误和关键信息，减少不必要的日志输出。	环境区分 和 全局变量管理 是前端项目中必不可少的部分，确保项目在不同环境中的行为一致且可控。
5. 主题定制与动态换肤	UI 框架主题定制 在 theme.less** 中定义自定义 CSS 变量，覆盖 Element UI、Ant Design 或 Vant 等组件库的默认样式，实现个性化品牌定制。 动态换肤：通过 CSS 变量实现动态换肤功能，用户可以在应用中自由切换主题，无需重新加载页面。	动态换肤 提升了用户体验和项目的灵活性，确保组件库主题能够适配多种场景。
6. 代码规范与风格统一	- 使用 ESLint 和 Prettier 进行代码风格检查和自动格式化，在代码提交前确保代码符合项目规范。 - 配置 Husky 和 lint-staged，确保每次提交代码前自动进行风格检查和格式化，防止低质量代码进入主分支。	代码规范化 是保证团队协作和代码可维护性的关键，避免了风格不一致和代码混乱问题。
7. Gitflow 工作流管理	Gitflow：使用 Gitflow 工作流管理分支，按照 master、develop、feature、release 和 hotfix 进行分支管理，确保开发和发布过程有条不紊。 - 在每个功能开发的最后阶段，合并到 develop 分支进行集成测试，确保在发布之前解决潜在的合并冲突和问题。	Gitflow 是大型项目标准的版本管理模式，确保多团队协作和项目迭代顺畅。
8. 跨域问题解决方案	CORS：在后端服务器配置 CORS 头，允许前端不同域的请求访问资源，确保跨域数据交互安全。 Nginx 反向代理：通过 Nginx 配置反向代理，将所有前端请求转发到同一域名下，规避浏览器的跨域限制。 - 在开发阶段通过 webpack-dev-server 配置跨域代理，简化本地开发和调试中的跨域问题。	跨域处理 是前后端分离项目的关键，通过反向代理和 CORS 配置，确保数据交互的安全性和兼容性。
9. 部署与自动化构建 (CI/CD)	手动部署：通过 npm run build 打包项目，将打包后的静态文件手动上传到服务器，适用于简单项目或小规模团队。 CI/CD 自动化部署：通过 Jenkins 或 GitHub Actions 配置持续集成与持续部署流程，每次提交代码后自动运行测试、打包，并将构建结果自动部署到服务器。 Docker 和 Kubernetes：在生产环境中采用容器化部署，使用 Docker 构建镜像，并通过 Kubernetes 实现集群管理和自动扩容，确保系统的高可用性和扩展性。	CI/CD 自动化部署 和 容器化部署 提升了项目发布的稳定性和效率，适合大型项目的持续交付需求。
10. 性能监控与错误捕获	- 使用 Sentry 或 LogRocket 监控项目的线上性能和错误，实时捕获用户侧的异常情况，便于及时修复。 - 结合 Google Lighthouse 进行性能监控，定期评估项目的加载速度、交互响应等性能指标，确保项目处于最佳状态。	性能监控 和 错误捕获 是生产环境中保证项目稳定运行的关键，确保问题能够在第一时间被发现和修复。

3. 前端工程化

作为项目负责人，经过多个项目迭代后 不断总结的前端工程化方案，工程化是一套解决问题的思想 涉及前端的 **开发、测试、部署** 为了 **降本** 和 **增效** 两个方面

使用 **Node.js** 构建脚手架工具，通过 **Commander.js** 或 **Inquirer.js** 实现命令行交互，并结合 **Yeoman** 或 **Plop.js** 进行项目模板生成，通过命令行参数快速生成标准化的项目结构、文件模板、依赖管理和构建工具，在短时间内搭建基础架构，确保项目的规范统一，还集成了 **Git Hooks** 和 **CI/CD** 流程，支持自动化构建、测试和部署，覆盖项目创建、运行、测试、提交和发布。



搭建基 建-公司 内部自定义脚手架 详细步骤	实现步骤	备注
1. 项目 初始化	1. 使用 npm init 或 yarn init 创建项目，定义 package.json，合理配置依赖、脚本。 2. 配置 .gitignore，并引入 Git 分支管理规范 Gitflow，确保分支管理规范，使用 master（生产）、develop（开发）、feature（功能）、release（发布）和 hotfix（修复）等分支。 3. 引入 Commitizen (git cz) 强制执行 Git 提交规范，规范化提交信息，自动生成 changelog。 4. 采用 Monorepo 和 pnpm workspaces 管理多组件库和插件的代码，确保各子包的依赖管理和版本控制高效。	Gitflow 和 Commitizen 是主流的版本管理和提交规范，pnpm workspaces 与 Monorepo 实现高效的多包管理。
2. 配置开 发环境	1. 使用 Webpack 或 Vite 构建工具，确保构建速度和编译效率，使用 Babel 处理现代 JS 特性。 2. 引入 ESLint 和 Prettier 进行代码检查和格式化，保证代码质量。 3. 利用 Husky 和 lint-staged 配置 Git 钩子，确保代码提交前经过自动化格式化和检查。 4. 引入 Storybook，为组件库提供独立的开发和测试环境，使开发者能高效开发、测试每个组件。	pnpm workspaces 可以高效管理多个组件库的依赖，Storybook 提供了可视化的组件开发和展示环境，提升开发体验。

3. 模块化设计	1. 使用 插件化架构 设计脚手架，允许开发者按需选择模块，最大化灵活性，确保功能模块独立。 2. 提供统一的 API 接口 和钩子机制，使插件之间的通信和组合更加便捷。 3. 使用 pnpm workspaces 配合 Monorepo 模式，将各功能模块和组件化封装，确保可维护性和可扩展性。 4. 确保每个模块设计为 低耦合、高内聚，使组件库和插件能独立开发、测试，并实现高效复用。	pnpm workspaces 高效管理模块依赖，提升复用性， Monorepo 提供了模块和组件的统一管理架构。
4. 项目模板创建	1. 在 templates 文件夹中准备多种技术栈模板（React、Vue、Node.js 等），并确保每个模板符合最佳实践。 2. 使用 Inquirer.js 实现交互式命令行选择框架、状态管理工具、测试框架等。 3. 提供 模板生成脚本，根据用户选择自动生成项目结构，自动安装依赖。	模板系统 和 用户交互设计 提升了脚手架的易用性，生成脚本提高了开发效率。
5. CI/CD	1. 使用 Jenkins 或 GitHub Actions 实现 CI/CD 流程，确保每次提交自动化测试、构建和发布。 2. 自动化打包、压缩、优化构建的静态资源，提升性能。 3. 将打包的结果发布到 npm 或私有仓库，确保用户通过命令行便捷安装：npm install -g <your-cli>。	CI/CD 流程 是现代开发标准，自动化测试、构建、发布提高了团队的开发效率。
6. 文档与测试	1. 提供详尽的 README 和 API 文档，通过 docsify 或 VuePress 自动生成文档页面。 2. 使用 Jest 或 Mocha 编写单元测试，确保各组件和功能模块的稳定性。 3. 通过 Storybook 展示每个组件的使用方法，并结合 pnpm workspaces 管理组件的依赖和测试环境，确保模块化开发和测试无缝进行。	Storybook 为组件库提供了独立开发和测试环境， pnpm workspaces 实现了多个包之间的依赖管理和共享测试环境。
7. 发布脚手架	1. 发布到 npm 或私有仓库，使用 SemVer 管理版本，并提供长期支持（LTS）。 2. 持续迭代、修复问题，并通过 changelog 记录版本更新，确保用户了解新版本内容。 3. 定期维护和更新脚手架，结合社区反馈，优化功能和用户体验。	SemVer 版本管理和 changelog 是项目长期维护的重要工具，确保用户了解更新内容。

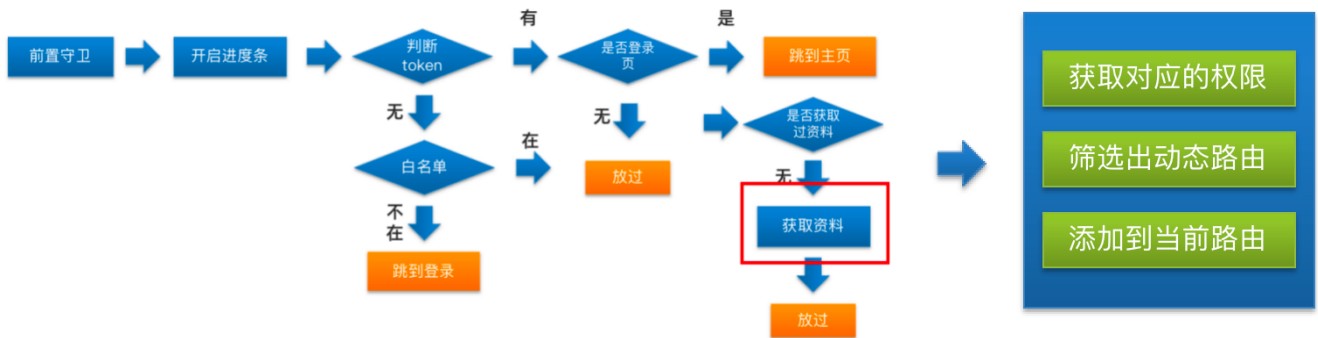
4. 项目亮点

1. RBAC权限

1. 权限应用-拆分静态路由-动态路由



2. 权限应用-根据用户权限添加动态路由



怎么匹配?

```
▼ roles: {,...}
menus: ["department", "role",
  0: "department"
  1: "role"
  2: "employee"
  3: "permission"
  4: "approval"
  5: "attendance"
  6: "salary"
  7: "social"]
```



```
export default {
  // 路由信息
  path: '/department',
  name: 'department', // 一级路由
  component: layout, // 一级路由
  children: [{
    path: '', // 二级路由地址为空时 表示 /department
    component: () => import('@/views/department'),
    name: 'department', // 可以用来跳转 也可以
    meta: {
      // 路由元信息 存储数据的
      icon: 'tree', // 图标
      title: '组织' // 标题
    }
  }]
}
```

- 权限拦截处筛选-添加路由-代码位置(src/permission.js)

```
import router from '@router'
import nprogress from 'nprogress'
import 'nprogress/nprogress.css'
import store from '@store'
import { asyncRoutes } from '@router'

/**
 *前置守卫
 *
 */
```

```

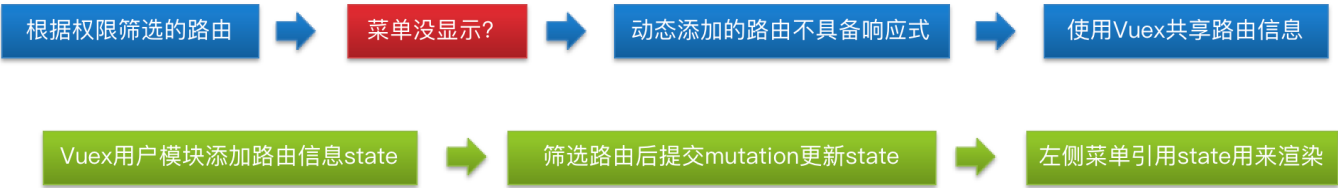
const whiteList = ['/login', '/404']
router.beforeEach(async(to, from, next) => {
  nprogress.start()
  if (store.getters.token) {
    // 存在token
    if (to.path === '/login') {
      // 跳转到主页
      next('/') // 中转到主页
      // next (地址) 并没有执行后置守卫
      nprogress.done()
    } else {
      // 判断是否获取过资料
      if (!store.getters.userId) {
        const { roles } = await store.dispatch('user/getUserInfo')
        // console.log(roles.menus) // 数组 不确定 可能是8个 1个 0个
        // console.log(asyncRoutes) // 数组 8个
        const filterRoutes = asyncRoutes.filter(item => {
          // return true/false
          return roles.menus.includes(item.name)
        }) // 筛选后的路由
        router.addRoutes([...filterRoutes, { path: '*', redirect: '/404', hidden: true
      }]) // 添加动态路由信息到路由表
        // router添加动态路由之后 需要转发一下
        next(to.path) // 目的是让路由拥有信息 router的已知缺陷
      } else {
        next() // 放过
      }
    }
  } else {
    // 没有token
    if (whiteList.includes(to.path)) {
      next()
    } else {
      next('/login') // 中转到登录页
      nprogress.done()
    }
  }
})

/** *
 * 后置守卫
 * **/
router.afterEach(() => {
  console.log('123')
  nprogress.done()
})

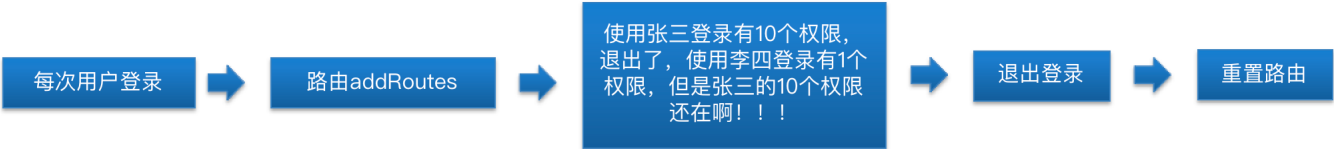
```

- 针对每个路由模块的配置文件添加name属性，和权限的数据进行对应。

3. 权限应用-根据权限显示左侧菜单



4. 权限应用-退出登录重置路由



- 退出登录时-重置路由-代码位置(src/store/modules/user.js)

```
import { resetRouter } from '@router'

// 退出登录的action
logout(context) {
  context.commit('removeToken') // 删除token
  context.commit('setUserInfo', {}) // 设置用户信息为空对象
  // 重置路由
  resetRouter()
}
```

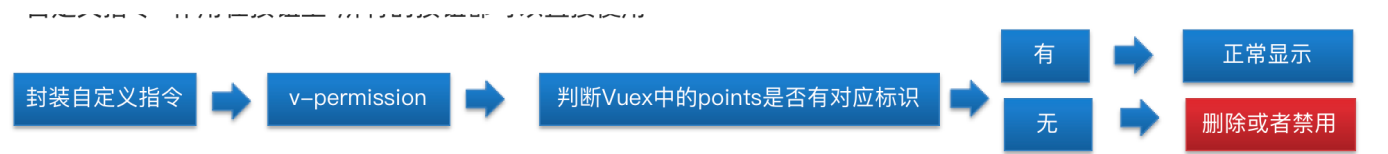
5. 权限应用-功能权限-按钮权限标识



注意：管理员账户不论分配不分配，都拥有所有的权限

6. 权限应用-自定义指令应用功能权限

- 自定义指令- 作用在按钮上-所有的按钮都可以直接使用



- 封装自定义指令-代码位置(src/main.js)

```
// 封装自定义指令 用来控制操作权
Vue.directive('permission', {
  // 会在指令作用的元素插入到页面完成以后触发
  inserted(el, binding) {
    // el 指令作用的元素的dom对象
    console.log(el)
    const points = store.state.user.userInfo?. roles?. points || [] // 当前用户信息的操作权
    if (!points.includes(binding.value)) {
      // 不存在就要删除或者禁用
      el.remove() // 删除元素
      // el.disabled = true
      // 线上的权限数据和线下的代码进行对应
    }
  }
})
```

- 应用自定义指令-代码位置(src/views/employee/index.vue)

```
<el-button v-permission="'add-employee'" size="mini" type="primary"
@click="$router.push('/employee/detail')">添加员工</el-button>
```

2. 大文件上传

1. 分片上传

通过 Blob.slice() 实现文件分片。Blob.slice() 可以根据指定的起始点和终点，将文件分割为一个较小的片段

1. 动态调整分片大小

- 默认分片大小为 5MB：这是一个比较常见的默认值，在多数情况下能够平衡上传速度与网络稳定性。5MB 的分片既不会占用过多的网络带宽，又能保持相对较快的上传速度。
- 根据网络状况调整分片大小：

- 使用 `navigator.connection.effectiveType` API 检测用户的网络状况，如果网络条件较差（如 3G 网络），可以将分片大小动态调整为 1MB；如果网络状况较好（如 WiFi 或 4G 网络），则将分片大小调整为 5MB 或 10MB，以提高上传效率。

- 动态调整策略：

- 网络状况：通过 `navigator.connection.effectiveType` 检测网络质量，并动态调整分片大小。
- 设备性能：考虑用户设备的内存和性能，通过 `navigator.deviceMemory` 判断设备性能，如果设备较低端，选择较小的分片大小（如 1MB）。
- 文件大小：对超大文件（如大于 1GB）可以选择较大的分片（如 10MB），而对较小文件可以使用 1MB 的分片，确保灵活应对各种场景。

2. 切片唯一值

- 文件 hash 生成：

- 使用 `spark-md5` 库来生成文件的唯一标识（hash 值），通过文件内容计算得到 MD5 值。
- 为了避免阻塞主线程，使用 `web-worker` 进行异步 hash 计算。
- 细化点：
 - 每次计算 hash 时，将文件按块分割，每个块计算部分 hash，最终合并结果。使用 `web-worker` 可以避免页面卡顿。

- 本地存储：

- 使用 `localStorage` 或 `IndexedDB` 存储文件的 hash 值和上传进度。`localStorage` 简单易用，`IndexedDB` 适合更大数据量。
- 细化点：
 - 存储结构：`localStorage.setItem('fileHash', hashValue)`，`localStorage.setItem('fileProgress', uploadedBytes)`。使用文件名、大小等特征作为键，确保唯一性。
 - 如果要支持多个文件同时上传，可以用 JSON 格式记录每个文件的 hash 和上传进度。

2. 断点续传

1. 上传进度管理

- 查询已上传字节数：

- 上传文件前，前端向服务器发送一个 `GET` 请求，携带文件的 hash，服务器返回该文件的已上传字节数。
- 细化点：
 - 通过 HTTP 请求向服务器发送 hash 值：`GET /upload-status?hash=<fileHash>`。
 - 服务器返回已上传的字节数（或分片号）：例如 `{ "uploadedBytes": 10485760 }`（已上传 10MB）。

- 记录上传进度：

- 上传过程中，每次成功上传后，前端更新本地存储中的进度值，以便中断后继续上传。
 - 细化点：
 - 每次上传一个片段（或部分文件后），调用 `localStorage.setItem('fileProgress', uploadedBytes)` 更新进度。
 - 进度可以通过浏览器刷新或页面重载后从 `localStorage` 或 `IndexedDB` 读取，继续上传。
-

2. 文件上传逻辑

- 分片上传与起始位置：
 - 上传时，前端从上次中断的字节位置继续上传（根据服务器返回的字节数）。每次上传时传递文件的 `hash` 和起始字节位置，确保从正确位置开始上传。
 - 细化点：
 - 每次上传通过 `POST` 请求，并携带文件的起始字节位置和 `hash` 值。
 - 例如：`POST /upload?hash=<fileHash>&startByte=<uploadedBytes>`，请求体中包含文件数据，服务器根据 `startByte` 继续接收数据。
 - 前端上传逻辑：
 1. 前端计算文件 `hash`。
 2. 向服务器发送 `GET` 请求，获取已上传的字节数。
 3. 根据服务器返回的字节数，从该字节处开始上传剩余文件数据。
 4. 每次上传一个块后更新本地存储进度。
 5. 上传完成后清理本地存储数据。
-

3. 上传完成后的清理机制

- 前端清理：
 - 文件上传完成后，前端清理 `localStorage` 或 `IndexedDB` 中的进度信息。
 - 细化点：
 - 上传完成后调用 `localStorage.removeItem('fileHash')` 和 `localStorage.removeItem('fileProgress')` 清理记录。
- 后端清理：
 - 服务器端完成文件合并后，删除临时存储的分片数据。
 - 细化点：
 - 在文件上传完成后，服务器应删除临时存储的文件片段，避免磁盘空间浪费。

3. 并发上传

- 任务队列：使用一个数组存储所有上传任务，通过设定并发数来限制同时进行的任务数量。
- 动态调度：每当一个任务完成后，启动下一个任务，直到所有任务处理完毕。

实现思路

1. 设定最大并发数：如同时允许最多 3-5 个分片并发上传。
2. 维护任务队列：通过 `Promise.allSettled` 配合异步函数 `async/await` 控制任务的执行和等待 来管理上传任务，确保超过并发限制的任务处于等待状态。
3. 上传完成后启动下一个任务：每当一个上传任务完成后，动态启动下一个分片上传任务，确保整个上传流程顺畅执行。
4. 失败重试机制
 - 失败处理：为了保证上传的成功率，可以为每个分片上传设置重试机制，避免由于网络问题或服务器临时性故障导致上传失败。

- 技术实现：

- 在上传任务中，捕获上传失败的错误，设置最多重试次数（如 3 次），每次失败后等待一段时间后重试。

4. 秒传

1. 前端发送 hash 给服务器

- 计算出文件 hash 后，前端将该 hash 值发送给服务器，向服务器询问该文件是否已经存在。
- 服务器接收该 hash 后，会检查数据库或文件存储系统，查看是否已经存在该文件。

2. 服务器校验文件是否存在

- 文件存在：如果服务器检测到该 hash 对应的文件已经存在，则直接返回成功响应，无需重新上传，前端立即完成上传流程。
- 文件不存在：如果服务器确认文件不存在，前端再开始正常的文件上传流程。

3. 双Token/无感刷新

1. 双Token机制

- 访问令牌（Access Token）：用于访问受保护资源，通常有较短的有效期（如30分钟）。
- 刷新令牌（Refresh Token）：用于获取新的访问令牌，通常有效期较长（如几个月或更长时间）。

1. 实现思路

- 登录和Token发放：
 - 用户登录后，服务器生成并返回访问令牌和刷新令牌。
 - 刷新令牌的存储位置和策略应考虑安全性（如使用HttpOnly和Secure标志的Cookies）。
- Token使用：
 - 前端在请求时将访问令牌放在Authorization头部，服务器验证后返回响应。
- Token过期处理：
 - 当访问令牌过期时，前端使用刷新令牌请求新的访问令牌。
 - 在请求拦截器中添加逻辑，检测访问令牌是否过期，并在必要时自动刷新。

双token代码实现

```
// api.js
import axios from 'axios';

// 创建Axios实例，用于配置请求和响应的全局设置
const instance = axios.create({
  baseURL: 'https://api.example.com', // 设置请求的基础URL
  timeout: 1000, // 设置请求超时时间（毫秒）
});

// 请求拦截器：在请求发出之前执行
instance.interceptors.request.use(config => {
  // 从sessionStorage中获取访问令牌
  const accessToken = sessionStorage.getItem('accessToken');
```

```

// 如果存在访问令牌，则将其添加到请求头部
if (accessToken) {
  config.headers.Authorization = `Bearer ${accessToken}`;
}

// 返回修改后的请求配置
return config;
}, error => {
  // 请求错误时返回拒绝的Promise
  return Promise.reject(error);
});

// 响应拦截器：在响应返回之后执行
instance.interceptors.response.use(response => {
  // 如果响应成功，则直接返回响应数据
  return response;
}, async error => {
  // 获取原始请求配置
  const originalRequest = error.config;

  // 如果响应状态码是401（未授权）并且原始请求未重试过
  // originalRequest._retry：标记请求是否已重试，防止无限循环重试。
  if (error.response.status === 401 && !originalRequest._retry) {
    originalRequest._retry = true; // 标记该请求为已重试

    // 从sessionStorage中获取刷新令牌
    const refreshToken = sessionStorage.getItem('refreshToken');

    if (refreshToken) {
      try {
        // 使用刷新令牌请求新的访问令牌
        const { data } = await axios.post('https://api.example.com/refresh', {
refreshToken });

        // 更新sessionStorage中的访问令牌
        sessionStorage.setItem('accessToken', data.accessToken);

        // 更新Axios默认请求头中的访问令牌
        instance.defaults.headers.Authorization = `Bearer ${data.accessToken}`;

        // 重新发起原始请求
        return instance(originalRequest);
      } catch (refreshError) {
        console.error('Refresh token failed:', refreshError);

        // 刷新令牌失败时处理，比如清除用户数据并重定向到登录页面
        sessionStorage.clear(); // 清除所有存储的用户数据
        window.location.href = '/login'; // 重定向到登录页面
      }
    }
  }
});

```

```

    }
  }

  // 如果请求出错，返回拒绝的Promise
  return Promise.reject(error);
});

export default instance;

```

- `originalRequest._retry`：标记请求是否已重试，防止无限循环重试。

2. 无感刷新实现

```

// auth.js
const REFRESH_INTERVAL = 5 * 60 * 1000; // 设置自动刷新Token的时间间隔（5分钟）

// 启动自动Token刷新
function startAutoRefresh() {
  // 设置定时器，定期执行Token刷新操作
  setInterval(async () => {
    // 从sessionStorage中获取访问令牌和刷新令牌
    const accessToken = sessionStorage.getItem('accessToken');
    const refreshToken = sessionStorage.getItem('refreshToken');

    // 如果存在访问令牌和刷新令牌
    if (accessToken && refreshToken) {
      try {
        // 使用刷新令牌请求新的访问令牌
        const { data } = await axios.post('https://api.example.com/refresh', {
          refreshToken
        });

        // 更新sessionStorage中的访问令牌
        sessionStorage.setItem('accessToken', data.accessToken);
        console.log('Token refreshed successfully'); // 打印Token刷新成功的消息
      } catch (error) {
        console.error('Token refresh failed:', error);

        // 刷新Token失败时处理，比如提示用户重新登录
        sessionStorage.clear(); // 清除存储的用户数据
        window.location.href = '/login'; // 重定向到登录页面
      }
    }
  }, REFRESH_INTERVAL); // 设置刷新间隔时间
}

export default startAutoRefresh;

```


3. 需要考虑的问题

- **并发请求**：token已过期，当多个请求同时需要刷新Token时，如何处理Token的竞争和一致性。
- **Token被篡改或伪造**：如何处理Token被篡改或伪造的情况，确保系统的安全性。刷新令牌泄露风险：若刷新令牌泄露，可能导致长期访问权限被滥用
- **刷新时机**：确定在用户操作不被打断的情况下何时进行无感刷新。

1.token过期后的并发请求

解决方案

1. 全局变量和Promise缓存

- **isRefreshing**：用于标记当前是否正在进行Token刷新操作。如果为 **true**，说明正在刷新Token。
- **failedRequestsQueue**：用于缓存等待Token刷新的请求。刷新完成后，所有请求将会从这个队列中处理。

2. 请求拦截器

- **config.headers.Authorization**：将存储在 **sessionStorage** 中的访问Token添加到每个请求的 Authorization头部，以确保请求能够通过身份验证。

3. 响应拦截器

- **处理401错误**：
 - **originalRequest._retry**：标记原始请求是否已经重试过。如果没有重试过，则尝试刷新Token。
 - **Token正在刷新**：
 - 如果Token正在刷新中，将当前请求加入 **failedRequestsQueue** 队列，并返回一个Promise，等待Token刷新完成。
 - **开始刷新Token**：
 - 如果Token没有正在刷新，则发起刷新Token的请求。
 - **成功刷新Token**：
 - 更新 **sessionStorage** 中的Token，并将新的Token设置到 **instance.defaults.headers.Authorization**。
 - 处理等待的请求队列，将刷新后的Token应用到队列中的请求。
 - 标记Token刷新完成，清空队列。
 - **刷新Token失败**：
 - 清空 **sessionStorage** 中的Token，重定向到登录页面以重新认证用户。

代码实现

```
// api.js
import axios from 'axios';

const instance = axios.create({
  baseURL: 'https://api.example.com', // API基础URL
  timeout: 1000, // 请求超时时间
});

// 全局变量和Promise缓存
let isRefreshing = false; // 标记Token是否正在刷新
```

```

let failedRequestsQueue = []; // 存储等待Token刷新的请求队列

// 请求拦截器
instance.interceptors.request.use(config => {
  // 从sessionStorage中获取访问Token
  const accessToken = sessionStorage.getItem('accessToken');
  if (accessToken) {
    // 将访问Token添加到请求头部
    config.headers.Authorization = `Bearer ${accessToken}`;
  }
  return config;
}, error => Promise.reject(error));

// 响应拦截器
instance.interceptors.response.use(response => response, error => {
  const originalRequest = error.config; // 原始请求配置

  // 当响应状态码是401（未授权），且原始请求未重试过
  if (error.response.status === 401 && !originalRequest._retry) {
    originalRequest._retry = true; // 标记原始请求已重试

    // 如果Token正在刷新，将请求加入队列并等待刷新完成
    if (isRefreshing) {
      return new Promise((resolve, reject) => {
        failedRequestsQueue.push({ resolve, reject });
      }).then(token => {
        // 刷新成功后，将新的Token添加到请求头部
        originalRequest.headers.Authorization = `Bearer ${token}`;
        return instance(originalRequest); // 重新发送原始请求
      }).catch(err => Promise.reject(err));
    }

    // 开始Token刷新
    isRefreshing = true;

    return new Promise((resolve, reject) => {
      // 发起刷新Token的请求
      axios.post('https://api.example.com/refresh')
        .then(({ data }) => {
          // 刷新Token成功，更新sessionStorage中的Token
          sessionStorage.setItem('accessToken', data.accessToken);
          instance.defaults.headers.Authorization = `Bearer ${data.accessToken}`;

          // 处理等待Token刷新的请求队列
          failedRequestsQueue.forEach(({ resolve }) => resolve(data.accessToken));
          failedRequestsQueue = []; // 清空队列

          isRefreshing = false; // 标记Token刷新完成
          originalRequest.headers.Authorization = `Bearer ${data.accessToken}`;
          resolve(instance(originalRequest)); // 重新发送原始请求
        });
    });
  }
});

```

```

    })
    .catch(err => {
      // 刷新Token失败, 处理错误
      isRefreshing = false;
      failedRequestsQueue.forEach(({ reject }) => reject(err));
      failedRequestsQueue = [];
      sessionStorage.clear(); // 清除所有Token
      window.location.href = '/login'; // 重定向到登录页面
      reject(err);
    });
  });
}

return Promise.reject(error);
});

export default instance;

```

2.Token被篡改或伪造

1. 生成安全的Token: 使用RS256算法生成具有足够熵值的Token。
2. 验证Token的有效性: 签名验证Token, 并设置合理的过期时间。
3. 保护Token传输: 使用HTTPS传输Token, 设置HttpOnly和Secure Cookies标志。
4. 定期检查和更新密钥: 定期轮换密钥, 使用密钥管理系统管理密钥。
5. 使用Token黑名单: 实现Token黑名单机制, 并在验证时检查黑名单。

1. 使用强加密算法生成Token

实施步骤:

- 选择加密算法: 推荐使用RS256（非对称加密），因为它使用公钥和私钥进行加密和解密，提高了安全性。
- 生成Token:
 - 生成私钥和公钥: 在生成Token之前，首先需要生成一对RSA密钥。
 - Token生成代码示例（Node.js）：

```

const jwt = require('jsonwebtoken');
const fs = require('fs');

const privateKey = fs.readFileSync('path/to/private.pem');
const publicKey = fs.readFileSync('path/to/public.pem');

// 生成访问Token
const generateAccessToken = (payload) => {
  return jwt.sign(payload, privateKey, { algorithm: 'RS256', expiresIn: '30m' });
};

// 生成刷新Token
const generateRefreshToken = (payload) => {

```

```
return jwt.sign(payload, privateKey, { algorithm: 'RS256', expiresIn: '30d' });
};
```

说明：

- 使用 **RS256** 加密算法生成Token。
- 使用 **fs** 读取私钥和公钥文件。

2. 验证Token的有效性

实施步骤：

- **Token签名验证**：使用公钥验证Token的签名和有效性。
- **设置合理的过期时间**：确保访问Token过期时间短，刷新Token过期时间长。
- **Token验证代码示例**（Node.js）：

```
const verifyToken = (token) => {
  try {
    return jwt.verify(token, publicKey, { algorithms: ['RS256'] });
  } catch (error) {
    throw new Error('Invalid token');
  }
};
```

说明：

- 使用 **jwt.verify** 验证Token的签名和有效期。
- 处理Token验证时的异常。

3. 保护Token传输

实施步骤：

- **启用HTTPS**：
 - 确保所有Token相关的请求都通过HTTPS进行。配置HTTPS通常需要获取SSL/TLS证书，并在服务器上进行配置。
- **设置HttpOnly和Secure标志**：
 - **Cookie设置代码示例**（Express.js）：

```
res.cookie('accessToken', accessToken, { httpOnly: true, secure: true, sameSite: 'Strict' });
res.cookie('refreshToken', refreshToken, { httpOnly: true, secure: true, sameSite: 'Strict' });
```

说明：

- **httpOnly**：防止JavaScript访问Cookie。
- **secure**：确保Cookie仅通过HTTPS传输。
- **sameSite**：防止CSRF攻击。

4. 定期检查和更新密钥

实施步骤：

- 密钥轮换：
 - 定期生成新的密钥对，并更新系统配置。
 - 在轮换密钥时，保持旧密钥以处理旧Token的验证。
- 使用密钥管理系统：
 - 使用AWS KMS、Azure Key Vault等密钥管理系统来存储和管理密钥。
 - 密钥管理代码示例（AWS KMS）：

```
const AWS = require('aws-sdk');
const kms = new AWS.KMS();

// 获取密钥
kms.getSecretValue({ SecretId: 'your-secret-id' }, (err, data) => {
  if (err) console.log(err, err.stack);
  else console.log(data.SecretString);
});
```

说明：

- 定期更新密钥并处理密钥过期。
- 使用密钥管理系统进行密钥存储和访问。

5. 使用Token黑名单

实施步骤：

- 建立Token黑名单：
 - 可以使用数据库（如Redis）或内存存储Token黑名单。
- Token黑名单管理代码示例（Redis）：

```
const redis = require('redis');
const client = redis.createClient();

// 添加Token到黑名单
const addTokenToBlacklist = (token, expiresIn) => {
  client.set(token, 'blacklisted', 'EX', expiresIn);
};

// 检查Token是否在黑名单
const isTokenBlacklisted = (token) => {
  return new Promise((resolve, reject) => {
    client.get(token, (err, result) => {
      if (err) reject(err);
      resolve(result === 'blacklisted');
    });
  });
};
```

说明：

- 使用Redis存储Token黑名单。
- 在Token验证时检查黑名单。

3.刷新时机：

目标：在用户活动期间，通过检测用户行为自动刷新Token，确保Token在用户操作时始终有效，同时减少对用户的干扰。

1. 设置用户行为监测
2. 触发Token刷新
3. 刷新Token时机选择

1. 设置用户行为监测

实现：

- 监测用户活动：监听用户的点击、输入、滚动等事件，以判断用户是否在活动。
- 设置活动监测时间间隔：记录用户最后一次活动的时间，用于判断是否需要刷新Token。

代码示例（JavaScript）：

```
let lastActivityTime = Date.now(); // 记录用户最后一次活动时间

// 更新最后活动时间
const updateLastActivityTime = () => {
  lastActivityTime = Date.now(); // 更新为当前时间
};

// 监听用户的点击事件
document.addEventListener('click', updateLastActivityTime);
// 监听用户的键盘按键事件
document.addEventListener('keydown', updateLastActivityTime);
// 监听用户的滚动事件
document.addEventListener('scroll', updateLastActivityTime);
```

说明：

- `updateLastActivityTime` 函数用于更新用户的最后活动时间。
- 通过 `addEventListener` 监听用户的点击、键盘按键和滚动事件，确保所有用户活动都被捕获。

2. 触发Token刷新

实现：

- 设定刷新阈值：当用户的最后活动时间距离当前时间超过一定阈值（如5分钟）时，触发Token刷新。
- 实现Token刷新：通过异步函数进行Token刷新操作，确保刷新过程在后台进行，不干扰用户操作。

代码示例（JavaScript）：

```

const REFRESH_THRESHOLD = 5 * 60 * 1000; // 刷新阈值：5分钟

// 检查用户活动状态并进行Token刷新
const checkAndRefreshTokenOnActivity = async () => {
  const currentTime = Date.now(); // 获取当前时间

  // 如果用户最近有活动且Token接近过期，执行刷新
  if (currentTime - lastActivityTime < REFRESH_THRESHOLD) {
    await refreshAccessToken(localStorage.getItem('refreshToken'));
  }
};

// 定时器每1分钟检查一次Token状态
setInterval(checkAndRefreshTokenOnActivity, 1 * 60 * 1000); // 每1分钟运行一次

```

说明：

- **checkAndRefreshTokenOnActivity** 函数检查用户的最后活动时间，如果用户最近有活动且Token即将过期，则触发刷新。
- **setInterval** 函数设置定时器，每1分钟检查一次Token状态。

3. 刷新Token时机选择

实现：

- **在用户操作期间**：通过定时器检查用户的活跃状态，在用户活跃时进行Token刷新。
- **减少干扰**：在用户活跃期间执行刷新操作，避免在用户操作过程中进行Token刷新。

代码示例 (JavaScript)：

```

// 刷新Token函数示例
const refreshAccessToken = async (refreshToken) => {
  try {
    // 向服务器发送Token刷新请求
    const response = await fetch('/api/refresh-token', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ refreshToken }) // 传递刷新Token
    });
    const data = await response.json(); // 解析服务器响应
    if (data.accessToken) {
      // 如果服务器返回新的访问Token，保存到本地存储
      localStorage.setItem('accessToken', data.accessToken);
    }
  } catch (error) {
    console.error('Token刷新错误:', error); // 处理刷新过程中可能出现的错误
  }
};

```

说明：

- `refreshAccessToken` 函数向服务器发送刷新Token请求，并处理响应。
- 如果服务器返回新的访问Token，将其存储到本地存储中。

4. 虚拟长列表

1. 实际应用场景

1. 电商后台管理系统

- **场景描述：**在电商后台中，管理员经常需要处理大量的商品数据和订单数据。通过虚拟长列表实现商品列表或订单列表的高效渲染，避免因数据量过大而导致页面卡顿。
- **具体落地方案：**
 1. 商品或订单数据存储在服务器端，前端通过分页或懒加载方式获取。
 2. 前端使用虚拟长列表进行渲染，同时结合筛选和排序功能。
 3. 优化滚动体验，确保大数据量下的渲染依然流畅。

2. 大数据展示系统

- **场景描述：**在大数据展示中，设备或日志信息通常有数万条，通过虚拟长列表可以高效展示这些数据，用户在查看时不会感到延迟或卡顿。
- **具体落地方案：**
 1. 将日志或设备信息分批次加载，避免一次性加载过多数据。
 2. 使用虚拟列表展示当前可见数据，并支持筛选和分类。
 3. 结合数据预加载机制，提前加载下一页数据，提升用户体验。

2. 实现细节

1. 定高列表的

实现步骤：

- **初始化：**设定每个列表项的固定高度 `itemHeight`，获取当前视口的高度 `viewHeight`。
- **计算可见元素：**通过当前的滚动位置 `scrollTop` 计算可视范围内的起始和结束元素索引：

```
const startIndex = Math.floor(scrollTop / itemHeight);
const endIndex = Math.floor((scrollTop + viewHeight) / itemHeight);
```

- **仅渲染可见元素：**只将 `startIndex` 到 `endIndex` 范围内的数据渲染到页面。

2. 不定高

采用高度缓存机制，结合滚动条的动态调整来实现。

- **记录高度：**初次渲染时记录每个元素的高度，并根据滚动位置动态调整可视区域的高度。
- **滚动条同步调整：**动态计算整个列表的总高度，并根据已渲染元素的高度调整滚动条位置。

可以使用如 `react-virtualized`、`vue-virtual-scroll-list` 等库来实现这种功能，而不必从零开始手动开发。

3. 性能优化

- **节流与防抖**：对于滚动事件，必须进行节流或防抖优化，避免频繁触发渲染，导致页面卡顿。实际项目中可以使用 `lodash` 或 `underscore` 中的 `throttle` 或 `debounce`，或者 `requestAnimationFrame` 等技术手段优化滚动性能，确保大数据量的流畅体验。
- **缓冲区机制**：不仅仅渲染当前可见区域，还可以向上或向下多渲染几个元素作为缓冲区，避免滚动过程中出现闪屏或白屏。缓冲区大小可以根据实际需要调整，比如渲染 5 到 10 个额外元素。

5. 组件库封装

组件库架构能力维度	实现思路
1. Monorepo 管理模式	1. Monorepo 优势：通过 Monorepo 管理多个包，优化了 依赖共享、代码复用、统一配置 的开发体验，解决了多仓库管理带来的维护复杂性。可根据项目规模灵活选择 Lerna（轻量场景）或 Nx（适合大型项目），尤其在 Nx 的 任务缓存 和 分布式任务执行 下提升构建速度。 2. Monorepo 实践中的挑战与解决方案：解决跨包依赖复杂性，采用集中管理与版本发布策略，防止版本号紊乱。自动化构建和发布脚本与 CI/CD 集成，实现跨团队合作。
2. pnpm workspace 使用	1. 高效依赖管理与缓存机制：相比 npm 和 yarn，pnpm workspace 提供了更高效的依赖管理，支持软链接机制，并解决了模块间 Hoisting 冲突，大幅减少磁盘占用。特别是在 Monorepo 场景下，pnpm workspace 通过共享依赖优化了构建时间和空间占用。 2. 一致性与性能保障：确保所有子项目依赖版本统一，避免多个版本冲突问题，尤其是在微前端架构中，能确保各模块无缝集成与运行。
3. Changelog 自动生成	1. 标准化版本发布与变更记录：通过 semantic-release 实现 Git 提交自动生成 changelog，确保版本发布的规范性和一致性，支持多包的自动化变更日志管理。根据 Conventional Commits 标准化提交记录，自动生成详细的 版本发布说明，提高团队协作效率。 2. 复杂 Monorepo 场景的 Changelog 管理：在组件库中，每个子包都有独立的更新日志和 changelog 管理，清晰跟踪每个包的变更历史。
4. 版本管理与发布策略	1. 独立版本 vs 统一版本：根据组件库间的依赖关系，选择 独立版本 或 统一版本 策略。在依赖较少的情况下，独立版本管理更灵活；而强依赖组件库则需要保持统一版本号，确保发布流程的简洁性。 2. 自动化发布与语义化版本控制：通过 CI/CD 集成 实现自动化发布，在主分支合并时自动触发测试、构建和发布。采用 Semantic Versioning (Semver) 规范，明确区分修复、功能添加与破坏性更新。
5. 组件库封装设计	1. 按需加载与 Tree Shaking：确保组件库支持 按需加载，避免全量引入组件，使用 babel-plugin-import 配置按需加载，确保与 Webpack 或 Rollup 的 Tree Shaking 配合使用，移除未使用代码，提升构建性能。 2. TypeScript 类型支持：提供全局和按需加载的 TypeScript 类型声明，确保开发者获得完备的类型推断支持，提升组件库的可维护性与开发体验。
6. 文档与测试	1. 文档自动化与展示平台：通过 Storybook 或 Docusaurus 生成高度交互的组件文档，展示各组件的状态、交互效果，提升组件库的可用性和开发体验。 2. 测试覆盖率与自动化测试：使用 Jest 进行单元测试，确保组件功能的完整性；使用 Cypress 进行端到端测试，保证在实际业务中的可靠性。使用 codecov 或 coveralls 监控测试覆盖率，确保组件库的高质量输出。
7. 版本兼容与升级策略	1. 严格遵循 Semver 规范：采用 Semantic Versioning，确保版本号清晰区分功能更新、修复和重大变更。对于 破坏性更新，提供详细的迁移指南，确保用户可以平滑升级。 2. 向后兼容策略：为常用 API 保持 向后兼容，避免小版本更新时对用户造成困扰。提供渐进式迁移策略，帮助用户快速适应新版本变化。

8. 组件库的架构演进	1. 从基础组件到业务组件的扩展：组件库的架构应支持从基础 UI 组件扩展到业务组件，保证各模块的独立性和复用性。业务组件通过组合基础组件，提供更高层次的封装。 2. 微前端与分布式开发支持：组件库设计要适应 微前端架构，支持跨项目共享组件，结合 Module Federation 或 Qiankun 进行分布式组件管理，支持多个项目共同使用组件库而不产生版本冲突。
9. 多端适配与性能优化	1. 响应式设计 with 跨端支持：组件库需要支持响应式设计，确保在不同屏幕尺寸和设备上的一致性表现。使用 CSS 变量 或 媒体查询，实现灵活的样式适配，支持 PC、移动端和大屏应用场景。 2. 性能优化与动态加载：通过 lazy loading 优化性能，按需加载组件；对于大数据量展示，使用 虚拟滚动 (Virtual Scrolling) 优化渲染性能。确保组件库在高性能场景下依然流畅运行。
10. 国际化与本地化支持	1. 国际化 (i18n) 策略：组件库要支持多语言配置，使用 vue-i18n 或 react-intl 等插件，实现文案的动态加载与切换，确保组件库能够在全球化项目中使用。 2. 多地区本地化支持：组件库需支持 本地化 的时间、货币、单位等格式，通过配置实现自动化处理，确保业务在不同地区和文化下的正确使用。

6. 复杂万能表单组件

万能表单组件维度	全面优化与深入总结
1. 需求背景	1. 业务场景与挑战：传统静态表单在应对动态复杂业务时，难以满足需求，尤其在 后台管理系统、大型 ToB 项目 中，表单字段复杂、动态配置多、联动逻辑频繁等场景对表单系统的灵活性提出了极高要求。 2. 高可复用性与低维护成本：该表单组件需要通过 配置驱动 实现高复用性，以 最小的开发成本 应对各种复杂的业务场景，避免手动编码的繁琐和维护难度。
2. 技术选型	1. React-Hook-Form + useReducer：react-hook-form 是轻量且高性能的表单管理库，特别适合复杂多表单项场景，减少 不必要的渲染，确保性能的最优。通过 useReducer 处理复杂表单状态，结合 useContext 实现跨组件的 状态共享与通信。 2. 递归渲染与动态组件：递归生成表单项，结合 React.memo 进行深度性能优化，避免表单联动时的 重复渲染，确保性能瓶颈得到有效控制。 3. TypeScript 全覆盖：使用 TypeScript 为表单系统提供类型支持，确保在复杂项目中的稳定性和开发体验，减少运行时错误。
3. 配置驱动表单实现方案	1. 动态配置与结构定义：通过 JSON 或 JS 对象 定义表单结构、字段类型、验证规则、联动逻辑等，避免手动硬编码。配置文件包含核心表单信息：字段类型（如 input、select）、验证规则（必填、正则匹配）、联动逻辑（dependencies）、数据源（dataSource）等，确保 灵活性与可扩展性。 2. 递归生成与联动机制：使用递归函数处理复杂嵌套表单，并结合 useEffect 实现字段之间的动态联动，确保表单能够 实时响应数据变化。 3. 动态验证与异步加载：通过 react-hook-form 提供的 setValue 和 trigger，动态调整校验规则，支持异步数据源加载和表单回填。
4. 复杂状态管理与联动机制	1. useReducer 管理复杂表单状态：useReducer 在处理复杂状态变化时，能很好地组织表单状态，清晰管理多层嵌套和字段联动。通过 dispatch 实现状态变更，确保表单值、验证信息等保持同步。 2. 高级联动与依赖处理：表单字段之间的联动通过 dependencies 机制来实现，允许根据某个字段的变化动态调整其他字段的显示状态、验证规则和数据源。通过 useEffect 监听字段变化，确保联动逻辑的动态更新。 3. 高性能动态校验：根据用户输入实时触发校验，结合防抖机制，减少不必要的多次校验，确保性能和用户体验的平衡。
	1. 局部渲染与防抖机制：针对高频次输入场景，使用 lodash.debounce 实现防抖处理，避免表单频繁触发校验逻辑，确保仅在必要时更新表单状态，提升整体性能。

5. 高性能与异步处理优化	2. 惰性加载与虚拟滚动：在大规模表单中，采用 按需渲染 和 虚拟滚动 技术，按用户滚动加载表单项，避免一次性渲染全部表单项的性能瓶颈。 3. 异步数据源支持与回填：支持异步加载 select、checkbox 等数据源，并根据后端返回值进行表单数据的自动回填，确保复杂场景下的表单使用体验。对于异步回填，可以使用 Promise.all 同步加载多个表单项的数据源，提升加载速度。
6. 可扩展性设计	1. 自定义表单元素与动态注册机制：组件库支持开发者传入自定义的表单组件，如 DatePicker 或 Rich Text Editor，通过 开放接口 允许开发者注册并扩展自定义组件。可通过 React Context 共享上下文，实现跨组件通信。 2. 国际化支持与多语言：表单的所有文案、提示信息支持 国际化（i18n），通过配置文件传递多语言内容。结合 i18next 等国际化插件，提供多语言切换功能，确保表单在全球化项目中适用。 3. 布局灵活性与响应式支持：支持自定义布局，包括单列、多列、网格和响应式布局，满足多终端适配需求。高级用户可通过 拖拽式布局编辑 实时调整表单结构。
7. 文档与自动化测试支持	1. Storybook 与 Docusaurus 文档生成：使用 Storybook 生成交互式文档，展示各个表单元素的使用场景、状态和交互效果，结合 Docusaurus 自动生成文档网站，帮助开发者快速理解组件的用法。 2. 单元测试与端到端测试：通过 Jest 和 Cypress 进行全面的单元测试和端到端测试，确保组件库的稳定性与可靠性。使用 mock 数据 模拟表单的实际使用场景，确保复杂交互逻辑能够得到充分测试。 3. 测试覆盖率与持续集成：结合 codecov 或 coveralls 监控测试覆盖率，通过 CI/CD 集成持续测试与自动化发布，确保组件库质量的持续提升。
8. 版本管理与发布策略	1. Semver 版本管理与向后兼容：组件库遵循 Semver 版本规范，确保在发布破坏性更新时标注为大版本，并为用户提供详细的 迁移指南。在功能迭代中，保持常用 API 的 向后兼容性，避免小版本更新对现有使用者造成影响。 2. 自动化版本发布与 Changelog 管理：结合 semantic-release 实现版本的自动化发布，并通过提交记录生成详细的 Changelog，帮助用户理解每次更新的改动和重要性。结合 CI/CD 系统，确保版本发布流程的自动化和规范化，提高团队协作效率与版本管理的透明度。
9. 微前端与分布式开发支持	1. 微前端架构支持与跨项目共享：表单组件库支持 微前端架构，通过 Module Federation 或 Qiankun 实现跨项目组件共享，支持多个业务线共用表单库，减少重复开发和维护成本。 2. 分布式开发与跨团队协作：组件库设计应适应 分布式团队开发模式，通过 Monorepo 管理多个组件包，并结合 pnpm workspace 管理依赖。通过 Lerna 或 Nx 优化包之间的构建、测试与发布流程，确保跨团队开发时的协作高效性与代码一致性。
10. 性能监控与优化	1. 前端性能监控与异常处理：结合 Sentry 或 LogRocket 实现表单的线上性能监控，及时捕捉用户侧的异常和表单交互问题，确保用户体验的稳定性。 2. 性能优化策略

7. 单点登录

实现方式总结：

- Token 验证**：采用 **JWT** 作为身份凭证，通过 **HMAC** 或 **RSA** 签名确保安全性，并存储在客户端的 **localStorage** 中。
- 多因子认证**：为高安全性场景，如金融或医疗系统，引入多因子认证机制，结合短信验证码或动态密码，增强登录安全性。
- 性能优化**：使用 **Redis** 缓存 **Token** 数据，结合 **Nginx** 等负载均衡器，确保系统在高并发下的响应速度和稳定性。
- 用户体验优化**：通过自动登录机制和 **localStorage** 存储，提供用户无缝的登录体验，减少重复登录操作。

类别	核心知识点	应用场景与最优方案	实现方式
			1. 在中心认证服务器上搭建身份验证系统（例如

项目背景	多系统整合与身份认证	在大型企业中，通常存在多个业务系统，例如：内部管理系统（如 ERP、CRM）、员工考勤系统、文档管理系统等。SSO 能够简化用户身份认证，实现一次登录访问多个系统的功能，提升工作效率。	Keycloak 或 Auth0）。 2. 各业务系统重定向到认证服务器进行登录验证。 3. 登录后认证服务器生成 JWT Token 并返回。 4. 客户端和服务端在请求时通过 Authorization 头携带 Token。
协议选择	OAuth 2.0 + OpenID Connect	通过 OAuth 2.0 和 OpenID Connect 实现身份认证与授权，适用于大多数 Web 应用场景。OAuth 2.0 负责授权，OpenID Connect 负责身份验证，实现用户在多个应用中的单点登录。	使用 OAuth 2.0 的授权码模式： 1. 配置 OAuth 2.0 认证服务器（如 Keycloak）。 2. 前端调用 /authorize 获取授权码。 3. 后端通过 /token 端点获取 Access Token。 4. 用 JWT 存储用户的身份信息。
跨系统登录	Access Token + Refresh Token	不同系统如 admin.company.com 和 docs.company.com 使用 JWT 进行身份认证。通过 Access Token 保持登录状态，Refresh Token 负责续期，避免频繁重新登录。	1. 登录成功后，生成 Access Token 和 Refresh Token。 2. Token 存储在客户端的 localStorage 或 sessionStorage 中。 3. 使用 Refresh Token 自动续期，例如在用户发起请求时检测 Token 是否过期，如果过期则发送刷新请求获取新的 Access Token。
跨域问题处理	CORS 配置与 iframe + postMessage	在跨域 SSO 过程中，可以通过 CORS 允许不同子域名共享 Cookie 或 Token。对于完全不同域名的系统，可以使用 iframe 结合 postMessage 实现跨域会话传递。	1. 在 SSO 服务器上设置 CORS 允许跨域访问。 2. 在各个子系统中嵌入指向 SSO 服务器的 iframe ，用户登录后通过 postMessage 向父窗口传递 Token。 3. 使用 window.addEventListener('message', callback) 监听登录结果并获取 Token。
安全性考虑	CSRF 防护、XSS 防护与 Token 签名	防止 CSRF 和 XSS 攻击。设置 SameSite Cookie 属性、防止 Token 被盗用，并通过 HTTPS 传输数据，确保通信安全。	1. 在 SSO 服务器上设置 SameSite 属性防止 CSRF 攻击。 2. 使用 JWT 的 HMAC 或 RSA 签名验证 Token 完整性。 3. 启用 HTTPS，强制所有请求通过 HTTPS 进行。 4. 使用 helmet 等中间件设置 CSP，防止 XSS 攻击。
身份验证与会话管理	短期会话与长期登录	用户可以选择短期登录或长期保持登录状态。短期会话设置较短的 Token 过期时间，长期登录使用 Refresh Token 实现无缝登录。	1. 在用户登录时，生成 Access Token（过期时间较短）和 Refresh Token（过期时间较长）。 2. 定期检查 Token 是否过期，如果过期则自动使用 Refresh Token 续期。 3. 客户端可以在登录成功后提供“保持登录”选项，将 Refresh Token 保存在 localStorage 。
多系统会话同步	中心化注销与会话同步	用户在某一系统中注销时，必须确保其他关联系统的会话同步注销。通过中心化的 SSO 服务器管理所有系统的会话状态，实现统一的注销。	1. 在 SSO 服务器实现注销接口 /logout ，通知所有子系统同步注销。 2. 在各系统嵌入 iframe ，当用户注销时，调用每个系统的注销接口。 3. 后端通过 broadcast-channel 或 redis-pub-sub 实现集群系统间的同步会话注销。
移动端 SSO 实现	WebView + 深度链接 (Deep Link)	在移动端应用中，通过 WebView 实现单点登录，并通过 Deep Link 在不同 App 之间共享登录信息。	1. 使用 WebView 嵌入 SSO 登录页面。 2. 登录成功后，通过 Deep Link 传递 Token 给其他 App。 3. 实现 App 之间的身份验证信息共享，例如通过 React Native 的 Linking API 实现 Deep Link 跳转并传递 Token。
性能优化	Token 缓存、Redis 缓存与负载均衡	在高并发场景下，使用 Redis 缓存 Token 信息，减少数据库查询，并通过负载均衡器分发认证请求，提升系统的性能和稳定性。	1. 使用 Redis 缓存 Token 及用户会话数据。 2. 配置负载均衡器（如 Nginx）分发 SSO 请求，保证系统的可用性。 3. 结合 JWT 实现无状态验证，减少服务器端的会话管理开销。
SSO 与微服务集成	API Gateway + Token 传递	在微服务架构中，通过 API Gateway 作为身份验证入口，各微服务通过 JWT 共享用户身份信息，确保安全的跨服务访问。	1. 在 API Gateway 中设置身份验证入口，所有请求先通过 Gateway 验证身份。 2. 使用 JWT 存储用户信息，JWT 通过 Gateway 传递给各微服务。 3. 在微服务中验证 JWT 的签名和有效性，确保用户合法访问各服务。
第三方登录与集成	OAuth 2.0 第三方登录集成	项目中集成 Google、Facebook、微信等第三方登录，通过 OAuth 2.0 授权码模式，用户登录后获取第三方 Token 并存储在 SSO 系统中。	1. 配置第三方登录，如 Google、Facebook、微信 OAuth 应用。 2. 前端跳转到第三方认证平台，用户授权后返回 Token。 3. 将第三方返回的 Token 存储在中心认证服务器中，用户后续访问系统时携带该 Token。

多因子认证	MFA（多因子认证）增强安全性	为了增强安全性，结合多因子认证（MFA），用户登录后还需进行额外的身份验证（如短信验证码或手机认证器）。	1. 配置 MFA 机制，例如短信验证码或 Google Authenticator。 2. 用户登录后，通过中心认证服务器发送短信验证码或生成动态密码。 3. 验证通过后，用户才可继续访问系统。 4. MFA 配置可以结合 Auth0 或 Keycloak 等身份验证服务进行。
SSO 部署与扩展	云服务集成与高可用部署	SSO 系统可以通过云服务（如 Auth0、AWS Cognito）部署，并结合 Docker 和 Kubernetes 实现容器化部署，确保系统的高可用性和可扩展性。	1. 通过 Auth0、AWS Cognito 配置中心认证服务，使用云服务的高可用特性。 2. 使用 Docker 容器化部署 SSO 服务器，结合 Kubernetes 实现自动扩容。 3. 配置负载均衡器分发流量，确保系统在高并发下的性能。
用户体验优化	自动登录、无缝切换与登录态保持	通过 Token 的长期有效性和自动登录，用户在不同系统间无缝切换。使用 JavaScript 自动检测用户登录状态并跳转到 SSO 页面，提升用户体验。	1. 在前端使用 localStorage 存储 Token，检测是否存在有效的 Token。 2. 若 Token 存在且未过期，自动跳转至应用主页面。 3. 在前端页面中使用 JavaScript 自动判断登录状态并跳转，提供无缝登录体验。

3. 项目性能衡量指标

1. 后端性能监控指标

TP50、TP90、TP99、TP999 特别用于 衡量请求的响应时间，这里需要后端做优化

指标	应用场景	示例	推荐值
TP50	判断系统的典型性能，但不够全面，不能反映长尾请求情况	50%的请求响应时间 ≤ 100ms	≤ 100ms
TP90	评估系统在较高负载下的性能，识别较慢请求	90%的请求响应时间 ≤ 200ms	≤ 200ms
TP99	监控系统性能的极端情况，确保极少数请求不会过慢	99%的请求响应时间 ≤ 500ms	≤ 500ms
TP999	评估系统最差情况下的表现，适用于高可用性要求高的系统	99.9%的请求响应时间 ≤ 1000ms	≤ 1000ms

2. 前端性能监控指标

Web-vitals 官方标准

指标	作用	优化方向	优秀标准	需改进标准	差标准
FCP	首个内容绘制时间，反映页面加载进度	减少阻塞资源，优化CSS和JS	1.8秒以内	1.8-3秒	超过3秒
LCP	最大内容绘制时间，衡量页面主体内容加载速度	优化图片、字体加载，懒加载	2.5秒以内	2.5-4秒	超过4秒
FID	首次输入延迟，衡量交互响应速度	减少JS执行时间，优化任务调度	100毫秒以内	100-300毫秒	超过300毫秒
CLS	累积布局偏移，衡量视觉稳定性	设置明确尺寸，避免动态加载	0.1以内	0.1-0.25	超过0.25
TTFB	首字节时间，反映服务器响应速度	优化服务器响应，使用CDN和缓存	500毫秒以内	500毫秒到1秒	超过1秒

4. 性能优化

1. 浏览器在一帧中都做了什么

浏览器中的画面都是一帧一帧渲染出来的，通常来说渲染的帧率与设备的刷新率保持一致。一般情况下，设备的屏幕刷新率为1秒钟60次，一次16.7ms(毫秒) 当**每一帧绘制毫秒低于16.7ms时，页面渲染时流畅的**；当大于16.7毫秒时，会出现一定程度的卡顿现象。

每一帧的渲染流程主要包括：事件处理 → JavaScript 执行 → 样式计算 → 布局 → 分层 → 绘制 → 合成 → 显示。所有这些操作必须在 16.7 毫秒内完成，才能保证页面流畅。

2. Google性能优化工具区别

优化工具	主要功能	数据来源	评估维度	应用场景	适用人群
Lighthouse	综合网页性能分析 ，包含性能、可访问性、最佳实践、SEO等	模拟环境 （本地测试，用户可定制不同网络条件和设备）	性能 、可访问性、最佳实践、SEO、PWA	全面检测 网页性能问题，适用于开发过程中的网页优化	开发者、性能优化工程师
Performance	Chrome DevTools 中内置的网页性能分析工具	模拟和实际用户数据 （包括时间线分析和JavaScript堆栈跟踪）	FPS （帧率）、加载时间、内存占用、CPU利用率等	分析网页在不同操作中的详细性能，包括 动画流畅度、加载速度 、内存和CPU使用情况	前端开发者、性能调优专家
PageSpeed Insights	测量网页的 加载性能并提供优化建议 ，重点关注实际用户体验	实际用户数据 （来自Chrome 用户体验报告）	FCP 、LCP、CLS、FID、TBT等核心 Web 生命体征	快速评估网页在 实际用户设备上的性能表现 ，适用于 SEO优化 和提升用户体验	网站管理员、SEO优化人员

3. 性能优化大全

1.Lighthouse

1.使用方式

使用方式	步骤	备注
Chrome DevTools	1. 打开 Chrome 浏览器并导航至要测试的网页 2. 按下 F12 或右键点击页面，选择“检查”打开开发者工具 3. 切换到 “Lighthouse” 标签 4. 选择要测试的项目（性能、可访问性、SEO 等） 5. 点击“生成报告”按钮，等待 Lighthouse 生成结果	适用于快速分析单个网页性能
命令行 (CLI)	1. 安装 Node.js 和 npm 2. 运行 npm install -g lighthouse 安装 Lighthouse 3. 使用命令 lighthouse <URL> 运行测试 4. 查看生成的 HTML 或 JSON 格式报告	适用于自动化测试和集成 CI/CD 工具
PageSpeed Insights	1. 打开 PageSpeed Insights 2. 输入要测试的 URL 3. 点击“分析”按钮 4. 查看 Lighthouse 生成的报告	适用于在线测试，生成实际用户环境下的性能数据
npm 包集成	1. 安装 Lighthouse npm 包 2. 在项目中通过脚本集成 Lighthouse 进行性能测试 3. 运行集成的测试脚本，查看结果	适用于项目中集成性能分析工具

2.核心优化

评估维度	核心优化建议
性能	减少渲染阻塞资源（CSS、JS），延迟加载非必要资源，启用压缩和缓存 优化图片和视频（WebP、延迟加载） 减少主线程工作量，使用 CDN，预加载关键资源
可访问性	提供 替代文本、页面标题，确保 足够的对比度 使用 语义化 HTML 标签，确保表单、动态内容和键盘导航可访问
最佳实践	使用 HTTPS，避免 eval()，更新依赖库 减少混合内容，确保图片有 明确尺寸，避免不安全的依赖
SEO	提供 描述性标题和 meta 描述，使用正确的 HTTP 状态码 结构化页面内容，优化加载速度，确保移动端友好
PWA	实现 离线功能（Service Worker），启用 响应式设计，使用 HTTPS ，配置 Web App Manifest

2. Performance

1. 使用方式

使用方式	步骤	备注
Chrome DevTools	1. 打开 Chrome 浏览器并导航至要测试的网页 2. 按下 F12 或右键点击页面，选择“检查”打开开发者工具 3. 切换到 Performance 标签 4. 点击“录制”按钮开始捕获性能数据 5. 完成测试操作后，点击“停止”按钮，查看性能报告	实时查看 页面的性能瓶颈，如 CPU、内存使用率、帧率等
命令行 (CLI)	1. 安装 Node.js 和 npm 2. 通过 npm 安装 Chrome Headless 浏览器 3. 使用 puppeteer 或类似工具运行页面性能测试 4. 导出性能数据，查看帧率、加载时间等指标	适用于 自动化性能测试 ，可与 CI/CD 集成
远程设备测试	1. 将移动设备连接到电脑 2. 在 Chrome DevTools 中，点击 设备 图标，选择连接的设备 3. 在移动设备上操作网页，同时在 DevTools 的 Performance 标签中查看性能数据	适用于测试 移动设备的性能表现

2. 核心优化

优化维度	核心优化建议	备注
减少渲染阻塞	1. 减少渲染阻塞资源 (如 JavaScript 和 CSS) 2. 使用 异步加载 或 延迟加载 非关键资源	确保页面的关键渲染路径尽可能短
优化 JavaScript 执行	1. 减少 JavaScript 体积 和复杂度 2. 使用 代码分割 和 Tree Shaking 来移除无用代码	减少主线程占用，提升页面响应速度
提升图片和多媒体加载	1. 压缩图片，使用现代图片格式 (如 WebP) 2. 延迟加载 非关键图片和视频	减少图片加载时间，提升页面加载性能
减少重排和重绘	1. 使用 CSS 动画 代替 JavaScript 动画 2. 避免频繁的 DOM 操作	减少页面重排次数，优化帧率
使用浏览器缓存	1. 启用浏览器缓存，缓存静态资源 2. 设置合适的缓存头，减少重复加载	提升重复访问时的加载速度
减少请求数	1. 合并请求 或使用 HTTP/2 2. 减少 HTTP 请求数量	减少网络延迟，提升页面加载性能
优化首次内容渲染	1. 优化首屏渲染，减少首屏加载资源 2. 使用 CDN 加速资源加载	提升用户的首次加载体验
使用 GPU 加速	对性能密集型操作，如动画和 3D 渲染，使用 GPU 加速	减少 CPU 占用，提高渲染效率

3.webpack打包优化

优化维度	核心优化建议	备注
代码分割	1. 使用 SplitChunksPlugin 将依赖分割成多个 bundle, 优化加载速度 2. 通过 动态导入 (Dynamic Import) 按需加载模块	减少首屏加载时间, 优化资源利用
Tree Shaking	1. 启用 Tree Shaking 移除未使用的代码 2. 配置 sideEffects: false 优化依赖库打包	减少打包体积, 移除无用代码
缓存	1. 启用 缓存 , 使用 Cache-Control 缓存策略 2. 为静态资源添加 hash 或 chunkhash	提升重复访问时的加载速度
图片和资源优化	1. 使用 image-webpack-loader 等工具压缩图片 2. 使用 url-loader 或 file-loader 处理静态资源	优化资源加载速度, 减少资源占用
代码压缩	1. 启用 TerserPlugin 压缩 JavaScript 文件 2. 使用 css-minimizer-webpack-plugin 压缩 CSS	减少打包体积, 提升加载性能
多线程打包	1. 使用 thread-loader 启用多线程打包 2. 使用 parallel-webpack 提高打包速度	提升打包性能, 减少打包时间
预编译依赖	1. 使用 DllPlugin 和 DllReferencePlugin 预编译依赖库 2. 缓存未变化的第三方库	减少构建时间, 提高打包效率
按需加载	1. 配置 lazy loading 按需加载非关键资源 2. 使用 import() 动态导入模块	优化资源利用, 减少不必要的加载
使用生产模式	1. 使用 mode: 'production' 启用生产环境配置 2. 启用 webpack.DefinePlugin 配置环境变量	启用默认优化选项, 优化打包效果
CSS 和 JS 分离	1. 使用 MiniCssExtractPlugin 分离 CSS 文件 2. 避免将所有 CSS 打包到一个文件中, 使用按需加载	减少首屏加载时间, 提升加载性能
模块联邦 (Module Federation)	1. 使用 Module Federation 共享跨应用模块 2. 在多个项目之间共享依赖库	提升模块复用, 减少重复打包依赖
DevTool 优化	1. 在开发环境中使用 cheap-module-source-map 提升构建速度 2. 在生产环境中禁用 source-map 减少体积	优化开发体验, 减少生产环境体积

4.代码层面优化

技术栈	优化方案	备注
原生JS	1. 减少 DOM 操作，将频繁的 DOM 操作缓存或批量更新 2. 使用 事件委托 处理大量事件监听 3. 懒加载 非必要资源，如图片和脚本 4. 使用 requestAnimationFrame 优化动画性能 5. 节流 和 防抖 优化用户频繁操作 6. 使用 Web Worker 处理繁重任务，避免阻塞主线程	原生JS 需手动管理状态和 DOM 操作，优化代码执行效率尤为重要
Vue框架	1. 启用 Vue 3 Composition API 复用逻辑，提升代码可维护性 2. 使用 v-if 替代 v-show 在条件渲染时节省性能 3. 通过 keep-alive 缓存动态组件，提升切换性能 4. 使用 async components 实现组件按需加载 5. 使用 Vuex 或 Pinia 管理复杂状态，避免直接修改组件数据 6. 使用 Lazily-loaded Routes 实现路由懒加载 7. 使用 Vue Devtools 分析性能瓶颈	Vue 提供了良好的数据绑定和 DOM 管理机制，合理使用指令和组件缓存至关重要
React框架	1. 使用 React.memo 和 PureComponent 避免不必要的重新渲染 2. 启用 useCallback 和 useMemo 优化函数和变量的重复创建 3. 通过 Lazy Loading 和 Suspense 实现组件按需加载 4. 合理使用 Context API，避免频繁状态更新影响子组件 5. 使用 React Profiler 识别和优化性能瓶颈 6. 启用 Code Splitting 将大型应用拆分为小块，减少首次加载时间 7. 通过 Error Boundaries 捕获渲染错误，提升应用稳定性	React 框架注重组件化开发，渲染优化和状态管理是提高性能的关键

2.JavaScript

1.Promise

Promise 维度	总结
1. Promise 概述	1. 解决回调地狱：Promise 解决了嵌套回调（回调地狱）的问题，提供了更为优雅的异步处理方案。通过链式调用 .then()，可以将复杂的异步逻辑按顺序组织，避免层层嵌套。 2. 状态管理与不可逆：Promise 有三种状态 pending（进行中）、fulfilled（已成功）和 rejected（已失败），并且一旦状态发生变化，就 不可逆，确保异步操作的结果一旦确定就不会被

	修改。
2. Promise 基本使用	1. 链式调用与错误捕获：通过 <code>.then()</code> 方法链式调用，逐步处理异步操作的 成功与失败，并通过 <code>.catch()</code> 方法捕获异常，进行统一的错误处理。可以将异步逻辑按步骤拆解，提高代码的可读性和可维护性。 2. 状态改变：Promise 允许通过 <code>resolve(value)</code> 和 <code>reject(reason)</code> 分别返回成功和失败结果。可以结合业务需求，在不同状态下做出相应处理。
3. Promise 静态方法	1. <code>resolve(value)</code>：返回一个以给定值解析的 Promise 对象，用于手动创建成功的异步操作。 2. <code>reject(reason)</code>：返回一个带有拒绝原因的 Promise 对象，用于手动创建失败的异步操作。 3. <code>Promise.all([p1,p2,p3....])</code>：接收一组 Promise 实例，所有实例成功后才返回成功结果；如果有一个失败，返回该失败结果，常用于处理多个异步任务的并发。 4. <code>Promise.race([p1,p2,p3....])</code>：谁最先完成（无论是成功还是失败），返回那个最先执行完毕的 Promise 结果，适合需要对最快结果做出响应的场景。 5. <code>Promise.allSettled([p1,p2,p3....])</code>：无论成功或失败，返回所有结果。
4. Promise 实例方法	1. <code>then(onFulfilled, onRejected)</code>：添加成功和失败的回调，返回一个新的 Promise 对象，支持链式调用，实现顺序处理异步任务。 2. <code>catch(onRejected)</code>：捕获链式调用中的错误，进行统一错误处理，尤其是在链式调用中后续步骤依赖于前面的操作时非常有用。 3. <code>finally(onFinally)</code>：无论成功还是失败，finally 都会在最后执行，常用于资源清理或结尾操作，确保在异步任务结束后进行必要的处理。
5. 手写 Promise 源码	1. 核心构造原理：Promise 是一个类，在构造函数 constructor 中接收一个执行器 executor，它包含两个回调函数 resolve 和 reject，分别代表异步操作成功和失败的处理方式。 2. 异步支持与回调队列：通过 onFulfilledCallbacks 和 onRejectedCallbacks 分别存储成功和失败的回调队列，确保异步任务完成后依次执行回调。 3. 状态管理与不可逆：一旦调用 resolve 或 reject，状态从 pending 变为 fulfilled 或 rejected，状态不可逆，确保操作结果的确性和可靠性。
6. Promise 高级实现	1. Promise.all 实现：<code>Promise.all</code> 是一个静态方法，接收多个 Promise 对象，只有当所有 Promise 都成功时，才返回一个新的成功的 Promise，否则返回第一个失败的结果。它是处理并发任务时非常有用的工具，适合场景如批量请求多个 API，确保所有请求都成功后再做进一步处理。 2. Promise.race 实现：<code>Promise.race</code> 返回最先改变状态的 Promise，无论是成功还是失败，非常适合需要对最快完成任务做出响应的场景，例如多服务请求时选择最快的返回值。
7. Promise 的应用场景	1. 并发任务处理：通过 <code>Promise.all</code> 处理多个异步任务，所有任务都完成后执行统一操作，适合并发 API 请求或批量处理任务。 2. 竞赛任务处理：通过 <code>Promise.race</code> 对比多个异步任务的速度，谁先完成返回谁的结果，适用于竞赛场景或需要快速响应的任务。 3. 统一异常处理：通过 <code>Promise.catch</code> 对多个链式操作中的错误进行统一捕获和处理，避免逐层嵌套的复杂性，提高代码可读性。 4. 异步流程控制：Promise 通过链式调用实现顺序处理异步操作，结合 finally 确保流程末尾资源清理。
8. Promise 的优势	1. 解决回调地狱问题：通过链式 <code>.then()</code> 方法，避免了传统回调函数嵌套过多导致的代码可读性差的问题，尤其在处理多层次异步操作时，Promise 提供了更好的结构化控制流程。 2. 错误处理机制完善：Promise 支持 <code>.catch()</code> 进行统一的错误捕获，并支持通过 <code>.finally()</code> 进行收尾操作，确保即使发生错误也能进行资源回收或状态重置。
9. Promise 的局限性	1. 无内置取消功能：Promise 一旦创建，无法中途取消执行，这在处理长时间未完成的异步任务时可能带来资源浪费的问题。 2. 缺乏序列化处理：对于需要一步步依赖前一步结果的操作，Promise 的链式调用较为繁琐且容易失去灵活性。在这种场景下，Async/Await 提供了更自然的顺序处理方式。

2. JS事件环

1. 事件循环的步骤

1. 执行全局脚本代码，这可以视为一个宏任务。
2. 执行完毕后，检查微任务队列，如果有微任务，执行微任务。
3. 微任务执行完毕后，开始下一个宏任务（如`setTimeout`定时器回调），重复步骤2。
4. 循环此过程。

2. 微任务与宏任务

JavaScript的异步任务可以分为两类：宏任务和微任务

宏任务：包括整体代码script、`setTimeout`、`setInterval`、I/O操作、UI渲染等。

微任务：`Promise.then`、`async/await`、`process.nextTick`（Node.js环境）等。

1. 在每个宏任务执行完毕后，如果存在微任务队列，那么会先清空微任务队列，即执行所有微任务。
2. 只有当微任务队列为空时，才会继续执行下一个宏任务。

3. setTimeout精度丢失

Js是一个单线程语言，同一时间只能做同一件事情。但单线程也带来了一些问题，比如线程阻塞，CPU利用率不高等问题

代码演示

```
let startTime = Date.now();
setTimeout(function () {
  console.log('任务完成于: ' + (Date.now() - startTime) + 'ms')
}, 0)

for (let i = 0; i < 9000; i++) {
  console.log(5 + 8 + 8 + 8)
}

console.log('主线程复杂任务耗时:' + (Date.now() - startTime) + 'ms')
```

解决方案(不彻底，没办法)

使用 `requestAnimationFrame` 代替 `setTimeout` 可以获得更高的精度。`requestAnimationFrame` 会在浏览器的下一次重绘之前调用回调函数，通常是 16.67ms（60帧每秒）。

```
let startTime = performance.now();

// 定义一个复杂任务函数，该函数执行大量计算
function complexTask() {
  // 模拟一个复杂的主线程任务
  for (let i = 0; i < 9000; i++) {
    console.log(5 + 8 + 8 + 8); // 执行计算并输出结果
  }
  // 输出主线程复杂任务的耗时
}
```

```
        console.log('主线程复杂任务耗时: ' + (performance.now() - startTime).toFixed(2)
+ 'ms');
    }

    // 定义一个动画帧回调函数
    function onAnimationFrame() {
        // 输出任务完成时的耗时，使用高精度时间计算
        console.log('任务完成于: ' + (performance.now() - startTime).toFixed(2) +
'ms');

        // 在下一次动画帧调用复杂任务函数，确保任务在绘制帧之前执行
        requestAnimationFrame(complexTask);
    }

    // 使用 requestAnimationFrame 调度 onAnimationFrame 函数
    // 这确保 onAnimationFrame 函数会在浏览器下一次重绘前执行
    requestAnimationFrame(onAnimationFrame);
```

4. ES6新特性

ES6 新特性维度	总结
1. let 和 const	块级作用域 ： <code>let</code> 和 <code>const</code> 定义的变量具有块级作用域，避免变量提升问题， <code>const</code> 用于声明不可变常量。
2. 箭头函数	简化函数表达式 ：箭头函数简化函数的书写，并且 不绑定自身的 <code>this</code> ，更适合回调函数。
3. 类 (Classes)	面向对象语法糖 ：通过类的语法，定义构造函数、方法和继承，提供了更直观的面向对象编程模式。
4. 模板字符串	多行字符串与插值 ：使用反引号 (<code>`</code>) 编写多行字符串，并通过 <code>\${}</code> 进行变量插值，提升字符串处理的灵活性。
5. 解构赋值	简化数据提取 ：通过解构赋值，快速从对象或数组中提取数据，减少手动赋值操作。
6. 默认参数与剩余参数	参数默认值与灵活传参 ：函数参数支持默认值，剩余参数 (<code>...</code>) 允许将不定数量参数转换为数组，提升函数灵活性。
7. Promises	异步操作管理 ：Promise 提供了更优雅的异步操作处理方式，避免回调地狱，通过 <code>.then()</code> 和 <code>.catch()</code> 实现链式调用。
8. ES6 模块化	模块导入与导出 ： <code>import/export</code> 提供了模块化支持，允许模块拆分与复用，替代传统的全局变量共享机制，提升代码组织性和维护性。
9. 新数据结构	Map 和 Set ：Map 提供键值对存储，Set 用于存储唯一值，WeakMap 和 WeakSet 提供弱引用支持，避免内存泄漏。
10. 迭代器与生成器	迭代与暂停机制 ：Iterator 提供遍历机制，Generator 函数允许函数执行的暂停与恢复，简化复杂迭代逻辑。
11. 新数组与对象方法	Array.from 与 Object.assign ：Array.from 将类数组对象转为数组，Object.assign 合并对象，简化数组与对象操作。
12. Symbol 类型	唯一性标识符 ：Symbol 创建唯一且不可变的标识符，常用于对象属性键，避免命名冲突。
13. Proxy 和 Reflect	元编程支持 ：Proxy 用于拦截对象操作，Reflect 提供默认行为调用，增强了对对象的控制能力，适合元编程场景。

5. 数组内置高阶函数

- 1. `map`：不会修改原数组，返回一个新数组
- 2. `filter`：不会修改原数组，返回一个过滤后新数组
- 3. `reduce`：数组迭代器，对数组元素进行累计、聚合、转换或计算
- 4. `forEach`可以使用`return`，退出当前循环，继续下次循环**
- 5. `some`：有一个为真，结果为真
- 6. `every`：全部都为真，结果为真，否则结果为假
- 7. `sort`：数组排序

6. 原型和原型链

概念维度	总结
1. 原型 (Prototype)	Prototype 是函数的一个属性，指向该函数的 原型对象 ，当函数作为构造函数被调用时，基于该构造函数创建的实例对象会共享访问这个原型对象上的属性和方法。这样可以实现 属性与方法的共享 ，节省内存。
2. 原型链 (Prototype Chain)	__proto__ 是对象的一个 内部属性 ，指向创建它的构造函数的原型对象 (prototype)。每个原型对象本身也有 __proto__ 属性，形成链式结构，称为 原型链 。当访问对象的属性或方法时，若对象本身没有，则会沿着原型链向上查找，直到找到或到达链的顶端 null 。

7. this指向

- 1. **全局上下文**：在全局执行环境中（非严格模式下），this指向全局对象；在浏览器中是window对象，在Node.js中是global对象。在严格模式 ('use strict') 下，this的值为undefined。
- 2. **函数调用**：当函数不作为对象的方法被调用时，this指向全局对象（非严格模式下）或undefined（严格模式下）。
- 3. **方法调用**：当函数作为对象的方法被调用时，this指向该方法所属的对象。
- 4. **构造函数**：在构造函数中，this指向新创建的对象。
- 5. **箭头函数**：箭头函数没有自己的this，它会捕获其所在上下文的this值作为自己的this值，无法通过bind、call或apply改变。

改变this指向：bind call apply

8. 事件委托冒泡和捕获

事件委托是一种利用事件冒泡原理来优化事件监听的技术。通过在父元素上设置监听器来管理子元素的事件，而不是直接在子元素上设置监听器。

1. 事件冒泡

当一个事件触发在DOM的一个元素上时，这个事件会向上冒泡到其父元素，一直到document对象。

优点：

- 1. **减少内存消耗**：减少事件监听器的数量，降低内存占用。
- 2. **动态元素管理**：对于动态添加的子元素，无需重新绑定事件监听器。

2. 事件捕获：

当一个事件从document对象传导到目标节点（触发事件的元素）的过程。在事件捕获阶段，事件会从最外层的父元素开始，逐级向下传递到目标元素。在addEventListener的第三个参数设置为true，即可在捕获阶段

9. 数据类型判断

数据类型判断方式	总结
1. <code>typeof</code>	基本类型判断 ： <code>typeof</code> 能正确判断 基本类型（除 <code>null</code> 外），对 函数 返回 <code>"function"</code> ，但对 数组和 <code>null</code> 返回 <code>"object"</code> ，无法准确区分对象和数组。
2. <code>instanceof</code>	原型链检测 ： <code>instanceof</code> 用于判断对象是否是某构造函数的实例，适用于检测 对象类型，但无法用于基本类型的判断。常用于自定义类和内置对象（如数组）的类型检测。
3. <code>Object.prototype.toString.call</code>	推荐使用，精确判断 ：通过 <code>Object.prototype.toString.call()</code> 可以精确判断包括 数组、 <code>null</code> 、函数、基本类型 在内的所有数据类型，返回 <code>[object Type]</code> 格式的字符串，是最可靠的类型判断方式。
4. <code>Array.isArray()</code>	数组专用判断 ： <code>Array.isArray()</code> 是专门用于判断数组的函数，能够正确判断数组类型，优于 <code>typeof</code> 和 <code>instanceof</code> 。

10. JS作用域

- 1. **全局作用域**：在代码中任何地方都能访问到的变量和函数。
- 2. **局部作用域**：只能在定义它们的函数或代码块内访问到的变量和函数。

局部作用域：

- **函数作用域**：变量和函数在整个函数内部都是可访问的，但在函数外部不可访问。
- **块级作用域**（ES6引入）：通过`let`和`const`声明的变量在其所在的代码块（如`if`语句、`for`循环）内有效。

作用域链：

当访问一个变量时，JavaScript会首先在当前作用域查找，如果没找到，会继续在上一层作用域查找，直到找到该变量或达到全局作用域。这种结构称为作用域链。

11.for in for Of 区别(掌握)

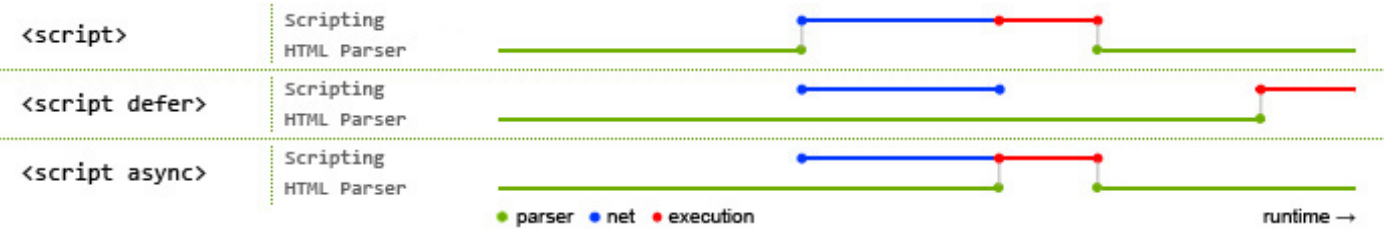
区别维度	总结
1. <code>for...of</code> 原理	遍历可迭代对象 ： <code>for...of</code> 用于遍历实现了 可迭代协议 （即实现了 <code>Symbol.iterator</code> 接口）的数据结构，遍历的内容是 元素本身 。常用于遍历 数组 、 字符串 、 Map 、 Set 、 NodeList 等可迭代对象。
2. <code>for...in</code> 原理	遍历对象的可枚举属性 ： <code>for...in</code> 用于遍历对象的 可枚举属性 ，包括对象原型链上的可枚举属性。遍历的内容是 属性名（键） ，不适合遍历数组元素，常用于遍历对象的属性。
3. <code>for...of</code> 可遍历类型	1. 数组（Array） ：遍历数组的元素。 2. 字符串（String） ：遍历字符串的每个字符。 3. Map/Set ：遍历 Map 键值对、 Set 的值。 4. arguments 对象 ：遍历函数参数。 5. NodeList ：遍历 DOM 节点列表。 6. 生成器对象 ：遍历生成器的每个返回值。
4. <code>for...in</code> 可遍历类型	1. 对象（Object） ：遍历对象的可枚举属性（包括原型链上的）。 2. 数组（Array） ：遍历数组的索引（不推荐用于数组）。 3. 字符串（String） ：遍历字符串的索引。 4. arguments 对象 ：遍历函数参数的索引。 5. 用户自定义可枚举属性 ：遍历对象上手动添加的可枚举属性。

12.url输入地址到浏览器的一个过程

1-8 可只说文字加错部分，9要详细说

1. **URL解析**：浏览器先解析URL，包括协议、主机名、端口号、路径等信息。
2. **DNS解析**：将域名解析为IP地址。
3. **建立TCP连接**：浏览器与服务器之间通过TCP协议建立连接，这个过程通常包括三次握手。
4. **发送HTTP请求**：浏览器向服务器发送HTTP请求
5. **服务器处理请求**：可能是返回请求的资源，也可能是执行后台逻辑。
6. **服务器返回响应**：服务器处理完请求后，将响应内容发送给浏览器。
7. **浏览器接收响应**：根据响应内容渲染HTML、解析JavaScript。
8. **关闭TCP连接**：响应完成后会被关闭，释放资源。
9. **渲染页面(重点说)**：浏览器将接收到的HTML、CSS、JavaScript等内容解析并渲染成可视化页面，显示给用户。
 1. **解析HTML**：把HTML文档解析成DOM树。
 2. **构建DOM树**：构建DOM树
 3. **解析CSS**：解析css，构建CSS树
 4. **合并DOM和CSSOM**：浏览器将DOM树和CSSOM合并，生成渲染树
 5. **布局**：浏览器根据渲染树计算每个元素在页面中的位置和大小，这个过程是重排。
 6. **绘制**：浏览器根据布局结果将每个元素绘制到页面上，包括背景、边框、文本等。
 7. **渲染完成**：页面渲染完成后，浏览器会将渲染好的页面显示给用户。

13.defer 和 async 的区别是什么？



绿线：HTML 的解析时间

蓝线：JS 脚本的加载时间

红色：JS 脚本的执行时间

- 1.defer和async属性让JS在网络加载过程都不会阻塞HTML的解析，都是异步执行的。
- 2.两者的区别，脚本加载完成之后，async是立刻执行,defer会在DOMContentLoaded之前执行

铺垫：

- 1. **DOMContentLoaded** 事件不需要等待页面的所有资源加载完成；
- 2. **load** 事件在页面的所有资源加载完成后触发。
- 3.先执行DOMContentLoaded事件 再执行load事件。

14.前端存储的区别是什么？

方式名称	标准说明	功能说明
Cookies	HTML5 前加入	1.会为每个请求自动携带所有的Cookies数据，比较方便，但是也是缺点，浪费流量； 2.每个domain(站点)限制存储20个cookie； 3.容量只有4K 4.浏览器API比较原始，需要自行封装操作。（js-cookie）
localStorage	HTML5 加入	1.兼容IE8+，操作方便； 2.永久存储，除非手动删除； 3.容量为5M，在浏览器所有同源窗口间共享
sessionStorage	HTML5 加入	1.功能基本与 localStorage 相似，但当前页面关闭后即被自动清理； 2.在同源窗口间不共享，属于会话级别的存储

15.深拷贝 浅拷贝

拷贝方式维度	总结
1. 浅拷贝	浅拷贝只复制第一层的引用，即对象的第一层属性是独立的，但嵌套的引用类型依然共享内存。 实现方式： 1. <code>...</code> 对象展开运算符 2. <code>Object.assign()</code> 3. 数组的 <code>slice()</code> 方法。
2. 深拷贝	深拷贝递归复制所有层级，确保新对象与原对象完全独立。 实现方式： 1. 递归函数：手写递归函数遍历所有属性，逐层复制。 2. <code>JSON.parse(JSON.stringify())</code>：简单但不能处理函数、<code>undefined</code>、正则、循环引用。 3. 第三方库：使用 <code>Lodash</code> 的 <code>_cloneDeep()</code> 处理复杂数据结构和循环引用。
3. 深拷贝的递归实现	能够处理 <code>Date</code> 、 <code>RegExp</code> 、 <code>Map</code> 、 <code>Set</code> 等特殊对象类型，并有效解决 循环引用问题 ，确保拷贝后的对象与原对象 完全独立 ，适合复杂场景。

```
// 手写深拷贝
function deepClone(obj, hash = new WeakMap()) {
  // 如果传入的对象是 null 或者不是对象类型，则直接返回该对象
  if (obj === null || typeof obj !== "object") {
    return obj; // 原始类型值直接返回，如：数字、字符串、布尔值、null、undefined、symbol、
    bigint
  }

  // 检查循环引用的情况，防止递归陷入死循环
  if (hash.has(obj)) {
    return hash.get(obj); // 如果该对象已经被克隆过，则直接返回之前的克隆结果
  }

  // 处理 Date 类型，返回新的 Date 实例
  if (obj instanceof Date) {
    return new Date(obj);
  }

  // 处理 RegExp 类型，返回新的正则表达式实例
  if (obj instanceof RegExp) {
    return new RegExp(obj);
  }

  // 处理 Map 类型
  if (obj instanceof Map) {
    const mapClone = new Map(); // 创建一个新的 Map 实例
    hash.set(obj, mapClone); // 将原始 Map 对象与克隆后的对象存入 hash 中
    obj.forEach((value, key) => {
      mapClone.set(key, deepClone(value, hash)); // 深拷贝 Map 中的每一个键值对
    });
    return mapClone;
  }
}
```

```

// 处理 Set 类型
if (obj instanceof Set) {
    const setClone = new Set(); // 创建一个新的 Set 实例
    hash.set(obj, setClone); // 将原始 Set 对象与克隆后的对象存入 hash 中
    obj.forEach(value => {
        setClone.add(deepClone(value, hash)); // 深拷贝 Set 中的每一个值
    });
    return setClone;
}

// 创建对象或数组的拷贝，根据是数组还是对象来选择相应的初始值
const clone = Array.isArray(obj) ? [] : {};
hash.set(obj, clone); // 将原始对象与克隆后的对象存入 hash 中，处理循环引用

// 遍历对象或数组的属性
for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
        clone[key] = deepClone(obj[key], hash); // 递归调用 deepClone 对每个属性进行深拷
    }
}

return clone; // 返回最终深拷贝的对象或数组
}

```

16. JS继承

原型链继承 构造函数继承 组合继承 原型式继承 寄生式继承 寄生组合式继承 extends继承

17. 模块化开发

分 CommonJS、AMD、UMD、ES6模块化，推荐使用 ES6 模块化

真正的模块化，它包括 CJS、AMD、CMD、UMD 和 ESM，经过多年演变，目前 Web 开发 倾向于 ESM，Node 开发 倾向于 CJS

特性	CJS	ESM
语法类型	动态	静态
关键声明	<code>require</code>	<code>import</code> 和 <code>export</code>
加载方式	运行时加载	编译时加载
加载行为	同步加载	异步加载
书写位置	任意位置	必须在顶层
指针指向	<code>this</code> 指向当前模块	<code>this</code> 指向 <code>undefined</code>
执行顺序	首次引用时加载模块，之后读取缓存	引入时生成引用，执行时才正式取值
属性引用	基本类型值复制不共享，引用类型浅拷贝共享	所有类型动态读取引用
属性修改	工作空间可修改引用的值	工作空间不可修改引用值，但可通过修改方法

模块化方案	定义	特点
IIFE	使用立即执行函数表达式封装模块化。	避免变量污染，全局唯一命名空间，适用于早期无模块化工具的场景。
CJS	Node.js 的模块化标准，每个文件是一个独立模块。	支持模块缓存和同步加载，常用于服务器端开发。
AMD	浏览器端模块化标准，通过 <code>define</code> 和 <code>require</code> 实现异步加载。	适合浏览器端异步加载模块，不符合现今广泛使用的模块化思维方式。
CMD	使用 Sea.js 规范，依赖就近，异步加载。	模块加载量更小，适合依赖就近的场景。
UMD	通用模块定义，兼容 CJS 和 AMD，且支持浏览器和服务端环境。	适用于混合开发环境的模块加载方案。
ESM	ES6 的原生模块标准，支持静态分析和异步加载。	现代 Web 开发的主流方案，支持 <code>import/export</code> ，浏览器原生支持，提升了开发体验和性能。

18. 取消请求

1. axios取消请求

使用 `CancelToken` 来实现。可以创建一个 `CancelToken` 实例，并将它传递给请求的配置中，然后在需要取消请求的地方调用 `CancelToken` 的 `cancel` 方法，取消正在进行的请求。

下面是一个简单的示例代码：

```
import axios from 'axios';

// 创建 CancelToken 实例
const source = axios.CancelToken.source();
```

```
// 发起请求并传递 CancelToken
axios.get('/api/data', {
  cancelToken: source.token
}).then(response => {
  // 请求成功的处理逻辑
}).catch(error => {
  // 请求失败的处理逻辑
  if (axios.isCancel(error)) {
    console.log('请求已取消', error.message);
  } else {
    console.log('请求出错', error.message);
  }
});

// 在需要取消请求的地方调用 cancel 方法
source.cancel('请求被取消了');
```

2.fetch取消请求

使用 **AbortController** 来实现，AbortController 是一个用于控制 Fetch 请求的 API，可以用来取消请求。

创建了一个 **AbortController** 实例，然后通过该实例的 **signal** 属性创建了一个 **signal** 对象，将其传递给 Fetch 请求的配置中。最后，通过调用 **AbortController** 实例的 **abort** 方法，可以取消该请求

```
// 创建 AbortController 实例
const controller = new AbortController();
// 从控制器中获取信号
const signal = controller.signal;

// 发起 Fetch 请求并传递信号
fetch('/api/data', { signal })
  .then(response => {
    // 请求成功的处理逻辑
  })
  .catch(error => {
    // 请求失败的处理逻辑
    if (error.name === 'AbortError') {
      console.log('请求已取消');
    } else {
      console.log('请求出错', error.message);
    }
  });

// 在需要取消请求的地方调用 abort 方法
controller.abort();
```

3. axios 重连机制

通过递归函数手动实现重试逻辑

```
import axios from 'axios';

// 定义一个函数，用于给 axios 实例添加重试逻辑
function axiosRetry(axiosInstance, options) {
  const { retries, retryDelay } = options; // 从 options 中解构出重试次数和重试间隔

  // 添加响应拦截器
  axiosInstance.interceptors.response.use(
    response => response, // 成功响应时直接返回响应
    async error => {
      const { config } = error; // 获取请求配置

      // 如果请求配置没有重试计数器，初始化为 0
      if (!config.__retryCount) {
        config.__retryCount = 0;
      }

      // 如果当前重试次数超过了设置的最大重试次数，拒绝请求并抛出错误
      if (config.__retryCount ≥ retries) {
        return Promise.reject(error);
      }

      // 增加重试计数器
      config.__retryCount += 1;

      // 在重试之前等待指定的时间
      await new Promise(resolve => setTimeout(resolve, retryDelay));

      // 重新发起请求
      return axiosInstance(config);
    }
  );
}

// 使用示例
const instance = axios.create({
  baseURL: 'https://api.example.com' // 设置基础 URL
});

// 为 axios 实例添加重试逻辑
axiosRetry(instance, {
  retries: 3, // 设置最大重试次数
  retryDelay: 1000 // 设置每次重试间隔时间为 1 秒
});

// 发起 GET 请求
```

```
instance.get('/endpoint')
  .then(response => console.log(response.data)) // 请求成功时打印响应数据
  .catch(error => console.error('Request failed:', error)); // 请求失败时打印错误信息
```

19.hash路由和history路由区别

路由方式	核心总结与应用场景
1. Hash 路由	基于 URL hash 部分的路由管理，使用 # 号管理页面导航，浏览器不会将 hash 部分发送到服务器，因此 不需要服务器配置。适用于无需 SEO 支持的应用，通过 hashchange 事件监测 URL 变化。
场景	Vue Router 会维护一个路由表，通过 hash 变化找到对应的组件加载到页面中。无需服务器配置，适合无需刷新页面的应用。
2. History 路由	基于 HTML5 History API 的路由管理，使用 pushState 和 replaceState 实现无刷新跳转。需要服务器端配置来处理直接访问和页面刷新，确保返回正确的页面，适合需要 SEO 支持的单页应用。
场景	需要与服务器配合，特别是在刷新页面时必须让服务器处理所有的路由请求，适合 SEO 友好的应用场景和更加现代的前端应用开发方式。

20.JS 执行上下文和生命周期以及执行栈

JS 执行机制维度	精简优化与核心总结
1. 执行上下文	1. 全局执行上下文：全局作用域下的上下文，在浏览器中是 window，在 Node.js 中是 global。 2. 函数执行上下文：每次调用函数时都会创建，包含局部变量、参数和作用域链。 3. Eval 执行上下文：仅在执行 eval() 时创建，不推荐使用。
2. 执行上下文组成	1. 变量对象 (VO)：存储变量、函数声明和形参，在函数上下文中称为 活动对象 (AO)。 2. 作用域链：用来解析变量，包含当前上下文和父级上下文。 3. this 指向：上下文会绑定 this，依据调用方式决定指向。
3. 执行上下文生命周期	1. 创建阶段：完成函数提升、变量提升和 this 的绑定。 2. 执行阶段：代码按顺序执行，变量和函数被赋值和调用。 3. 销毁阶段：上下文执行完毕后销毁，释放内存。
4. 执行栈	调用栈是后进先出结构：用于跟踪函数调用。 1. 入栈：函数被调用时，将其执行上下文推入栈顶。 2. 出栈：函数执行完毕后，执行上下文从栈顶弹出。 3. 栈底永远是全局上下文，所有函数执行完毕后只剩全局上下文。

21. 前端跨域解决方案

跨域解决方案	精简优化与核心总结
1. Proxy 代理	开发环境代理解决方案 ：通过 Vue 和 React 项目的 proxy 配置，可以将请求代理到不同的服务器，避免跨域问题。常用于开发环境，通过本地服务器转发请求实现跨域。
2. CORS（跨域资源共享）	服务器端配置跨域 ：通过服务器设置响应头，控制哪些域名可以访问资源，最常见的跨域解决方案，配置简单且适用于大部分场景。 如设置 Access-Control-Allow-Origin 允许特定域名跨域访问。
3. Nginx 反向代理	生产环境的反向代理 ：通过 Nginx 配置反向代理，将前端请求转发至目标服务器，实现跨域请求，常用于前后端分离架构下。
4. PostMessage	跨窗口通信 ：使用 window.postMessage 实现不同页面或 iframe 之间的通信，适用于子页面与父页面之间的数据传递。缺点是只适用于页面之间的通信，且需要手动管理安全性。
5. WebSocket 跨域	不受同源策略限制的长连接 ：WebSocket 可以用于实时通信场景，前后端可以双向通信，不受同源策略限制。常用于实时数据更新的应用。缺点是需要服务器支持 WebSocket 协议。
6. document.domain	主域相同，子域不同的跨域 ：通过设置 document.domain 为相同的主域，允许子域之间跨域。此方案只适用于同一主域下的跨域请求，简单且有效，但局限性较大。
7. window.name	利用 window.name 跨页面传递数据 ： window.name 属性支持页面跳转后保留数据，可以用于跨域数据传递。优点是兼容性好，缺点是只能传递字符串数据，适用场景有限。

22. 图片懒加载 接口懒加载

懒加载类型	精简优化与核心总结
1. 图片懒加载	核心思想：滚动加载 。图片懒加载指的是图片不在页面初始加载时全部加载，而是当图片即将进入视口时才进行加载。 实现思路 ： 1. 页面初始加载时使用占位图或 data-src 存储真实图片 URL。 2. 通过 IntersectionObserver 或滚动监听来监测图片是否进入视口。 3. 当图片进入视口时，替换 src 属性为真实 URL，加载图片资源。 4. 加载完成后取消对该图片的监听，优化性能。
应用场景	提高页面初始加载性能 ：适用于包含大量图片的页面，如电商网站、社交媒体等。通过懒加载减少图片资源的初始加载量，提升页面性能，降低用户首次加载时间。
2. 接口懒加载	核心思想：分页或无限滚动加载 。接口懒加载是指仅在用户需要时才发起请求，避免一次性加载大量数据。 实现思路 ： 1. 页面初始加载时仅请求第一页的数据。 2. 监听用户滚动事件，当页面接近底部时，触发新数据请求。 3. 将新请求的数据追加到已有列表中，并更新页面内容。 4. 当所有数据加载完成时，停止发起新请求。
应用场景	减少不必要的接口请求 ：适用于需要分页加载或无限滚动的场景，如社交媒体、新闻网站、数据列表等。通过懒加载减少服务器压力，同时提升用户体验。

3.TypeScript

1.any unknow never 类型区别

类型	精简优化与核心总结
1.any	接收任意类型 ： <code>any</code> 是最灵活的类型，可以接受任何值，但 失去了类型检查的安全性 。适用于需要临时处理未知类型数据或与第三方库集成的场景，但应 谨慎使用 ，避免在大规模项目中使用过多。
2.never	永远不会有值的类型 ： <code>never</code> 表示函数或代码 永远不会有返回值 。常用于 抛出错误 的函数或 无限循环 的函数。它确保调用它的代码不会执行到后续逻辑，适用于无法正常结束的代码分支，如异常处理。
3.unknown	严格的 any 类型 ： <code>unknown</code> 是比 <code>any</code> 更严格的类型，表示未知类型。只能在 明确类型检查后 才能对其进行操作。适用于需要处理未知类型但仍然 希望保持类型安全性 的场景。必须通过类型断言或类型检查后才能使用。

2.interface 和 type区别

特性	interface	type
用途	定义对象类型 ： <code>interface</code> 常用于定义对象结构，适合用来描述对象的形状和行为，支持继承和扩展。	定义类型别名 ： <code>type</code> 用于为任意类型创建别名，不仅可以定义对象类型，还可以用于联合类型、交叉类型、函数类型等。
扩展性	支持继承和声明合并 ：多个 <code>interface</code> 可以通过 <code>extends</code> 实现继承，且可以对同一个 <code>interface</code> 进行多次声明合并。	不支持声明合并 ： <code>type</code> 不支持继承和声明合并，但可以通过交叉类型（ <code>&</code> ）实现类似的功能。
场景	适用于面向对象设计 ：如果需要定义可扩展的对象类型，或需要继承其他对象类型，使用 <code>interface</code> 更加合适。	适用于复杂类型定义 ：如果需要定义联合类型、交叉类型或复杂类型结构，使用 <code>type</code> 更灵活。
功能扩展	具备更多面向对象功能 ： <code>interface</code> 更像是传统的面向对象编程中的接口，允许对接口进行扩展和实现，尤其适合描述类和对象的行为。	定义灵活类型 ： <code>type</code> 适合处理复杂的类型定义场景，支持使用类型运算符（如联合类型 <code> </code> 和交叉类型 <code>&</code> ）进行复杂类型的组合和重用。

3.? 可选链操作符

处理嵌套对象、动态数据和可选参数时，能安全地访问属性或调用方法，避免未定义或空值导致的运行时错误

4. 泛型

泛型是指将类型作为参数进行传递，通过参数传递类型解决代码复用问题。

泛型传参和函数传参要解决的问题是一样的。

泛型应用场景	精简优化与核心总结
1. 泛型函数	函数类型的泛型化 ：通过将类型作为参数传递给函数，泛型函数可以处理多种类型的数据，实现类型安全的代码复用。例如接收一个参数，并返回相同类型的值。
2. 泛型类	类的泛型化 ：通过泛型定义类的属性类型，可以创建更通用的类，适应不同数据类型的对象属性。适用于类的属性和方法可以适配不同类型时，如 <code>key-value</code> 对象的泛型定义。
3. 泛型接口	接口的泛型化 ：通过泛型定义接口，确保接口在不同的类型下能够适配。尤其适用于 API 返回数据类型不确定的场景，通过泛型接口可以定义通用的响应类型，确保返回类型的灵活性和安全性。
4. 泛型约束	限制泛型的类型范围 ：通过 <code>extends</code> 关键字可以对泛型参数进行约束，确保泛型类型在某些范围内使用，防止传递无效类型。适用于需要限制类型的场景，如某些泛型参数必须是字符串或数值类型。

5. 类型操作符

类型操作符	精简优化与核心总结
1. <code>keyof</code> 操作符	提取类型的属性键名 ： <code>keyof</code> 用于提取一个类型的所有键名，并将其转换为 属性字面量联合类型 。适用于需要获取某个类型的属性名集合的场景。 例如， <code>keyof</code> 可以将对象类型的键转换为联合类型 <code>name price</code> 。
2. <code>typeof</code> 操作符	获取变量的类型 ： <code>typeof</code> 用于在类型系统中获取一个值的类型，常用于从现有变量或函数中提取类型定义，避免重复定义类型。 例如，使用 <code>typeof</code> 可以获取对象或函数的类型，并将其应用于其他地方。

6. 工具类型

工具类型	精简优化与核心总结
1. Partial	将类型中的所有属性变为可选 ： <code>Partial<Type></code> 将接收的类型中所有属性设置为可选，常用于需要在类型基础上修改部分属性而不要求提供全部属性的场景。 适用于更新对象时，只需修改部分字段。
2. Omit	省略某些属性的类型 ： <code>Omit<Type, Keys></code> 从给定类型中 移除指定的属性 ，返回一个不包含这些属性的新类型。适用于需要在类型中去除不必要字段的场景。 <code>Omit</code> 可以帮助简化类型定义，省略不需要的字段。
3. Pick	从类型中选择属性 ： <code>Pick<Type, Keys></code> 从给定类型中 选取特定的属性 ，返回一个只包含这些属性的新类型。适用于需要从现有类型中提取特定属性的场景。 <code>Pick</code> 可以帮助创建更精简的类型定义，只保留需要的字段。

4. 构建打包工具

1.webpack

Webpack 默认只会识别js文件和JSON 文件，但通过翻译官（Loaders），Webpack打包所有类型文件

loader: 翻译官

plugin: 增强webpack功能

1.工作流程

1. 初始化：从 webpack.config.js 获取配置信息

↓

2. 解析入口：根据 entry 指定的入口文件，开始解析依赖图

↓

3. 模块解析：对每个文件进行依赖解析，并应用 loader 处理

↓

4. 编译模块：递归解析每个模块的依赖，生成依赖关系图

↓

5. 打包输出：根据依赖关系将模块打包为 bundle

↓

6. 生成输出文件：输出到指定的 output 目录

2.Webpack 热更新HMR

1. Webpack 监听文件变动：Webpack 开启 HMR 后，会监听项目中的模块文件。当文件发生变化时，Webpack 通过 `webpack-dev-server` 或 `webpack-dev-middleware` 进行模块热替换，而不是刷新整个页面。
2. 增量更新：Webpack 只会重新打包变化的模块，并通过 HMR runtime（运行时）将新模块推送到浏览器。
3. 模块热替换：浏览器收到更新的模块后，通过 HMR 运行时动态地替换页面上的模块内容，而无需刷新整个页面。
4. 模块回调：如果一个模块启用了 HMR，那么它会有一个 `module.hot.accept` 函数，可以定义如何处理更新后的模块。

3.webpack打包优化

优化维度	核心优化建议	备注
代码分割	1. 使用 SplitChunksPlugin 将依赖分割成多个 bundle, 优化加载速度 2. 通过 动态导入 (Dynamic Import) 按需加载模块	减少首屏加载时间, 优化资源利用
Tree Shaking	1. 启用 Tree Shaking 移除未使用的代码 2. 配置 sideEffects: false 优化依赖库打包	减少打包体积, 移除无用代码
缓存	1. 启用 缓存 , 使用 Cache-Control 缓存策略 2. 为静态资源添加 hash 或 chunkhash	提升重复访问时的加载速度
图片和资源优化	1. 使用 image-webpack-loader 等工具压缩图片 2. 使用 url-loader 或 file-loader 处理静态资源	优化资源加载速度, 减少资源占用
代码压缩	1. 启用 TerserPlugin 压缩 JavaScript 文件 2. 使用 css-minimizer-webpack-plugin 压缩 CSS	减少打包体积, 提升加载性能
多线程打包	1. 使用 thread-loader 启用多线程打包 2. 使用 parallel-webpack 提高打包速度	提升打包性能, 减少打包时间
预编译依赖	1. 使用 DllPlugin 和 DllReferencePlugin 预编译依赖库 2. 缓存未变化的第三方库	减少构建时间, 提高打包效率
按需加载	1. 配置 lazy loading 按需加载非关键资源 2. 使用 import() 动态导入模块	优化资源利用, 减少不必要的加载
使用生产模式	1. 使用 mode: 'production' 启用生产环境配置 2. 启用 webpack.DefinePlugin 配置环境变量	启用默认优化选项, 优化打包效果
CSS 和 JS 分离	1. 使用 MiniCssExtractPlugin 分离 CSS 文件 2. 避免将所有 CSS 打包到一个文件中, 使用按需加载	减少首屏加载时间, 提升加载性能
模块联邦 (Module Federation)	1. 使用 Module Federation 共享跨应用模块 2. 在多个项目之间共享依赖库	提升模块复用, 减少重复打包依赖
DevTool 优化	1. 在开发环境中使用 cheap-module-source-map 提升构建速度 2. 在生产环境中禁用 source-map 减少体积	优化开发体验, 减少生产环境体积

2.Vite

1.Vite工作流程总结

环境	核心工作流程与总结
1. 开发环境	按需加载与即时更新 ：Vite 启动开发服务器并加载 HTML 文件，利用浏览器原生的 ES 模块（ESM）机制进行模块化加载，按需编译模块。 1. 只在请求时使用 esbuild 编译模块，速度极快。 2. 通过 模块热替换（HMR） 实现高效的实时更新。
2. 生产环境	快速打包与优化 ：Vite 使用 Rollup 进行生产环境的打包。 1. 采用 Tree-shaking 和代码分割优化，减少包的体积。 2. 静态资源优化、压缩与缓存，生成高效的生产构建文件。

2.Vite工作原理

开发与生产环境原理	简要说明
1. 开发环境	Vite 利用 浏览器的 ESM 功能，不再像 Webpack 那样打包整个项目。模块按需加载，避免了项目启动时的冗余预处理。 1. 按需转换 ：使用 esbuild 高速编译请求的模块。 2. 模块热替换（HMR） 仅更新变化的模块，提升开发体验。
2. 生产环境	Vite 在生产环境下使用 Rollup 进行打包，优化模块与静态资源的加载。 1. 依赖预打包 ：使用 esbuild 将第三方库转换为 ESM 格式，提升加载效率。 2. 代码分割与 Tree-shaking ：减少不必要的模块加载，优化初始加载时间。

3.Vite的关键特性

关键特性	核心总结与说明
1. 快速启动	按需加载 ：Vite 不需要像 Webpack 那样在启动时打包整个项目，只处理基础 HTML，并按需加载模块。极大减少了启动时间，尤其在大型项目中表现尤为突出。
2. 极速 HMR	精确模块更新 ：Vite 基于 ESM，能够精确定位模块依赖，修改的模块及其直接依赖会被替换，极大提升开发时的效率和反馈速度。
3. 依赖预构建	使用 esbuild 预构建依赖 ：Vite 利用 Go 语言编写的 esbuild 对依赖库进行预构建，比传统 JavaScript 打包工具快很多。依赖的预构建显著提升了开发服务器的响应速度。
4. 按需编译	模块按需编译 ：Vite 只会在请求某个模块时，才编译并加载该模块，避免了不必要的模块打包，提升了开发环境的轻量化和响应速度。
5. Rollup 打包	生产环境优化 ：Vite 使用 Rollup 作为生产打包工具，支持代码分割、Tree-shaking、静态资源优化等，为生产环境生成高效的打包文件。
6. 内置 TypeScript 支持	高效 TypeScript 编译 ：Vite 原生支持 TypeScript，使用 esbuild 进行快速编译，使得开发体验快速流畅。

3.webpack和vite区别

以下是对 **Webpack** 和 **Vite** 区别的深入总结，涵盖它们在架构、工作原理、开发体验和生产优化等方面的不同之处，帮助开发者理解两者的关键差异，重点内容使用 `` 高亮。

1. 架构与工作原理

特性	Webpack	Vite
1. 打包方式	基于打包的构建工具：Webpack 需要在开发和生产中对项目文件进行 预打包。 1. 在开发阶段，Webpack 将整个项目的所有模块打包成一个或多个文件，通常会造成初始启动速度较慢。 2. 在生产阶段，Webpack 会对项目进行优化打包。	基于原生 ESM 的构建工具：Vite 利用浏览器原生的 ES 模块 (ESM)，在开发阶段不进行打包，而是直接加载模块。 1. Vite 在开发环境中按需加载模块，极大加快了项目启动速度。 2. 生产环境下，Vite 使用 Rollup 进行打包优化。
2. 模块加载方式	预先打包：Webpack 在开发时会将整个项目的所有模块进行打包，生成多个 bundle，通过这些打包文件加载模块。	按需加载：Vite 利用 ESM，只在浏览器请求某个模块时才加载该模块，避免了不必要的预处理，极大缩短了开发时的启动时间。
3. HMR (模块热替换)	基于 WebSocket 的 HMR：Webpack 的 HMR 需要对模块进行部分重新打包，再通过 WebSocket 将变化推送到浏览器，通常较为耗时，尤其在大型项目中表现不佳。	基于 ESM 的精准 HMR：Vite 基于 ESM 的 HMR 通过模块依赖图精准替换模块，只有修改的模块及其直接依赖会被更新，显著提升了模块更新的速度和开发体验。

2. 开发体验

特性	Webpack	Vite
1. 开发速度	相对较慢的启动速度 ：由于 Webpack 需要在启动时对整个项目进行打包，随着项目规模的增大，开发时的启动速度会显著变慢。	极速启动 ：Vite 的启动速度非常快，因为它不需要在开发时进行预打包，而是使用浏览器原生的 ESM 进行按需加载，尤其在大型项目中表现尤为明显。
2. 热更新速度	HMR 较慢 ：Webpack 的 HMR 在较大项目中效率不高，因为它需要对整个模块图进行部分打包并推送给浏览器，尤其是复杂依赖链时表现较差。	极速 HMR ：Vite 的 HMR 基于 ESM 精确定位模块依赖，仅更新修改的模块及其直接依赖模块，因此模块更新速度非常快，带来了极佳的开发体验。
3. 配置复杂性	复杂配置 ：Webpack 具有强大的功能和高度可配置性，但这也意味着它的配置相对复杂，尤其在项目规模变大后，配置文件往往变得冗长和难以管理。	简单配置 ：Vite 的配置相对简单，开箱即用，默认提供了许多常用功能，适合现代前端开发，减少了复杂配置的负担。

3. 生产环境优化

特性	Webpack	Vite
1. 生产打包工具	使用自身打包工具 ：Webpack 既负责开发阶段的打包，也负责生产阶段的优化。使用 Tree-shaking、代码分割、静态资源优化等技术，生成最终的构建文件。	使用 Rollup 作为打包工具 ：Vite 在生产环境下使用 Rollup 作为打包工具，Rollup 提供了更细致的代码分割和静态资源优化功能，生成高效的生产环境打包文件。
2. 依赖处理	依赖处理较慢 ：Webpack 对依赖库的处理是逐步进行的，因此在大型项目中构建时间可能较长。	依赖预构建 ：Vite 使用 esbuild 对依赖进行预构建，比传统的 JavaScript 解析器快很多，减少了依赖的构建时间，并提高了开发服务器的响应速度。

4. 关键差异总结

关键点	Webpack	Vite
启动速度	慢 ：需要在启动时进行打包，项目越大，启动时间越长。	快 ：基于 ESM 的按需加载机制，无需打包，启动速度非常快。
开发体验	复杂的 HMR 机制 ：HMR 较慢，依赖链复杂时表现较差，配置相对复杂。	轻量级 HMR ：基于 ESM 的模块依赖图，HMR 极快，开箱即用的配置，开发体验更好。
生产环境优化	高度定制化的优化 ：Webpack 生产环境下可进行深度优化，适合复杂的项目场景。	Rollup 高效打包 ：Vite 使用 Rollup 进行生产环境打包，提供了细致的代码分割和静态资源优化，生成高效打包文件。

5.HTTP

1. 常见HTTP状态码

成功（2XX）

状态码	原因短语	说明
200	OK	表示从客户端发来的请求在服务器端被正确处理
201	Created	请求已经被实现，而且有一个新的资源已经依据请求的需要而建立 通常是在POST请求，或是某些PUT请求之后创建了内容，进行的返回的响应
202	Accepted	请求服务器已接受，但是尚未处理，不保证完成请求 适合异步任务或者说需要处理时间比较长的请求，避免HTTP连接一直占用
204	No content	表示请求成功，但响应报文不含实体的主体部分
206	Partial Content	进行的是范围请求，表示服务器已经成功处理了部分 GET 请求 响应头中会包含获取的内容范围（常用于分段下载）

重定向（3XX）

状态码	原因短语	说明
301	Moved Permanently	永久性重定向，表示资源已被分配了新的 URL 比如，我们访问 http://www.baidu.com 会跳转到 https://www.baidu.com
302	Found	临时性重定向，表示资源临时被分配了新的 URL，支持搜索引擎优化 首页，个人中心，遇到了需要登录才能操作的内容，重定向 到 登录页
303	See Other	对于POST请求，它表示请求已经被处理，客户端可以接着使用GET方法去请求 Location里的URI。
304	Not Modified	自从上次请求后，请求的网页内容未修改过。 服务器返回此响应时，不会返回网页内容。（协商缓存）
307	Temporary Redirect	对于POST请求，表示请求还没有被处理，客户端应该向Location里的URI重新发起POST请求。 不对请求做额外处理，正常发送请求，请求location中的url地址

因为post请求，是非幂等的， 从302中，细化出了 303 和 307

简而言之：

- 301 302 307 都是重定向
- 304 协商缓存

客户端错误（4XX）

状态码	原因短语	说明
400	Bad Request	请求报文存在语法错误（传参格式不正确）
401	Unauthorized	权限认证未通过（没有权限）
403	Forbidden	表示对请求资源的访问被服务器拒绝
404	Not Found	表示在服务器上没有找到请求的资源
408	Request Timeout	客户端请求超时
409	Conflict	请求的资源可能引起冲突

服务端错误（5XX）

状态码	原因短语	说明
500	Internal Sever Error	表示服务器端在执行请求时发生了错误
501	Not Implemented	请求超出服务器能力范围，例如服务器不支持当前请求所需要的某个功能， 或者请求是服务器不支持的某个方法
503	Service Unavailable	表明服务器暂时处于超负载或正在停机维护，无法处理请求
505	Http Version Not Supported	服务器不支持，或者拒绝支持在请求中使用的 HTTP 版本

当前端看到控制台报出 400 时，前端先检查传参格式是否有误。

2. http缓存

缓存类型	机制	关键字段	工作原理与特性	状态码
强缓存	浏览器直接使用缓存，不向服务器发送请求。	Expires Cache-Control	1. Expires ：基于绝对时间控制缓存，时间到期后缓存失效，可能受客户端与服务器时间差异影响。 2. Cache-Control ：通过 max-age 设置相对时间，支持 no-cache 、 no-store 等选项，优先级高于 Expires ，能够更灵活地控制缓存策略。	无请求，直接使用缓存
协商缓存	浏览器向服务器发送请求，服务器验证缓存是否可用。	Last-Modified/If-Modified-Since ETag/If-None-Match	1. Last-Modified ：服务器返回资源的最后修改时间，浏览器请求时带上 If-Modified-Since 进行对比，判断资源是否发生修改。 2. ETag ：服务器生成资源的唯一标识符 ETag ，浏览器请求时通过 If-None-Match 检查资源是否变化， ETag 精度高于 Last-Modified ，解决了 Last-Modified 无法准确标识频繁修改和时间同步的问题。 3. 若缓存未失效，服务器返回 304 Not Modified ，浏览器使用本地缓存。	304 Not Modified
Expires	通过绝对时间控制缓存过期。	Expires	1. Expires 以绝对时间标识缓存失效时间。 2. 由于基于服务器时间，如果客户端和服务器的时间不一致，可能导致缓存问题。	无请求，直接使用缓存
Cache-Control	通过相对时间控制缓存行为，优先级高于 Expires 。	Cache-Control	1. max-age ：缓存有效期，单位为秒。 2. no-cache ：强制浏览器每次都需要向服务器验证缓存是否可用。 3. no-store ：完全禁止缓存，每次都需要从服务器获取资源。	无请求，直接使用缓存
Last-Modified	通过资源最后修改时间判断缓存有效性。	Last-Modified If-Modified-Since	1. 服务器返回资源的最后修改时间 Last-Modified ，浏览器请求时带上 If-Modified-Since 进行对比，判断资源是否变化。 2. 问题 ： Last-Modified 精度只能到秒，无法处理 1 秒内多次修改的情况。 3. 服务器时间可能不一致，导致缓存判断不准确。 4. 解决方案 ：通过结合 ETag 使用， ETag 可以更准确标识资源变化。	304 Not Modified
ETag	通过资源唯一标识符判断缓存有效性，精度更高。	ETag If-None-Match	1. 服务器生成资源唯一标识符 ETag ，浏览器请求时带上 If-None-Match 进行对比，判断资源是否变化。 2. 为什么需要 ETag ： 1. Last-Modified 只能精确到秒，若 1 秒内多次修改，无法准确标识资源变化。 2. 服务器时间不同步， Last-Modified 可能不准确。 3. ETag 更加精确 ，可解决这些问题。 ETag 是服务器自动生成或由开发者定义的唯一标识符，确保缓存准确性。 4. ETag 和 Last-Modified 可以一起使用，先验证 ETag ，再验证 Last-Modified 。	304 Not Modified

缓存机制流程总结：

缓存流程	解释
强缓存	1. 浏览器首先根据 <code>Expires</code> 或 <code>Cache-Control</code> 判断是否命中强缓存，若命中，则直接从缓存中加载资源，不发送请求到服务器。
协商缓存	2. 若强缓存未命中，浏览器向服务器发送请求，通过 <code>Last-Modified</code> 或 <code>ETag</code> 验证缓存是否仍然有效，若缓存有效，服务器返回 <code>304 Not Modified</code> ，浏览器继续使用本地缓存资源。
完整请求	3. 如果协商缓存也未命中，服务器将返回完整的资源内容，浏览器更新本地缓存并加载新资源。

6.Vue

1.Vue2和Vue3区别

区别点	Vue2	Vue3
1. 响应式系统	1. 基于 <code>Object.defineProperty</code> 实现。 2. 无法监听对象属性新增、删除和数组索引、长度变化。 3. 对深度嵌套对象性能较差。	1. 基于 <code>Proxy</code> 实现， 全面支持 对象动态属性、数组索引和长度变动。 2. 深度响应式性能优化，代理整个对象，提升性能。
2. 编译性能与体积优化	1. 编译与运行时 紧耦合，打包体积较大。 2. 对大量数据和深层嵌套对象的处理性能较差。	1. 支持 <code>Tree-shaking</code> ，减少打包体积。 2. 基于 <code>Proxy</code> 的响应式系统和 <code>Virtual DOM</code> 优化， 显著提升性能 。
3. 组合式 API	1. 使用 <code>Options API</code> ，逻辑分散在多个选项中，代码复用性较差，复杂组件维护困难。	1. 引入 <code>Composition API</code> ，允许通过函数组合逻辑， 提高代码复用性 ，支持复杂组件的高效开发。
4. 异步组件与 Suspense	1. 通过 <code>Vue.component()</code> 实现异步组件加载，缺乏过渡效果和错误处理机制。	1. 支持 <code>Suspense</code> ，处理异步组件的加载、过渡状态和错误处理，提升异步加载体验。
5. Fragment 和 Teleport	1. 模板必须有一个根节点，多根节点时需手动包裹。	1. 支持 <code>Fragment</code> ，允许多个根节点。 2. 引入 <code>Teleport</code> ，组件的 DOM 可以渲染到指定的外部节点，简化弹窗、模态框实现。
6. 自定义渲染器	1. 自定义渲染器支持有限，开发过程复杂。	1. 提供更灵活的 <code>Custom Renderer API</code> ，支持非 DOM 平台（如 WebGL、终端等）的渲染器开发。
7. v-model 改进	1. <code>v-model</code> 只能绑定单个数据属性，且只支持单个 <code>v-model</code> 。	1. 支持 多个 <code>v-model</code> ，增强了灵活性，可以为 <code>v-model</code> 自定义 <code>prop</code> 和 <code>event</code> 名称。
8. 全局 API 调整	1. 全局 API 通过 Vue 实例方法调用，无法轻松实现多应用实例间的全局配置管理。	1. 全局 API 通过应用实例调用 （如 <code>app.component</code> ），支持多实例模式，清晰管理全局配置。
9. TypeScript 支持	1. 有限的 TypeScript 支持 ，类型推导不够强大，大型项目中类型定义复杂。	1. 原生支持 TypeScript ，提供了更好的类型推导和类型检查，开发者可以更轻松使用 TypeScript 构建健壮应用。

2.nextTick

更新 DOM 是异步的，Vue 在侦听到数据变化后，并不会立即更新 DOM，而是将更新操作放入一个队列中，等到所有数据变化完成后，再进行一次统一的 DOM 更新。我们需要在 DOM 更新完成之后执行某些操作，此时就可以用 `nextTick`。

项	解释
1. 原理	Vue 的 DOM 更新是 异步的 ，当数据发生变化时，Vue 不会立即同步更新 DOM，而是将所有更新任务放入一个队列中，等到事件循环结束后，统一执行批量更新。这样可以避免频繁的 DOM 更新，提升性能。
2. 执行时机	nextTick 的执行顺序基于 浏览器事件循环 ：
	1. 同步任务 ：首先执行主线程中的同步任务。
	2. 微任务 ：执行 Promise.then 、 MutationObserver 等微任务。
	3. 宏任务 ：执行 setTimeout 等宏任务。
3. Vue 中 nextTick 执行	Vue 中的 nextTick 会优先放入 微任务队列 ，确保在 DOM 更新完成后立即执行。Vue 会优先使用以下机制来实现 nextTick ：
	1. Promise.then ：现代浏览器中最快的异步方式，优先使用。
	2. MutationObserver ：如果浏览器不支持 Promise ，则使用 MutationObserver 监控 DOM 变化。
	3. setImmediate ：在 IE 中使用 setImmediate 。
	4. setTimeout ：如果以上方式均不支持，则使用 setTimeout 模拟异步执行。
4. 实现流程	1. 数据变化 ：当数据发生变化时，Vue 会进行虚拟 DOM 的更新，但不会立即同步到真实 DOM。
	2. 异步批量更新 ：Vue 将所有的 DOM 更新任务放入一个队列中，等待当前事件循环结束后再进行批量更新。
	3. 执行 DOM 更新 ：在事件循环结束后，Vue 会从队列中取出任务，统一执行 DOM 更新。
	4. 执行 nextTick 回调 ：DOM 更新完成后，Vue 会执行通过 nextTick 注册的回调函数，确保这些回调在最新的 DOM 状态下执行。
5. 总结	Vue 的 DOM 更新是异步的， nextTick 保证在 DOM 更新完成后执行回调。Vue 使用浏览器的异步任务机制（如 Promise 、 MutationObserver 等）来实现 nextTick ，其核心是将回调函数推入异步任务队列中，等待当前 DOM 更新任务完成后再执行。

3.ref和reactive区别

特性	ref 实现原理	reactive 实现原理
响应式机制	基于对象的 <code>.value</code> 属性进行依赖追踪和更新	基于 <code>Proxy</code> 代理对象，递归地使所有属性响应式
用途	主要用于基本类型的响应式数据，也可用于包装对象	主要用于对象和数组的响应式处理，适用于深层嵌套的数据结构
依赖追踪	只追踪 <code>.value</code> 的访问和修改	通过 <code>Proxy</code> 自动拦截 <code>get</code> 和 <code>set</code> 操作，追踪对象所有属性的访问和修改
访问方式	必须通过 <code>.value</code> 来访问基本类型或包装的对象	直接访问对象或数组中的属性，无需通过 <code>.value</code>
深度响应式	不支持，只能包装单个值或浅层对象，内部属性不会变为响应式	支持，自动递归处理所有嵌套的对象属性，内部属性也是响应式

总结

- `ref`：适用于基本类型数据或需要通过 `.value` 显式访问的对象；对基本类型进行响应式处理，使用简单且直观。
- `reactive`：适用于对象和数组，会将所有嵌套的属性变为响应式，适合处理复杂的嵌套数据结构。
- 在实际项目中，有时我们需要混合使用 `ref` 和 `reactive`。例如，将一个复杂对象作为响应式数据，并且使用 `ref` 来处理基本类型或引用类型的某些特殊属性。

4.v-model实现原理

Vue 2：依赖于input事件和value的语法糖。
Vue 3：使用 `modelValue` 和 `update:modelValue` 实现双向绑定。

5.v-if 和 v-for优先级

Vue2 中 `v-for`优先级高于`v-if` vue3中相反

6.Vue2 和Vue3 diff算法

特性	Vue 2 diff 算法	Vue 3 diff 算法
依赖 key 的优化	依赖 key 提升性能，未设置 key 会导致低效的节点移动和删除操作	更依赖 key，结合 LIS 算法对列表做最小 DOM 操作
双端比较策略	使用头-头、尾-尾、头-尾、尾-头的四种方式比较	保留双端比较策略，并结合 LIS 算法优化节点移动操作
长列表性能	在长列表中表现一般，尤其在未正确设置 key 时性能较差	通过 LIS 算法，处理长列表时性能显著提升
DOM 操作最小化	节点复用和更新较为简单，可能发生多次 DOM 移动或替换操作	更智能的节点比较和移动，显著减少 DOM 操作
深层递归遍历	全量递归遍历，性能较低	更灵活地控制递归和比较操作，减少不必要的深层比较
性能表现	整体性能表现良好，但在长列表渲染和更新时性能较低	针对复杂场景和长列表更新进行了显著优化，性能提升明显

1.Vue2中的 diff 算法

特性	解释
整体策略	使用虚拟 DOM，基于双端比较的 diff 算法。
工作流程	通过深度优先遍历递归对比旧新虚拟 DOM 树，使用头-头、尾-尾等策略比较节点。
优化点	使用 key 提升列表渲染效率，减少不必要的 DOM 替换。
局限性	对复杂场景如长列表处理性能较低，递归遍历复杂度高。

2.Vue3中的 diff 算法

特性	解释
整体策略	引入更高效的 diff 算法，特别在列表渲染时结合 LIS 算法优化节点移动操作。
工作流程	通过 key 和位置优化列表节点的复用与插入，避免全量递归比较。
LIS 算法的使用	识别最长递增子序列，减少不必要的 DOM 更新和移动操作。
优化点	更少的递归遍历，更高效的节点复用和 DOM 操作最小化。
局限性	依然依赖 key 提升性能，未合理使用 key 时，性能优化有限。

7.Vue项目性能优化

序号	性能优化项	具体应用场景	深入优化措施及高亮内容
1	懒加载路由和	电商平台后台：多个页面和	1. 通过动态导入和路由懒加载，按需加载页面组件，避免不必要的全局加载。 2. Vue 2 中使用 import() 函数，Vue 3 中使用

	组件	组件的初始加载时间过长。	<code>defineAsyncComponent</code> 实现动态加载，减少首屏资源占用，提升加载速度。
2	虚拟滚动优化长列表渲染	聊天应用：消息列表数据量大，滚动时页面卡顿。	- 使用 <code>vue-virtual-scroller</code> 等虚拟滚动插件，仅渲染可见区域的列表项。 - 这种方式减少了 DOM 元素的渲染和内存占用，提升了大量数据场景下的滚动和渲染性能。
3	事件防抖与节流	实时数据监控系统：页面需要频繁处理用户输入、滚动等事件，性能下降。	- 使用 <code>debounce</code> 和 <code>throttle</code> 减少高频事件的触发次数，避免性能浪费。 - 在 Vue 2 和 Vue 3 中均可通过 <code>lodash</code> 或原生方法实现，提升页面交互响应速度，减少不必要的计算开销。
4	图片懒加载	内容展示平台：页面包含大量高清图片，导致加载时间过长。	- 使用 <code>vue-lazyload</code> 实现图片懒加载，在图片进入视口时才进行加载。 - 结合低分辨率占位图技术，优化用户首次加载体验，减少初始加载时间，尤其适用于图片密集的电商项目和内容展示平台。
5	SSR（服务端渲染）与 CSR 结合	新闻平台或博客系统：页面首次渲染时间长，SEO 要求较高。	- 使用 <code>Nuxt.js</code> 或 <code>Vue SSR</code> 提高首屏渲染速度，Vue 2 和 Vue 3 均可通过 SSR 提升 SEO 优化和页面性能。 - CSR 用于后续页面切换和交互，提高整体性能和用户体验。
6	使用 <code>v-if</code> 代替 <code>v-show</code>	表单系统：多个表单字段和组件动态显示，页面切换时性能下降。	- 对于不频繁显示的组件使用 <code>v-if</code>，仅在需要时才渲染组件。 <code>v-if</code> 会在组件销毁时释放内存，而 <code>v-show</code> 只是隐藏元素，Vue 2 和 Vue 3 中都应优先使用 <code>v-if</code> 处理动态内容渲染，提高性能。
7	组件缓存（ <code>keep-alive</code> ）	表单管理系统：用户在不同表单页之间频繁切换，页面重新加载导致性能下降。	- 使用 <code>keep-alive</code> 缓存未被销毁的组件，避免表单页之间的重复渲染。 - 这种缓存机制在表单切换时保持状态，Vue 2 和 Vue 3 中均可通过 <code>keep-alive</code> 提升性能和用户体验，减少不必要的页面加载。
8	Vuex 状态管理的优化	社交平台：全局状态管理过度复杂，导致更新频繁时性能下降。	- Vue 3 使用 <code>reactive</code> 实现轻量状态管理，Vue 2 中可以通过模块化优化 <code>Vuex</code> 结构。 <code>Vuex</code> 中使用懒加载和按需引入模块，减少全局状态的更新，优化全局状态管理的性能。
9	编译时静态提升与 <code>Tree-shaking</code>	内容管理系统：页面静态内容多，动态部分少，但每次更新都影响性能。	- Vue 3 通过编译时静态提升和 <code>Tree-shaking</code> 优化打包，剔除未使用的代码和模块。 - 静态内容在编译时直接提升，减少运行时的计算开销，避免动态内容的重复渲染，提高渲染性能，尤其适用于静态页面多的场景。
10	Reactivity 系统优化	数据驱动应用：大量数据依赖实时更新，性能容易受到影响。	- Vue 3 使用 <code>Proxy</code> 替代 Vue 2 中的 <code>Object.defineProperty</code> 实现响应式数据追踪，性能更高效。 - 响应式数据追踪更灵活，适合需要大量数据联动更新的复杂场景，避免了 Vue 2 中依赖过多带来的性能瓶颈。
			1. Vue 3 的 <code>Teleport</code> 可以将模态框等组件挂载

11	Teleport 优化 DOM 渲染	模态框组件：模态框等常驻 DOM 的元素会影响页面渲染效率。	到 DOM 外部，减少主页面的 DOM 层级。 2. 这种方式避免了 DOM 结构复杂化，尤其适用于含有多个全局弹窗和模态框的后台系统或电商平台，提升页面渲染性能。
12	watchEffect 替代 watch	数据监控平台：频繁依赖数据的监听，watch 逻辑复杂且效率低。	1. Vue 3 的 watchEffect 可以自动收集依赖，简化依赖数据的追踪逻辑。 2. 这种机制在复杂的监控系统中减少了依赖手动声明的复杂性，提高了数据响应速度和代码可维护性，避免了冗余的依赖收集。
13	Suspense 处理异步加载	电商平台：产品详情页面包含多个异步请求，加载时间长，用户体验差。	1. Vue 3 的 Suspense 组件能够优雅处理异步请求，在异步数据加载时提供加载提示。 2. 这种异步处理方式特别适合异步数据密集的场景，如产品页面，能够有效减少用户等待时间，提升用户体验。
14	动态组件按需加载	综合管理系统：系统包含多个不常用的模块和组件，导致打包体积过大。	1. 通过 defineAsyncComponent 实现动态加载，仅在需要时加载组件，减小首屏打包体积。 2. Vue 2 和 Vue 3 均可使用此技术，减少了资源消耗和加载时间，提升了系统的灵活性和性能。

8.Nuxt.js服务端渲染

类别	核心知识点	详细说明
渲染模式	服务端渲染 (SSR)	Nuxt.js 的核心特性之一是支持服务端渲染 (SSR)。通过 SSR，应用程序在服务器端渲染 HTML，并在客户端进行进一步的交互。相比客户端渲染 (CSR)，SSR 可以提升 SEO 和首屏加载速度。适合需要高 SEO 优化和初始页面加载速度快的应用，如电商平台、博客等。
静态生成	静态站点生成 (SSG)	Nuxt.js 提供了强大的静态站点生成功能，通过 nuxt generate 可以将应用预渲染为静态文件，并在部署时提供极快的页面加载体验。静态生成适合内容较为稳定的场景，例如博客、产品介绍等，Nuxt.js 在构建时生成 HTML，避免每次请求都进行服务器端渲染。
自动路由生成	文件系统路由	Nuxt.js 基于文件系统自动生成路由，开发者无需手动配置路由。通过在 pages 目录下创建 .vue 文件即可生成对应的 URL 路由，支持动态路由和嵌套路由，同时也支持 Catch-all 路由，用于处理复杂的多层级路径结构。
布局与页面管理	布局系统与视图管理	Nuxt.js 提供了强大的布局系统，可以在 layouts 目录下定义全局布局，并通过页面级别的 layout 属性指定不同页面的布局。同时支持 default.vue 作为默认布局。通过 layout 属性可以轻松实现公共部分复用，尤其适合管理后台、门户网站等需要多层次视图管理的项目。
数据获取	asyncData 与 fetch	Nuxt.js 提供了两种数据获取方式：asyncData 用于在页面渲染前获取数据并将其注入组件中，fetch 则用于获取数据后更新组件的状态。asyncData 支持 SSR，而 fetch 可在客户端和服务端通用，结合 Vuex 状态管理，适用于需要预渲染和动态交互的场景，提升用户体验和应用的性能。
API 路由与服务端开发	API 路由与服务端集成	Nuxt.js 支持通过 serverMiddleware 配置服务端中间件处理 API 路由，并与 Express、Koa 等 Node.js 框架集成。可以将 API 集成在 Nuxt 应用中，实现前后端一体化开发。ServerMiddleware 在 API 设计、身份验证、日志记录、代理等服务端逻辑处理上非常灵活，适合全栈应用开发。
		Nuxt.js 提供了灵活的插件系统，可以在应用启动时运行自定义代码，常用于

插件与模块化扩展	插件系统与 Nuxt.js 模块化架构	引入第三方库或设置全局混入。同时，Nuxt.js 的模块化架构使其可扩展性极高，社区提供了丰富的模块，如 PWA、Google Analytics、Auth 等。模块化设计支持在 <code>modules</code> 配置中快速引入和集成功能，提升开发效率和应用功能。
国际化支持	内置国际化与本地化支持	Nuxt.js 支持通过 <code>nuxt-i18n</code> 模块实现国际化，支持动态切换语言和区域化设置。基于 URL 或 Cookie 自动识别用户的语言偏好，并加载对应的语言包。结合静态生成功能，Nuxt.js 可以根据不同的语言预生成静态页面，适合需要多语言支持的全球化应用。
SEO 优化	动态 Meta 管理与页面优化	Nuxt.js 通过 <code>head</code> 属性可以动态管理每个页面的 <code><head></code> 元素，动态设置标题、描述、关键词等 SEO 相关的 Meta 标签。结合 SSR 和静态生成，Nuxt.js 提供了极佳的 SEO 优化效果。对于内容动态变化的页面，可以使用 <code>asyncData</code> 结合 API 进行动态数据渲染，确保搜索引擎抓取最新的内容。
性能优化	代码分割与懒加载	Nuxt.js 通过自动代码分割功能优化性能，每个页面只加载当前所需的 JS 代码，避免了不必要的资源加载。同时，支持对组件和路由进行懒加载，通过 <code>import()</code> 函数动态加载组件和页面资源，提升初始加载速度。对于需要处理大量图像或视频的应用，Nuxt.js 提供了内置的 <code>nuxt/image</code> 模块用于图像优化。
状态管理	Vuex 状态管理	Nuxt.js 集成了 Vuex 作为状态管理解决方案，支持模块化管理全局状态。结合 <code>nuxtServerInit</code> 方法，可以在服务端初始化时获取全局数据并注入 Vuex 仓库，适合处理复杂的应用状态和多页面共享数据场景。
开发体验	热模块替换与自动编译	Nuxt.js 提供了优秀的开发者体验，支持模块热替换（HMR），在保存文件后无需刷新页面，保持组件状态的持久性。同时，Nuxt.js 会自动编译并更新代码，结合 <code>nuxt dev</code> 实现实时反馈，提升开发效率。
PWA 支持	渐进式 Web 应用（PWA）支持	通过 <code>@nuxt/pwa</code> 模块，Nuxt.js 支持将应用升级为 PWA，包括离线访问、缓存管理、推送通知等功能。PWA 提升了移动设备上的用户体验，允许用户在离线时继续使用应用，并通过 Service Worker 实现资源预缓存和性能优化，适合构建响应迅速且高效的移动 Web 应用。
静态资源管理	<code>static</code> 与 <code>assets</code> 目录管理	Nuxt.js 提供了 <code>static</code> 和 <code>assets</code> 两个目录用于管理静态资源。 <code>static</code> 目录中的文件会直接映射到根目录，可以通过 <code>/static</code> 路径直接访问，而 <code>assets</code> 目录主要用于存放未编译的资源（如 SCSS、图片等），通过 webpack 处理后用于组件中。静态资源可以通过 CDN 加速，提升全站加载速度。
中间件与守卫	路由中间件与页面守卫	Nuxt.js 支持全局和页面级别的中间件，通过 <code>middleware</code> 属性可以定义路由守卫，用于控制页面访问权限和验证用户身份。中间件在页面渲染前执行，适合身份验证、权限控制等逻辑。结合 Nuxt.js 的插件系统，可以实现灵活的身份验证与权限管理机制，常用于保护后台管理系统和用户隐私数据。
TypeScript 支持	TypeScript 原生支持与类型安全	Nuxt.js 原生支持 TypeScript，开发者可以通过 <code>@nuxt/typescript-build</code> 模块启用 TypeScript，确保在开发过程中进行类型检查和错误提示，提升代码的可靠性和维护性。结合 Vuex 的类型支持，Nuxt.js 应用中的全局状态管理和 API 数据处理可以实现全面的类型安全。
部署与集成	Vercel、Netlify 与 Docker 部署支持	Nuxt.js 支持多种部署方式，可以通过 Vercel、Netlify 等平台实现一键部署，结合 CI/CD 自动化流程，简化部署操作。Nuxt.js 还支持通过 Docker 镜像进行自托管部署，适合需要更高安全性和定制化的生产环境。在生产模式下，Nuxt.js 会自动进行代码优化和性能调优，确保应用的高效运行。
安全性与 CSP 策略	内容安全策略（CSP）与防御 XSS	Nuxt.js 提供了内置的安全设置，支持通过 <code>@nuxtjs/helmet</code> 模块设置内容安全策略（CSP），防止跨站脚本攻击（XSS）。可以强制所有资源通过 HTTPS 加载，并限制外部脚本的来源。结合 SSR，Nuxt.js 在页面渲染时可

	攻击	以有效防止注入攻击，确保用户数据的安全性，适合金融、医疗等对安全性要求较高的应用。
SSG 与 SSR 混合渲染	静态生成与服务端渲染混合使用	Nuxt.js 支持在同一应用中灵活使用 SSG（静态站点生成）与 SSR（服务端渲染）模式，开发者可以根据页面的需求选择合适的渲染方式。例如，首页可以使用静态生成提高加载速度，而某些动态页面则使用 SSR 保证数据的实时性，结合缓存策略最大化提升性能和 SEO。
缓存与性能优化	缓存策略与页面性能优化	Nuxt.js 通过内置的缓存策略和懒加载机制优化页面性能。可以通过 <code>cache-control</code> 头部设置静态资源的缓存过期时间，并结合 CDN 实现全局加速。同时，支持使用 Webpack 的持久化缓存功能，减少每次编译时的打包时间。对于需要处理大量图像的应用，Nuxt.js 提供了 <code>@nuxt/image</code> 模块，支持自动图像优化和懒加载。
SSR 性能优化	缓存和持久化服务端渲染	Nuxt.js 在 SSR 模式下支持通过 <code>nuxtServerInit</code> 方法实现全局数据的预获取，并且可以通过 Redis、Memcached 等缓存方案对渲染结果进行缓存，减少服务端负载。同时，Nuxt.js 可以通过负载均衡和自动扩展服务器实例来支持高并发访问，适合电商、社交平台等高流量应用。
模块联邦与微前端架构	Module Federation 与微前端支持	Nuxt.js 支持通过 Webpack 的 Module Federation 实现微前端架构，支持多个应用共享模块。开发者可以将不同的功能模块独立开发和部署，并通过模块联邦实现动态加载和集成，适合大型应用的拆分与维护。结合 Nuxt.js 的模块化架构设计，可以在项目中逐步引入微前端技术，提升应用的扩展性和团队协作效率。

8.React

1.Vue3和React区别

特性	Vue 3	React
1. 框架和库	渐进式 框架，官方提供完整生态（如 Vue Router、Vuex 等），统一开发体验，适合快速构建项目。	UI 库，只关注视图层，依赖社区驱动的生态（如 React Router、Redux 等），灵活性高，但架构决策需自行完成。
2. 开发模式	使用 模板语法 和 Composition API。模板语法类似 HTML，逻辑与视图分离，Composition API 提供灵活复用。	基于 JSX，逻辑与视图紧密结合，通过 Hooks 管理状态和副作用，强调函数式编程，简洁但学习曲线较陡。
3. 响应式系统	基于 Proxy，深度响应式，自动追踪对象和数组的变化，动态新增/删除属性性能高效。	使用 useState 和 useReducer 显式管理状态，单向数据流，通过状态更新触发组件重新渲染。
4. 性能与优化	内置编译优化（如静态提升、缓存）和响应式依赖管理，渲染性能高效，自动化优化较多。	基于 Fiber 架构，支持任务中断与恢复，但需要手动使用 React.memo 等进行性能优化。
5. 生态与工具	官方提供完整工具链（如 Vue Router、Vuex、Vite），集成良好，开发体验一致。	社区提供丰富的第三方工具（如 Next.js、Redux），自由度高，开发者需自行选择和集成。
6. 学习曲线	模板语法直观，上手简单，Composition API 适应复杂项目，学习曲线平滑。	JSX 和 Hooks 强调函数式编程，灵活但需理解状态管理和性能优化，学习曲线较为陡峭。

1.React 10个 hooks

规则：

- 只能在函数组件或自定义 Hooks 中使用 Hooks，不能在普通的 JavaScript 函数中使用。
- Hooks 必须在组件的最顶层调用，**不能在循环、条件判断或嵌套函数中使用。

Hook 名称	用途	适用场景
1. <code>useState</code>	用于在函数组件中管理状态，返回当前状态和更新状态的方法。	适合处理局部状态，如表单输入、计数器等。
2. <code>useEffect</code>	用于处理副作用，如数据获取、订阅、手动操作 DOM。依赖数组控制副作用执行时机。	等效于类组件中的 <code>componentDidMount</code> 、 <code>componentDidUpdate</code> 和 <code>componentWillUnmount</code> ，适合处理副作用逻辑。
3. <code>useContext</code>	用于消费 <code>Context</code> 数据，避免通过 <code>props</code> 在多层组件中传递数据。	适合组件树中多个组件共享数据的场景，如主题、认证等全局数据。
4. <code>useReducer</code>	适用于复杂状态逻辑，通过不同的动作来更新状态，类似于 <code>Redux</code> 的 <code>reducer</code> 。	适合有多个状态更新逻辑或状态管理较复杂的场景，如表单处理或复杂状态管理。
5. <code>useCallback</code>	用于缓存函数定义，避免不必要的函数重新创建。	适合传递函数给子组件时，避免因为父组件的重新渲染而导致子组件重复创建回调函数。
6. <code>useMemo</code>	用于缓存计算结果，避免每次渲染时重复执行开销较大的计算。	适合需要优化性能的场景，如依赖重计算的数据处理或复杂的计算。
7. <code>useRef</code>	用于访问 DOM 元素或在组件生命周期内保存可变数据， <code>ref</code> 变化不会触发组件重新渲染。	适合保存不需要触发更新的数据，或操作 DOM 元素时，如访问表单元素、保存计时器等。
8. <code>useImperativeHandle</code>	用于自定义暴露给父组件的 <code>ref</code> ，通常与 <code>forwardRef</code> 一起使用。	适合通过 <code>ref</code> 控制子组件行为的场景，如控制子组件的外部方法调用。
9. <code>useLayoutEffect</code>	与 <code>useEffect</code> 类似，但会在所有 DOM 变更后同步执行，确保在浏览器绘制前进行 DOM 操作。	适合需要在 DOM 渲染之前读取布局信息或同步修改 DOM 的场景，如处理动画或测量 DOM 元素大小。
10. <code>useDebugValue</code>	用于为自定义 Hook 在 <code>React</code> 开发者工具中添加调试信息，帮助开发者跟踪 Hook 状态。	适合在开发阶段为复杂的自定义 Hook 提供调试信息，提升调试过程的可读性。

2. `useEffect`-模拟生命周期钩子

类组件生命周期钩子	<code>useEffect</code> 模拟方式
<code>componentDidMount</code>	<code>useEffect(() => { /* 逻辑 */ }, []);</code> 传入空数组作为依赖，副作用仅在组件挂载时执行一次。
<code>componentDidUpdate</code>	<code>useEffect(() => { /* 逻辑 */ }, [dependency]);</code> 依赖数组中传入要监测的状态，副作用在指定状态变化时执行。
<code>componentWillUnmount</code>	<code>useEffect(() => { return () => { /* 清理逻辑 */ }; }, []);</code> 通过返回清理函数，副作用在组件卸载时执行清理操作。

总结：

- **省略依赖数组**：副作用在每次组件更新时都会执行。
- **传入空数组**：副作用只在 **组件挂载** 时执行，模拟 `componentDidMount`。
- **传入依赖数组**：副作用在指定依赖变化时执行，模拟 `componentDidUpdate`。
- **返回清理函数**：在组件卸载时执行清理，模拟 `componentWillUnmount`。

3. `useCallback` 和 `useMemo`区别

`useCallback` 缓存的是函数，`useMemo` 缓存的是值。

特性	useCallback	useMemo
缓存对象	缓存 函数。	缓存 计算结果。
使用场景	当需要缓存函数，避免子组件因为接收到新的函数 <code>props</code> 而不必要地重新渲染。	当需要缓存计算结果，避免每次渲染时重复执行耗时的计算。
返回值	返回 缓存的函数，只有在依赖项变化时，函数才会被重新创建。	返回 缓存的值，只有在依赖项变化时，值才会被重新计算。
适用场景示例	用于 优化回调函数，尤其是当回调函数作为 <code>props</code> 传递给子组件时，避免子组件重复渲染。	用于 优化性能开销较大的计算，如复杂的数据处理或依赖特定值的计算，避免不必要的重复计算。
关注点	关注 函数的重用，确保函数只在依赖项变化时重新创建。	关注 值的重用，确保值只在依赖项变化时重新计算。

总结：

- **useCallback**：适用于需要缓存 函数 的场景，特别是当该函数作为 `props` 传递给子组件时，能避免子组件因函数重新创建而不必要地重新渲染。
- **useMemo**：适用于需要缓存 计算结果 的场景，能优化性能，避免每次渲染时重复执行性能开销较大的计算操作。

4.setState同步还是异步

同步中是异步，异步中同步

- 当在 React 事件处理函数中调用时，`setState` 是同步的，立即更新组件。
- 在异步操作（如 `setTimeout`、`fetch` 回调等）中调用时，`setState` 可能是异步的，React 会延迟更新以优化性能。

5.Fiber架构个方面

Fiber 是一种增量渲染机制，能够将渲染工作拆分为多个小任务，从而支持任务的中断、暂停、恢复和取消，尤其适合复杂、耗时的 UI 更新。

内容	详细说明
1. 背景	React 15 中的 Reconciliation 阶段是同步执行的，导致更新操作可能会阻塞 UI 渲染。
2. Fiber 结构	Fiber 节点 代表了虚拟 DOM 中的一个节点。每个 Fiber 节点包含以下属性： 1. element : 描述节点的 JSX 元素。 2. return : 指向父 Fiber。 3. child : 指向第一个子 Fiber。 4. sibling : 指向下一个兄弟 Fiber。 5. pendingProps : 挂起的属性。 6. memoizedProps : 上一次的属性。 7. updateQueue : 更新队列。 8. effectTag : 当前 Fiber 需要的副作用类型（如插入、删除、更新）。
3. 协调 (Reconciliation)	Reconciliation 过程分为 Render 阶段 和 Commit 阶段 : 1. Render 阶段 : 构建 Fiber 树，计算要更新的内容。 2. Commit 阶段 : 应用计算结果，更新实际 DOM。
4. 调度 (Scheduling)	调度器 决定了何时执行 Fiber 更新。 优先级调度 机制按任务重要性排序，确保高优先级任务优先处理。主要策略包括： 1. 同步任务 : 优先级最高，立即执行。 2. 过渡任务 : 中等优先级，适用于动画和过渡效果。 3. 低优先级任务 : 最低优先级，适用于延迟更新。
5. 时间分片 (Time Slicing)	时间分片 使得 React 可以将长时间的渲染任务拆分成多个小块，以避免阻塞主线程。通过 requestIdleCallback 和 setTimeout 调度任务，提升 UI 响应性。
6. 中断 (Interruptions)	中断机制 允许 React 暂停处理低优先级任务，优先处理高优先级任务。Fiber 使用 scheduler 进行任务调度和中断。
7. 更新链 (Update Queue)	每个 Fiber 节点维护一个 Update Queue ，记录所有的更新操作。更新链包括： 1. Update 对象 : 包含挂起的操作和优先级信息。 2. Work In Progress : 表示正在进行中的 Fiber 节点的工作。
8. Effect Tag	Effect Tag 记录 Fiber 节点在 Commit 阶段 的副作用，如 PLACEMENT （插入）、 DELETION （删除）等。

React 重写 **requestIdleCallback** 的原因

1. **requestIdleCallback** 的问题

虽然 **requestIdleCallback** 允许开发者在浏览器的空闲时间内执行任务，但它存在以下几个问题：

- 1. **不可预测性**: **requestIdleCallback** 依赖于浏览器的空闲时间，开发者无法准确预测何时会有空闲时间。这使得任务执行的时间和频率变得不确定，尤其在高性能要求的应用中表现不佳。
- 2. **浏览器支持问题**: **requestIdleCallback** 并非在所有浏览器中都表现一致，尤其是在低性能设备上，它的表现可能更糟糕，导致任务调度的稳定性较差。
- 3. **任务超时问题**: **requestIdleCallback** 提供的任务可能会被浏览器推迟甚至超时，特别是当页面繁忙时，低优先级的任务可能很久都无法执行。

2. React 重写 **requestIdleCallback**

React 团队在 Fiber 架构中为了克服 **requestIdleCallback** 的这些局限性，重新实现了自己的任务调度系统，主要原因包括：

- 1. **更精细的任务调度控制**: Fiber 的调度系统允许 React 更精准地控制任务的优先级, 并保证高优先级任务的即时响应, 而低优先级任务可以被分片处理或延迟执行。这让 Fiber 在高性能场景下比 `requestIdleCallback` 更高效。
- 2. **可预测的执行模型**: 通过 Fiber 的任务调度机制, React 可以明确控制每个任务的执行顺序和频率, 确保高优先级任务不会被阻塞, 且不会依赖浏览器的不可预测空闲时间。
- 3. **一致的跨浏览器表现**: React 自己实现的调度系统不依赖于浏览器原生的 `requestIdleCallback`, 保证了在不同浏览器和设备上的一致性, 特别是在低性能设备上也能保持稳定表现。

6.React项目性能优化点

序号	性能优化项	具体项目及优化场景	优化措施及重点内容
1	应用结构优化	电商平台管理后台项目: 该项目包括多个模块, 如订单管理、商品管理、用户管理。首屏加载时间较长, 用户体验较差。	1. 通过 <code>Code Splitting</code> 拆分模块, 使用 <code>React.lazy</code> 和 <code>Suspense</code> 实现按需加载。 2. 仅加载首屏需要的模块, 其他模块延迟加载, 显著缩短首屏加载时间。
2	虚拟化长列表	大数据分析平台项目: 该项目需展示数千条数据的长列表, 用户滚动时遇到明显的卡顿现象。	1. 使用 虚拟滚动技术 (如 <code>react-window</code>、<code>react-virtualized</code>) 优化长列表的渲染, 只渲染可见区域的数据。 2. 通过虚拟滚动大大减少 DOM 渲染量, 提升滚动和页面性能。
3	避免重复渲染	CRM 系统项目: 父组件状态更新时, 导致子组件不必要的重复渲染, 影响了整体性能。	1. 使用 <code>React.memo</code> 缓存子组件, 防止不必要的重新渲染。 2. 使用 <code>useMemo</code> 和 <code>useCallback</code> 缓存计算结果和回调函数, 减少组件内的性能损耗。
4	状态管理优化	企业级后台管理系统: 使用 <code>Redux</code> 进行状态管理, 但每次状态变化时, 全局状态更新影响了大量不相关组件的渲染。	1. 将 <code>Redux</code> 状态管理范围限制在必要组件, 避免全局状态更新导致的大量不必要的组件渲染。 2. 使用 <code>React Context</code> API 或局部状态管理解决跨组件通信问题, 减少 <code>Redux</code> 的依赖。
5	懒加载图片和组件	内容管理系统: 页面有大量图片和非首屏组件, 导致首屏加载时间较长。	1. 使用 <code>react-lazyload</code> 实现图片的懒加载, 避免一次性加载所有图片, 只加载可视区域内的图片。 2. 对非首屏组件进行延迟加载, 提高页面初始渲染速度。
6	合理使用 <code>useEffect</code>	用户信息管理系统: 某组件依赖过多, 导致每次状态变更时 <code>useEffect</code> 被频繁触发, 执行冗余的副作用操作, 影响页面性能。	1. 优化 <code>useEffect</code> 的依赖项, 确保仅在依赖项变更时才触发副作用。 2. 将副作用拆分成独立逻辑, 减少冗余计算, 避免不必要的资源占用。
7	DOM 节点合并	金融报表系统: 页面渲染复杂, 包含大量嵌套的 DOM 结构, 导致渲染性能较差, 用户交互不流畅。	1. 使用 <code>Fragment</code> 减少不必要的 DOM 节点, 简化页面结构。 2. 减少 DOM 层级嵌套, 降低浏览器渲染树的复杂度, 提升页面性能。

8	避免匿名函数	电商前台系统：在 JSX 中直接声明匿名函数，导致父组件更新时，子组件频繁触发重新渲染，导致性能下降。	1. 避免在 JSX 中直接声明匿名函数，将函数定义在组件外部，确保函数引用不变，防止不必要的重新渲染。
9	路由优化	在线教育平台：该平台包含大量课程页面，路由模块较多，导致初次加载时加载所有页面的代码，影响性能。	1. 使用 React Router 的 loadable 实现路由懒加载，仅在用户访问时加载相应路由模块。 2. 将不常用页面的路由延迟加载，减少初始 JavaScript 包的体积。
10	样式优化	移动端电商平台：项目中使用大量动态样式，样式每次渲染时都会重新计算，导致渲染性能下降。	1. 避免动态样式计算，使用静态样式表或 CSS-in-JS 方案，提高渲染性能。 2. 提前计算并缓存需要的样式，避免每次渲染时重复计算。
11	静态资源优化	新闻门户网站：大量的图片、字体等静态资源，导致页面加载较慢。	1. 使用 CDN 加速静态资源加载，将图片、字体、CSS 文件等通过 CDN 加载，减少服务器压力。 2. 压缩和优化静态资源，减少资源体积，加快页面加载。
12	SSR 和 CSR 结合	社交媒体平台：初次加载页面速度较慢，用户体验不佳。	1. 使用 SSR（服务器端渲染） 生成初始 HTML 内容，减少首次白屏时间。 2. 页面交互和后续切换使用 CSR（客户端渲染） ，结合 SSR 和 CSR 提升首屏加载速度和交互性能 。
13	减少 JavaScript 体积	企业管理系统：JavaScript 文件较大，导致页面加载和执行速度较慢。	1. 使用 Tree Shaking 移除未使用的代码，减少 JavaScript 包体积。 2. 使用 Terser 或 UglifyJS 等工具对代码进行压缩，减小文件体积，提升加载和执行速度。
14	Service Worker 优化	PWA 应用：希望通过 Service Worker 提升应用的离线体验和加载速度。	1. 使用 Service Worker 缓存静态资源和接口数据，减少重复请求和网络依赖，提升离线加载速度和可用性。 2. 提前加载未来可能访问的资源，增强用户体验。
15	资源预加载	在线视频平台：平台加载视频资源较慢，用户等待时间过长。	1. 使用 Preload 和 Prefetch 优化关键资源的加载顺序，提前加载视频资源，确保视频播放时资源已准备就绪。 2. 对重要模块和大资源进行预加载，加快资源访问速度。
16	高效事件处理	聊天应用：用户在频繁输入、滚动时，出现明显的性能卡顿。	1. 使用 lodash.throttle 和 lodash.debounce 对频繁触发的事件进行节流和防抖，减少事件处理频率，避免性能浪费。
17	减少依赖包体积	文档管理系统：项目中引入了大量的第三方依赖库，导致打包体积过大，影响了页面加载速度。	1. 使用 webpack-bundle-analyzer 分析打包体积，剔除不必要的依赖，优化包体积。 2. 使用更轻量化的库替代体积较大的库，如用 date-fns 替换 moment.js ，进一步优化体积。
			1. 将应用转化为 PWA（渐进式 Web 应

18	使用 PWA	新闻阅读应用：希望应用可以在网络不稳定的环境下使用，并加快资源加载速度。	用），通过缓存静态资源和后台加载提升加载速度。 2. 提升离线可用性，确保用户即使在无网络情况下依然能够浏览已加载的内容。
----	--------	--------------------------------------	--

7.React Native

以下是进一步优化后的 React Native 核心内容总结，全面覆盖了 React Native 的关键技术点，深入剖析架构设计、性能优化、原生集成、状态管理等核心概念，以高逼格、详尽的角度呈现最优方案：

类别	核心知识点	详细说明
跨平台架构	React Native 核心架构	React Native 的架构由 JavaScript 层、Bridge 层和 Native 层构成，JavaScript 负责逻辑和 UI 渲染，Native 层执行原生组件操作，Bridge 实现 JS 和 Native 之间的通信。新版本中的 JSI (JavaScript Interface) 替代了 Bridge，提升了性能并实现了同步通信。
虚拟 DOM 与渲染	Reconciler、Fiber 与异步渲染	React Native 通过虚拟 DOM 和 Reconciler 实现高效渲染，通过 React Fiber 进行异步调度与时间切片，确保在高性能场景下不会出现 UI 卡顿。Fiber 的并发模式使得 React Native 能在计算密集型任务执行时保持流畅的用户体验。
跨平台 UI 组件	原生组件与平台特定组件	提供了大量跨平台基础组件，如 <code>View</code> 、 <code>Text</code> 、 <code>ScrollView</code> 、 <code>Image</code> 等，能够在 iOS 和 Android 上保持一致的表现。同时，可以通过 <code>Platform</code> API 判断当前平台，从而渲染特定的原生组件，实现针对 iOS 和 Android 的定制化开发。
性能优化	减少 Re-render、FlatList 优化	使用 <code>shouldComponentUpdate</code> 、 <code>React.memo</code> 、 <code>PureComponent</code> 避免不必要的渲染，特别是对于高频更新的组件。 <code>FlatList</code> 是处理长列表的最佳实践，支持惰性加载、分页、窗口回收等特性，确保长列表渲染时的内存占用最小化。
动画与交互	Animated API、Reanimated 与 LayoutAnimation	<code>Animated</code> 和 <code>Reanimated</code> 是 React Native 中的高性能动画库，前者通过优化 JS-to-Native 通信，后者通过直接在 Native 线程上执行动画。 <code>LayoutAnimation</code> 提供了简单的页面过渡动画支持，适合布局变化场景的动画处理，减少代码量且提升用户体验。
导航管理	React Navigation、原生导航与动态路由	React Navigation 是 React Native 中最流行的导航库，支持栈导航、Tab 导航、Drawer 导航等，同时可以通过 <code>react-navigation-stack</code> 与原生导航系统（如 <code>UINavigationController</code> 、 <code>Fragment</code> ）进行深度集成，满足复杂场景下的导航需求，并支持基于动态路由的导航管理。
状态管理	Redux、MobX、React Hooks 与 Context	在复杂项目中，Redux 和 MobX 是最常见的状态管理工具，前者基于严格的状态流控制和中间件扩展，适合大型应用。React Hooks（如 <code>useState</code> 、 <code>useReducer</code> 、 <code>useContext</code> ）提供了更简洁的状态管理方式，结合 Context 可以实现无需中间库的全局状态管理。
数据持久化与同步	AsyncStorage、本地数据库（SQLite、Realm）与数据同步	<code>AsyncStorage</code> 是 React Native 提供的简单键值存储 API，适合存储小型用户数据。对于复杂的存储场景，可以集成 SQLite 或 Realm 数据库，实现更高效的数据操作。同时可以结合 <code>redux-persist</code> 实现 Redux 的持久化状态管理，保证应用在重启后数据的保持与同步。
通信与原生集成	NativeModules、JSI 与 Native-to-JS 通信	通过 <code>NativeModules</code> ，可以将原生功能（如相机、蓝牙、文件系统等）封装成 JS 模块，供 React Native 调用。JSI (JavaScript Interface) 是新一代通信机制，取代传统的 Bridge，通过提供直接调用原生方法的能力，极大地提高了 JS 和 Native 通信的性能，降低通信延迟。
网络请求与数据处理	Fetch、Axios 与 WebSocket 实时通信	React Native 支持通过 <code>fetch</code> 或 <code>Axios</code> 进行网络请求，后者支持更多配置和拦截器处理。对于实时数据传输和双向通信， <code>WebSocket</code> 是常用解决方案，适合即时通讯、游戏等需要实时更新的数据场景。也可以结合 GraphQL 进行高效的数据查询和操作。
多线程与异步处理	Worker 线程、多线程与异步任务处理	React Native 中，主线程用于 UI 渲染，逻辑密集型任务可以通过 <code>react-native-threads</code> 开启多线程处理，避免阻塞主线程。同时 <code>InteractionManager</code> 用于处理复杂交互时的任务调度，确保 UI 响应优先，提升用户体验。对于后台任务，可以结合 <code>react-native-background-fetch</code> 实现定时任务调度。
原生模块开发	iOS 与 Android 原生模块集成	在 React Native 中，可以通过 Objective-C/Swift 编写 iOS 原生模块，或通过 Java/Kotlin 编写 Android 原生模块，提供平台特定功能，并通过 <code>NativeModules</code> 暴露给 JavaScript 调用。使用 <code>react-native-create-library</code> 可以快速创建跨平台原生模块。
调试与测试	Flipper、Jest、Detox 自动化测试	Flipper 是 React Native 官方推荐的调试工具，支持网络请求、日志、布局检查等功能。Jest 是推荐的单元测试框架，支持 Mock 数据和组件渲染测试。Detox 是用于 E2E 测试的工具，可以模拟真实用户操作，验证应用功能的完整性和稳定性。
热更新与动态发布	CodePush 动态更新与 OTA 更新	通过微软的 CodePush 服务，可以在不经过 App Store/Google Play 审核的情况下实现代码热更新，直接将更新推送到用户设备。CodePush 适用于小规模代码更新，如 UI 修复、Bug 修复，但对于需要原生代码变更的情况，仍需通过原生应用商店发布完整版本更新。
	Metro Bundler、	

打包与构建	Hermes 引擎优化启动速度与内存占用	Metro 是 React Native 的默认打包工具，支持高效的增量编译。Hermes 是为 React Native 优化的轻量级 JavaScript 引擎，通过字节码执行、即时编译等技术，显著提升了应用启动速度和内存占用，特别适合资源有限的 Android 设备。Hermes 引擎目前主要应用于 Android，但 iOS 支持也在逐步完善中。
跨平台 API 与平台特性	Platform API、文件名规则与平台差异化处理	React Native 通过 Platform API 判断当前运行平台，并根据平台执行特定代码逻辑。同时，可以通过 .ios.js 或 .android.js 文件名编写平台特定的代码，解决不同平台的兼容性问题。例如在 iOS 中使用 ActionSheetIOS ，在 Android 中使用 ToastAndroid 。
国际化与本地化	i18n、react-intl、Date 和 Number 格式化	React Native 支持通过 i18n-js 或 react-intl 实现应用的国际化和本地化处理。i18n-js 提供简单的国际化功能，通过配置不同语言的翻译文件实现语言切换。 react-intl 则支持更高级的国际化需求，如日期、货币、数字的本地化格式化，能够为用户提供更加本地化的体验。
安全与加密	数据加密、HTTPS 和证书管理	React Native 应用中，确保数据传输的安全性至关重要，所有网络请求应使用 HTTPS。同时可以通过 react-native-keychain 或 SecureStore 将敏感数据（如密码、令牌）加密存储在设备中。在数据传输方面，建议使用 JWT 或 OAuth2 进行身份验证，确保用户数据的安全性和隐私保护。
高级图形与可视化	Skia、WebGL、高性能图形渲染与数据可视化渲染的应用	React Native 支持通过 Skia 或 WebGL 进行高性能图形渲染，特别适用于游戏、3D 图形和数据可视化等场景。Skia 是 Google 提供的 2D 图形库，具有极高的渲染效率。WebGL 是一种跨平台的图形 API，可以结合 expo-gl 在 React Native 中使用，适用于需要复杂图形

进程管理与内存优化	内存管理、多线程与后台任务处理	React Native 中，确保应用的内存使用得当对性能至关重要。使用 InteractionManager 优化高优先级任务，使用 requestIdleCallback 处理低优先级任务。在内存敏感的场景下（如 Android 低端设备），需要重点关注 FlatList 的内存管理，合理配置缓存策略与分页加载。此外，通过 react-native-background-fetch 可以实现后台任务的高效调度，确保应用在后台运行时不会过度消耗设备资源。

8.Next.js服务端渲染

类别	核心知识点	详细说明
渲染模式	服务端渲染（SSR）	Next.js 的核心特性之一是服务器端渲染。通过 getServerSideProps ，在每次请求时从服务器获取数据并渲染页面，适合需要动态数据的场景。SSR 优势在于初始页面加载时获取 SEO 友好的完整 HTML，且能保证内容的实时性，但相较于静态生成，性能负担较高。
静态生成	静态站点生成（SSG）	通过 getStaticProps 和 getStaticPaths 实现静态页面生成，适合内容变化不频繁且需要高性能的场景。SSG 会在构建时生成 HTML 文件，提供极高的加载速度和 SEO 友好性。可通过 ISR（增量静态生成）结合实现部分内容的动态更新，避免频繁重建。
增量静态生成	Incremental Static Regeneration（ISR）	Next.js 独特的特性之一，允许在不重新构建整个应用的情况下，逐步更新静态生成的页面。通过 revalidate 参数，页面可以在后台自动更新，保持静态生成的高性能同时确保内容实时更新，适合博客、新闻等动态更新频率较高的应用。
混合渲染	混合渲染模式（SSG 与 SSR 结合）	Next.js 支持同一应用内使用 SSG 和 SSR 混合渲染模式，可以根据具体页面需求选择合适的渲染方式。例如，首页使用 SSG 提高加载速度，动态数据页面使用 SSR 确保数据实时性，最大限度地提升应用的性能与用户体验。
路由管理	文件系统路由	基于文件系统的自动化路由管理，开发者无需手动配置路由。通过在 pages 目录下创建 .js 文件，即可生成对应的 URL 路由。支持嵌套路由、动态路由和 catch-all 路由，灵活性高。
API 路由	内置 API 路由支持	Next.js 提供了 API 路由，通过在 pages/api 目录下创建文件，可以轻松构建服务端 API，无需额外配置。每个 API 路由都是一个独立的 Node.js 函数，适合处理服务器端逻辑、数据库连接、身份验证等操作，实现前后端一体化开发。
动态路由	动态路由与 Catch-all 路由	支持基于文件名的动态路由，例如 [id].js ，用于动态页面的生成。 getStaticPaths 可用于生成静态路径的静态页面， getServerSideProps 用于动态渲染。 Catch-all 路由（如 [...slug].js ）可处理多个动态路径，适合处理多层次路径结构的场景。
数据获取	getStaticProps 、 getServerSideProps	getStaticProps 用于静态生成页面时获取数据， getServerSideProps 用于每次请求时获取数据并进行服务器端渲染， getStaticPaths 用于生成静态路径，三者结合

	和 <code>getStaticPaths</code>	可实现灵活的数据获取模式，满足不同业务场景下的页面数据需求。
API 数据获取与缓存	<code>SWR</code> 与 <code>react-query</code> 数据管理	在客户端数据获取时，Next.js 推荐使用 <code>SWR</code> 或 <code>react-query</code> 实现数据的高效管理与缓存。 <code>SWR</code> 提供了自动缓存、自动重试、数据失效重新验证等特性， <code>react-query</code> 则是一个更强大的数据管理库，支持数据缓存、预取、分页等功能。两者都支持对 API 数据进行优化管理。
Image 优化	Next.js Image 组件	内置的 <code>next/image</code> 组件支持自动图像优化、懒加载和自适应图像大小，帮助开发者减少页面加载时间。图像会根据屏幕分辨率和设备类型动态调整大小，并通过 WebP 等格式优化，极大地提升性能和用户体验，特别适合电商类应用或需要大量图像展示的页面。
CSS 处理	CSS Modules、全局样式与 <code>styled-jsx</code>	Next.js 支持 CSS Modules，实现了模块化的样式处理，确保样式的作用范围仅限于组件内部。还支持全局样式和 <code>styled-jsx</code> ，开发者可以使用 <code>styled-jsx</code> 或引入 <code>styled-components</code> 等库，来实现组件级别的样式隔离和动态样式生成，适合开发复杂应用时的样式管理。
性能优化	代码分割与懒加载	Next.js 提供了自动的代码分割功能，每个页面只加载当前所需的 JS 代码，避免了不必要的资源加载。同时，支持对组件进行懒加载，通过 <code>React.lazy</code> 和 <code>Suspense</code> 实现按需加载，提高页面渲染速度，减少初始加载时间，适合大型应用的性能优化。
国际化支持	内置国际化与区域化支持	Next.js 原生支持国际化，通过 <code>i18n</code> 配置可以轻松实现多语言网站开发。支持基于 URL 的语言切换和动态加载不同语言包，同时也提供了区域化设置（如日期、货币格式等）的自动处理，适合需要面向全球用户的应用。
开发体验	Fast Refresh 实时热更新与自动编译	Next.js 提供了优秀的开发者体验，Fast Refresh 功能支持组件级别的热更新，在保存文件后无需刷新整个页面，保证状态的持久性。同时，Next.js 会在开发过程中自动编译和更新代码，帮助开发者提高开发效率。
API 路由与数据库集成	Prisma、Mongoose、TypeORM 等数据库集成	Next.js 支持与多种数据库 ORM 集成，如 Prisma、Mongoose、TypeORM，开发者可以通过 API 路由直接与数据库交互，实现数据存储与管理。Prisma 提供了强大的类型安全查询生成器，适合大型应用的复杂数据操作，而 Mongoose 和 TypeORM 则更适合中小型项目。
静态文件处理	<code>public</code> 目录与静态资源管理	Next.js 的 <code>public</code> 目录用于存放静态文件，如图像、字体、视频等，任何位于 <code>public</code> 目录的文件都可以通过根路径直接访问。静态资源会被高效缓存，并支持 CDN 加速，提升全站的静态资源加载性能。
SEO 优化	Meta 标签管理、动态 Head、动态路由 SEO	Next.js 支持通过 <code>next/head</code> 组件动态管理页面的 <code><head></code> 元素，实现灵活的 Meta 标签管理。可以根据不同页面动态设置标题、描述、关键词等 SEO 关键内容。对于动态路由页面，Next.js 通过 SSR 生成完整的 HTML 内容，确保搜索引擎能够抓取到最新的页面内容，提高 SEO 效果。
TypeScript 支持	TypeScript 与类型安全	Next.js 原生支持 TypeScript，开发者可以在项目中使用 TypeScript 实现类型检查，避免运行时错误。结合 React 的类型系统，Next.js 项目中的组件、API 路由和数据获取函数都能获得强大的类型提示与自动补全，提升开发效率和代码质量。
SSR 与 CSR 混合模式	SSR、CSR、SSG 的混合使用	Next.js 允许开发者在同一个应用中灵活使用 SSR、CSR（客户端渲染）和 SSG，适合复杂的业务场景。通常首页使用 SSG 或 ISR 提高性能，动态页面使用 SSR 确保内容实时更新，某些交互页面则可以使用 CSR 以提升用户体验。
错误边界与监控	Error Boundaries 与 Sentry 集成	通过 React 的 <code>Error Boundary</code> ，Next.js 提供了对应用错误的捕获与处理，确保在渲染过程中发生错误时不会导致整个应用崩溃。同时可以集成 Sentry 进行前后端的错误监控和性能跟踪，方便开发者对应用问题进行实时监控和调试。
安全与性能优化	Helmet、HTTPS、内容安全策略（CSP）	Next.js 支持通过 <code>next-helmet</code> 或 <code>next-secure-headers</code> 插件设置 HTTP 头部，添加安全相关的内容安全策略（CSP）、强制 HTTPS 和其他安全策略，防止常见的 Web 安全攻击，如 XSS、CSRF 攻击等。结合现代 Web 性能优化技术，Next.js 也能很好地处理资源缓存和懒加载，提升网站加载速度。
部署与扩展	Vercel 部署与 CI/CD 集成	Next.js 是由 Vercel 团队开发，具有与 Vercel 平台的深度集成，支持一键部署至 Vercel，同时自动实现 CI/CD 流程。每次提交代码后，Vercel 会自动构建并部署最新版本的应用。还支持通过 Docker、自托管等方式部署，适合多种部署需求。
模块化与微前端架构	Module Federation 与微前端架构支持	Next.js 支持 Webpack 的 Module Federation 特性，能够轻松实现微前端架构，支持多个应用之间的模块共享。通过微前端架构，开发者可以将不同功能模块独立部署和更新，提升应用的可维护性和扩展性，适合大型企业级应用。
前端与后端集成	全栈开发与 Serverless 函数	Next.js 支持构建全栈应用，提供了内置的 API 路由，用于处理服务器端逻辑、数据库操作等。同时与 Vercel Serverless 函数无缝集成，能够将 API 路由部署为无服务器函数，按需调用，大幅提升应用的可扩展性和灵活性，降低服务器负载。

9. 微前端

1. 为什么要使用微前端

项目背景：企业级电商平台管理系统

一个企业级的电商平台，包含多个独立子系统：订单管理、库存管理、用户管理、财务报表等。不同团队负责不同的模块，且部分子系统已经使用 Vue 2 进行开发，而新开发的模块可能使用 Vue 3 或 React。该系统要求能够独立部署、快速迭代，同时需要保障各模块的独立性、安全性和高效加载。

序号	项目需求	不使用微前端的痛点/问题	使用微前端 Qiankun 的优势	为什么必须使用微前端
1	订单管理系统与库存管理系统技术栈不同	不使用微前端时，所有团队必须统一技术栈（如 Vue 2），否则整合过程复杂且容易出现依赖冲突。不同技术团队无法独立开发。	1. Qiankun 支持不同技术栈共存，如订单管理系统使用 Vue 2，库存管理系统使用 React。 2. 各模块可以独立开发、测试和上线，互不依赖。 3. Qiankun 提供子应用隔离，避免了技术栈冲突和依赖问题，且支持现有项目的逐步升级和维护。	必须使用微前端的原因：由于各团队已使用不同的技术栈，强行统一会导致开发效率低下，且技术迁移成本巨大。微前端架构可以让不同技术栈共存，支持团队按需选择技术，提升开发效率和项目灵活性。特别是在订单管理与库存管理不同团队分工的项目中，微前端至关重要。
2	财务报表系统频繁更新，影响其他模块	不使用微前端时，系统每次更新必须整体部署，即使只更新了一个小模块，所有模块都需要重新部署。	1. 微前端允许独立部署，财务报表系统可以独立上线，不需要影响其他模块。 2. 通过 Qiankun 的独立加载机制，更新后的子应用与其他系统并行运行，支持版本回滚，极大减少了发布时的风险和复杂度。	必须使用微前端的原因：在电商平台中，财务报表系统经常需要频繁更新，而不使用微前端时，更新任何模块都会影响整个系统。通过微前端架构，财务系统可以独立部署和更新，减少了系统停机时间，提高了系统的稳定性和维护效率。
3	旧订单管理系统迁移到新技术栈	如果不使用微前端，整个订单系统必须一次性重写，迁移成本高，且过程中业务中断，无法支持新旧系统并存，影响用户体验。	1. Qiankun 支持新旧系统共存，可以逐步替换和迁移订单系统的子模块。 2. 在过渡期内，旧订单系统仍可以正常使用，同时逐步迁移至 Vue 3 或 React。新旧系统通过微前端架构平滑集成，不会出现业务中断或大规模的技术迁移冲击。	必须使用微前端的原因：订单管理系统是电商平台的核心模块，强行重构会导致系统风险增大和用户流失。使用微前端架构可以实现新旧系统的平稳过渡，逐步迁移到新技术栈，保障业务的连续性，降低技术升级的风险。
4	提升电商平台的初始加载性能	不使用微前端时，所有模块（如用户管理、订单管理、库存管理）在系统启动时同时加载，导致页面加载时间长，影响用户体验。	1. Qiankun 支持按需加载，用户管理和库存管理系统只有在用户访问时才会加载，不影响平台的首屏加载性能。 2. 通过子应用的懒加载，未使用的模块不会占用资源，主应用的初始化时间大大缩短，提升了用户首次访问的体验。	必须使用微前端的原因：电商平台通常包含多个模块，如果每次加载所有模块，系统的初始响应速度会变慢，尤其是在高并发的情况下。使用微前端架构可以显著减少初始加载资源，通过按需加载提升首屏性能，改善用户体验。尤其适用于多模块的复杂

				平台系统。
5	系统模块间的错误隔离和安全性提升	不使用微前端时，系统是单一整体，任何一个模块的错误可能导致整个应用崩溃，安全漏洞也可能波及到全局，影响整个系统的稳定性。	1. Qiankun 支持模块隔离，即使订单管理模块出错，用户管理或库存管理系统不会受到影响，保证了系统的健壮性。 2. 子系统间通过微前端进行隔离，安全漏洞不会互相影响，模块之间的运行环境互不干扰，提高了系统整体的安全性和可维护性。	必须使用微前端的原因： 电商平台通常有多个子系统，如果某个模块出现故障，整个系统可能会受影响。 微前端通过模块化隔离，可以防止错误蔓延，保证系统的稳定性和安全性。这对于需要高可靠性和高安全性的场景，如电商和金融系统，微前端是必要的选择。
6	平台模块的灵活扩展和多团队协作开发	不使用微前端时，多个团队难以并行开发，团队之间的相互依赖导致开发效率低下，增加了项目管理的复杂性。	1. 微前端支持多个团队并行开发，各团队可以独立开发自己的子系统，并通过 Qiankun 接入主应用。 2. 各团队的技术栈可以不同，独立开发、测试和部署。通过 Qiankun 的子应用注册机制，主应用与子应用的集成简单，开发流程解耦，提高了多团队的协作效率。	必须使用微前端的原因： 电商平台中，用户管理、订单管理、库存管理等模块可能由不同团队开发，如果不使用微前端，团队之间的依赖会降低开发效率。 使用微前端可以使各团队专注于各自的模块，减少依赖，提升并行开发和部署的效率，减少管理复杂度。
7	技术栈升级与平台的扩展性	不使用微前端时，技术栈的升级和系统扩展需要整体重构，技术迁移成本高、风险大，系统难以应对业务需求的快速变化。	1. Qiankun 支持渐进式技术栈迁移，无需一次性重构整个系统。 2. 不同模块可以逐步升级到 Vue 3 或 React，主应用与子应用的技术栈可以不同，技术升级和新功能添加可以并行进行，提高了系统扩展性和可维护性。 3. 新模块可以平滑地与现有系统集成，逐步替换旧模块。	必须使用微前端的原因： 不使用微前端时，技术升级或系统扩展必须重构整个系统，业务中断风险大。 使用微前端，技术栈可以逐步迁移和升级，减少大规模技术变更带来的风险，同时保持系统的可扩展性，灵活应对未来的业务变化和技术更新。

2.qiankun核心使用

1. 多子应用注册与动态加载

在一个大型项目中，假设有多个子应用，如 `vueApp`、`reactApp`、`angularApp`，我们可以对每个子应用进行独立注册与按需加载。每个子应用可以使用不同的技术栈，并根据路由规则动态加载。

配置示例：

```
// 引入 Qiankun 的相关 API
import { registerMicroApps, start } from 'qiankun';

// 注册多个子应用
registerMicroApps([
  {
    name: 'vueApp', // Vue 应用
    entry: '//localhost:8080', // Vue 子应用的入口
    container: '#vue-container', // Vue 子应用的 DOM 容器
```

```
    activeRule: '/vue', // 路由匹配 /vue 时激活
  },
  {
    name: 'reactApp', // React 应用
    entry: '//localhost:8081', // React 子应用的入口
    container: '#react-container', // React 子应用的 DOM 容器
    activeRule: '/react', // 路由匹配 /react 时激活
  },
  {
    name: 'angularApp', // Angular 应用
    entry: '//localhost:8082', // Angular 子应用的入口
    container: '#angular-container', // Angular 子应用的 DOM 容器
    activeRule: '/angular', // 路由匹配 /angular 时激活
  },
]);

// 启动微前端
start();
```

优化点：

- 每个子应用有独立的入口、容器和激活规则，确保多个子应用可以按需加载，减少主应用的初始加载压力。

2. 多个子应用的沙箱隔离

每个子应用在运行时使用独立的 **沙箱机制**，确保不同子应用的全局变量、样式、脚本相互隔离，避免冲突。可以针对每个子应用启用不同的隔离策略，保证其独立性。

优化配置示例：

```
// 启用严格的沙箱隔离机制，确保多个子应用的独立性
start({
  sandbox: {
    strictStyleIsolation: true, // 启用严格的样式隔离，防止样式污染
  }
});
```

优化点：

- **严格样式隔离**：通过沙箱机制，确保多个子应用的全局样式互不影响，尤其在不同框架（如 Vue 和 React）共存时避免样式冲突。

3. 全局状态共享与跨应用通信

多个子应用之间可以通过 Qiankun 的 **全局状态管理** 实现状态共享和跨应用通信。假设多个子应用需要共享用户信息、权限数据等，可以使用 `initGlobalState` 在主应用和子应用之间进行状态同步。

配置示例：

```
import { initGlobalState } from 'qiankun';

// 初始化全局状态
const initialState = { user: 'admin', token: 'abc123' };
const actions = initGlobalState(initialState);

// 监听全局状态变化
actions.onGlobalStateChange((state, prev) => {
  console.log('全局状态已变化', state);
});

// 在子应用中修改全局状态（比如在 vueApp 中修改用户状态）
actions.setGlobalState({ user: 'guest' });
```

优化点：

- **全局状态同步**：多个子应用可以共享全局状态，如用户信息、权限、主题等，减少重复操作和通信延迟。

4. 独立部署与多环境管理

每个子应用可以独立部署在不同的环境或服务器上，主应用通过 **动态入口** 加载子应用的最新版本。可以为每个子应用配置不同的环境变量和域名，确保其在不同环境下独立运行。

优化配置示例（Nginx 配置）：

```
# Vue 子应用的独立部署
location /vueApp/ {
  proxy_pass http://localhost:8080;
}

# React 子应用的独立部署
location /reactApp/ {
  proxy_pass http://localhost:8081;
}

# Angular 子应用的独立部署
location /angularApp/ {
  proxy_pass http://localhost:8082;
}
```

优化点：

- **多子应用独立部署**：每个子应用独立部署，可以在不同的服务器或域名下运行，方便版本管理和独立发布，尤其适用于 **频繁迭代** 和 **多团队协作** 的项目。

5. 子应用的缓存与懒加载优化

为减少网络请求和资源浪费，可以为每个子应用启用 **缓存机制** 和 **懒加载**，只有在用户访问时才加载子应用，并缓存其资源以供后续访问。

优化配置示例：

```
registerMicroApps([
  {
    name: 'vueApp',
    entry: '//localhost:8080',
    container: '#vue-container',
    activeRule: '/vue',
    props: { cache: true }, // 启用缓存，减少重复加载
  },
  {
    name: 'reactApp',
    entry: '//localhost:8081',
    container: '#react-container',
    activeRule: '/react',
    props: { cache: true }, // 启用缓存
  },
]);

// 启动微前端时启用懒加载
start();
```

优化点：

- **懒加载与缓存**：子应用只有在被访问时才会加载，首次加载后将资源缓存下来，后续访问时无需重新加载，极大提升性能。

6. 子应用的生命周期管理与性能优化

通过生命周期钩子，可以精确控制多个子应用的加载、挂载和卸载行为。你可以在特定的生命周期阶段进行资源初始化或清理操作，优化内存使用。

优化配置示例：

```
const vueAppLifecycles = {
  beforeMount: () => console.log('Vue 子应用挂载前'),
  mount: () => console.log('Vue 子应用挂载完成'),
  unmount: () => console.log('Vue 子应用卸载完成'),
};

const reactAppLifecycles = {
  beforeMount: () => console.log('React 子应用挂载前'),
  mount: () => console.log('React 子应用挂载完成'),
  unmount: () => console.log('React 子应用卸载完成'),
};
```

```
// 注册子应用时绑定生命周期
registerMicroApps([
  { name: 'vueApp', entry: '//localhost:8080', container: '#vue-container', activeRule:
'/vue', lifeCycles: vueAppLifeCycles },
  { name: 'reactApp', entry: '//localhost:8081', container: '#react-container',
activeRule: '/react', lifeCycles: reactAppLifeCycles }
]);

start();
```

优化点：

- **生命周期管理**：可以在子应用加载前和卸载后清理不必要的资源，避免内存泄漏，确保系统在多个子应用切换时性能稳定。

7. 路由同步与隔离优化

多个子应用可以各自拥有独立的路由系统，主应用和子应用之间的路由可以同步，也可以完全隔离，确保主子应用的路由操作不会冲突。

优化配置示例：

```
// 加载多个子应用时，独立设置每个子应用的路由规则
const vueApp = loadMicroApp({
  name: 'vueApp',
  entry: '//localhost:8080',
  container: '#vue-container',
  activeRule: '/app/vue',
});

const reactApp = loadMicroApp({
  name: 'reactApp',
  entry: '//localhost:8081',
  container: '#react-container',
  activeRule: '/app/react',
});

// 主应用与子应用的路由系统可以同步，也可以独立运行
```

优化点：

- **路由隔离与同步**：通过路由隔离，确保各子应用的路由系统不会相互影响，必要时可以通过主应用同步控制子应用的路由导航。

3.qiankun原理

核 心 概 念	实现原理	源码解析
微		

应用注册与管理	通过 <code>registerMicroApps</code> 注册多个子应用，定义其入口、容器、激活规则和生命周期钩子。每个子应用注册后会按照指定的激活规则（如 URL 路由）进行加载与卸载。	源码中的 <code>registerMicroApps</code> 会将子应用注册信息存储在一个全局的子应用配置数组中，并监听路由变化，当路由匹配某个子应用的 <code>activeRule</code> 时，会触发该子应用的加载流程。
生命周期管理	子应用在挂载、更新、卸载的不同阶段都有对应的生命周期钩子，如 <code>beforeMount</code> 、 <code>mount</code> 、 <code>unmount</code> 等。通过这些钩子可以在子应用的生命周期中执行特定的逻辑，比如资源初始化和清理。	源码中通过调用子应用注册时的 <code>lifeCycles</code> 钩子，触发子应用的生命周期事件。例如在子应用加载时调用 <code>mount</code> ，卸载时调用 <code>unmount</code> ，确保每个阶段的资源和状态管理得到妥善处理。
沙箱隔离机制	Qiankun 采用了 Proxy 沙箱 和 快照沙箱 实现子应用之间的全局变量、样式隔离。每个子应用运行在自己的沙箱环境中，防止全局变量污染。 Proxy 沙箱 可以拦截对全局对象的访问，实现读写隔离。	源码中，沙箱机制基于 Proxy 实现。对于全局对象（如 <code>window</code> ），每次访问都会被拦截并代理到当前子应用的沙箱环境中，实现子应用间的全局变量隔离。在沙箱运行时，拦截子应用的全局变量操作。
动态加载与懒加载	通过 <code>loadMicroApp</code> 方法动态加载子应用，或通过主应用的路由激活规则懒加载子应用。子应用的资源只有在需要时才会加载，避免主应用的过度加载，提升性能。	在源码中，Qiankun 会根据 <code>entry</code> 动态加载子应用的 HTML、CSS、JS 资源，并挂载到指定的 DOM 容器中。懒加载通过 <code>activeRule</code> 判断是否需要加载资源，未激活的子应用资源不会加载。
子应用的通信机制	通过 <code>initGlobalState</code> 实现子应用之间的通信和全局状态管理。主应用和子应用可以共享全局状态，并通过 <code>onGlobalStateChange</code> 监听全局状态的变化，保持主子应用之间的状态同步。	源码中， <code>initGlobalState</code> 基于 观察者模式 实现，将状态变化通知所有订阅的子应用。每次状态变更时，都会触发 <code>setGlobalState</code> ，并通知所有通过 <code>onGlobalStateChange</code> 注册的监听器。
路由拦截与路由同步	Qiankun 通过拦截主应用的路由，判断路由是否匹配某个子应用的 <code>activeRule</code> 。如果匹配成功，Qiankun 会加载并激活对应的子应用。子应用可以拥有独立的路由系统，主应用可以同步子应用的路由状态。	源码中， <code>activeRule</code> 是通过字符串匹配、正则匹配等方式实现的。当路由变化时，Qiankun 会遍历所有注册子应用，找到匹配的子应用并加载。如果子应用的路由发生变化，主应用也可以同步更新路由状态。
沙箱快照机制	当某个子应用被卸载时，Qiankun 通过 快照沙箱 保存当前的全局状态，当该子应用再次被加载时，可以恢复之前的全局变量和样式，保持子应用状态的一致性。	源码中，在卸载子应用时，Qiankun 会将全局对象的状态保存为快照（如 <code>window</code> 、 <code>document</code> 的状态）。当子应用重新加载时，会恢复这些快照，从而保持子应用的运行环境不变。
子应用独立部署与	每个子应用可以独立开发、独立部署，主应用通过动态加载子应用的 HTML 入口文件。每个子应用的代码、样式和逻辑都是完全独立的，且子应用可以使用不同的技术栈（如 React、Vue、Angular）。	源码中，Qiankun 使用 <code>fetch</code> 请求子应用的 <code>entry</code> （入口 HTML），然后解析其中的资源链接，并通过 <code>document.createElement</code> 动态插入脚本和样式，从而实现子应用的独立运行。

运行		
性能优化	Qiankun 通过懒加载、缓存、沙箱机制等提升性能。懒加载确保未使用的子应用不会加载，缓存机制减少重复加载的资源，沙箱隔离机制减少全局污染，确保子应用高效运行。	源码中，当子应用首次加载后，会缓存子应用的静态资源。下次访问时，直接从缓存中加载，避免重复请求资源。同时，沙箱的严格隔离确保全局变量不会被滥用或污染，减少了潜在的内存泄漏和性能问题。
插件化与扩展性	Qiankun 提供了插件机制，允许开发者在子应用加载前、加载后插入自定义逻辑，或者扩展主应用与子应用的通信方式。可以灵活定制微前端的行为，适配不同的业务需求。	在源码中，通过 <code>registerMicroApps</code> 或 <code>start</code> 方法可以传入 <code>plugins</code> 参数，开发者可以通过插件机制扩展子应用的功能，执行一些额外的操作，比如日志记录、性能监控、错误处理等。

10. 提薪点-低代码

1. 收费产品

平台名称	所属公司	使用场景	框架/技术栈	特点和优势
宜搭	阿里云	企业级业务管理、流程自动化，适用于中大型企业的业务应用开发，如财务管理系统、人力资源系统、项目管理工具等。	基于 Ant Design 的前端框架，后端主要使用 Java 和 Spring Boot ，支持与阿里云其他产品集成。	拥有强大的数据管理功能和丰富的业务组件库，支持多种外部系统集成，适合复杂的企业应用场景。
飞书多维表格	字节跳动	适用于中小型企业内部的协作办公场景，如数据报表生成、简单的工作流审批、团队协作工具等。	使用 React 技术栈，后端基于字节跳动的云端服务，支持与飞书其他产品无缝集成。	界面简洁，适合轻量级业务需求，结合飞书的协作能力，提供了强大的团队协作与数据处理能力。
轻流	轻流科技	适用于中小企业的业务流程自动化、客户关系管理（CRM）、企业数据管理（ERP）等场景，尤其适合自定义表单与工作流审批场景。	前端基于 Vue.js ，后端使用 Node.js 和 Express 架构，数据库为 MySQL 。	强调表单自定义与流程自动化，用户可以通过拖拽方式快速构建业务流程，支持第三方系统集成，如企业微信、钉钉等。
APICloud	APICloud	适用于跨平台应用开发，尤其是移动端 App 的快速构建和发布。支持原生应用的开发场景，如电商、物流、教育类应用的快速迭代和部署。	基于 Vue.js 和 APICloud 自研框架，支持多种移动平台如 iOS 、 Android ，支持与主流移动 SDK 集成。	强调跨平台支持和移动端性能优化，适合需要快速开发移动端应用的场景，支持混合开发与原生性能优化。
Mingdao	明道云	适用于企业管理后台、审批系统、CRM 系统、ERP 系统等企业级应用的快速开发，强调低代码平台的定制能力。	前端基于 React ，后端使用 Node.js 和 MongoDB ，支持自定义业务逻辑与数据库操作。	强调企业级应用的定制开发，内置丰富的业务组件，支持复杂的审批和业务流程构建，且支持高度灵活的权限控制和数据可视化。

氚云	氚云科技	主要用于中小企业的轻量级应用构建，如销售管理、库存管理、项目跟踪等，提供简单易用的工作流和数据管理能力。	使用 Vue.js 技术栈，后端基于氚云自研框架，数据库为 MySQL，支持与钉钉等第三方系统集成。	提供高度可定制的表单与数据管理工具，特别适合业务流程管理、简单的业务自动化操作，强调与钉钉等企业协作平台的集成。
微搭	腾讯云	适用于微信小程序的快速开发、原型设计和企业内部管理系统开发，如 O2O 商业场景、营销推广、用户管理系统等，支持快速迭代与小程序生态深度集成。	基于 Vue.js 和 uni-app ，后端依赖腾讯云的多种服务，如数据库、对象存储、API 网关等。	与微信小程序深度集成，支持快速构建企业应用，特别适合 O2O、营销活动等小程序开发场景，依托腾讯云的基础设施实现高可扩展性和高性能。
Uino	酷云互动	适用于营销活动的快速搭建、数据采集与数据展示的场景，支持大屏展示、BI 报表、实时数据监控等场景。	前端使用 Vue.js ，后端基于 Node.js 和 MongoDB ，支持数据可视化和大屏展示。	强调数据展示和实时监控功能，特别适合需要大屏数据展示和复杂营销活动管理的场景，提供大量数据可视化模板，支持个性化定制。
帆软 FineBI	帆软科技	主要用于企业级 BI（商业智能）数据分析和报表生成，适用于业务数据驱动的场景，如销售数据分析、财务报表生成、用户行为分析等。	前端基于 React 和 AntV 数据可视化框架，后端使用 Java ，数据库为 MySQL 或 Oracle ，支持与多种外部数据源集成。	强调数据可视化和商业智能分析功能，提供丰富的数据分析模板，支持多维度数据报表生成和大数据量处理，适合大型企业的数据分析需求。

总结：

- **开源性：**目前国内主流的低代码平台大多为闭源软件，提供的主要是 **商业化产品** 和 **企业解决方案**。这些平台提供高度的可定制性和业务支持，但在源码方面不公开，用户只能在平台上进行业务开发，而无法获取或修改平台底层代码。

2. 免费产品

平台名称	所属公司	使用场景	技术栈/框架	特点和优势	Github链接
Formily	阿里巴巴	专注于复杂表单的开发，适合后台管理系统、数据驱动应用、动态表单构建。	前端：React、Vue	强大的表单生成器，支持跨框架使用，灵活的表单布局和数据绑定，支持表单动态生成和复杂的表单验证逻辑，广泛应用于企业内部系统。	Formily
Lowcode Engine	阿里巴巴	复杂组件拖拽式开发，适用于中大型企业内部系统开发，如 CRM 系统、ERP 系统、业务流程管理等。	前端：React、Vue，基于 Ant Design 组件库	功能全面，支持复杂企业应用的低代码构建，广泛应用于企业管理系统的开发，支持组件拖拽式 UI 和业务逻辑开发，集成度高，使用灵活。	Lowcode Engine
Appsmith	Appsmith 公司	企业内部应用开发，如仪表盘、业务管理工具、报表生成等，支持 API 集成和数据库管理。	前端：React，后端：Java、Spring Boot、MongoDB	强调与 API 和数据库的集成，支持多种数据源，用户可以通过拖拽快速创建复杂的业务逻辑，组件丰富，适合快速开发数据密集型应用，广泛用于企业内部工具开发。	Appsmith
Budibase	Budibase 公司	用于快速构建 CRUD 应用，适合内部管理系统、表单应用和小型企业的后台工具开发。	前端：Svelte，后端：Node.js、MongoDB、PostgreSQL	支持 CRUD 快速生成、表单生成，广泛应用于中小型企业的后台管理系统开发，操作简单，提供丰富的模板和预定义组件。	Budibase
NocoDB	NocoDB 团队	开源 Airtable 替代方案，将现有的数据库转换为智能电子表格，适合中小企业的数据库管理需求。	前端：Vue.js，后端：Node.js、Express、MySQL、PostgreSQL	将 SQL 数据库转化为电子表格，支持多种数据库，提供简单的界面和数据管理功能，广泛用于中小型企业的数据库管理系统开发，适合快速管理数据库记录。	NocoDB
Directus	RANGER Studio	提供无头 CMS 功能，适用于内容管理系统、数据管理平台，特别是有定制 API 和数据库集成需求的项目。	前端：Vue.js，后端：Node.js、Knex.js、SQLite、MySQL、PostgreSQL	强调数据管理和内容管理，支持定制 API 和多种数据库，适合构建复杂的数据管理平台，尤其适合开发者在 API 层面做自定义开发，广泛应用于内容管理系统开发。	Directus
Payload CMS	Payload 团队	高自定义内容管理系统，适用于内容管理系统、博客系统、电子商务平台等需要强大后台支持的项目。	前端：React，后端：Node.js、MongoDB	支持高度自定义的内容管理，提供强大的 API 和管理工具，适合企业和个人的内容管理需求，尤其适合电子商务和内容密集型平台，广泛用于内容管理系统开发。	Payload CMS
Tooljet	Tooljet 团队	内部应用的开发，如仪表盘、CRM 系统、数据管理平台，支持多种数据库和 API 集成。	前端：React，后端：Node.js、PostgreSQL	强调 API 集成和数据管理，支持拖拽式组件构建，适合数据密集型场景，如数据分析、报表管理、后台管理，广泛应用于企业内部的管理系统和工具开发。	Tooljet
Retool (开源部分)	Retool 团队	数据密集型企业内部工具开发，支持多种数据源和 API 集成，适合快速创建仪表盘和内部管理工具。	前端：React	尽管 Retool 是商业产品，但其部分组件开源，支持快速创建 CRUD 应用、数据管理工具和仪表盘，广泛用于企业的数据分析、管理系统开发。	Retool Open
Rowy	Rowy 团队	面向 Firebase 开发者的低代码工具，适用于构建后端应用，如数据管理、云函数编排等。	前端：React，后端：Firebase	与 Firebase 集成紧密，支持实时数据库和云函数管理，简化云端应用开发，适合云端开发者快速开发后端工具，广泛应用于 Firebase 生态中的工具开发。	Rowy
Plasmic	Plasmic 团队	前端低代码开发平台，专注于网站、营销页面和电商应用开发，提供设计师友好的页面构建器。	前端：React，后端：无	提供设计师和开发者共同协作的平台，侧重页面设计和前端组件的定制开发，适合电商网站、营销页面等需要强大设计能力的场景，广泛应用于前端和电商平台开发。	Plasmic
Saltcorn	Saltcorn 团队	数据驱动的应用开发，支持自定义的 CRUD 应用，适用于中小企业的内部工具开发。	前端：Node.js，后端：PostgreSQL	强调数据驱动的应用开发，支持自定义业务逻辑和数据管理能力，适合中小企业的内部工具开发，广泛用于构建数据管理工具和业务应用。	Saltcorn

总结：

- 广泛使用的开源低代码平台 中，阿里巴巴的 **Formily** 和 **Lowcode Engine** 是国内大公司中广泛使用的低代码开发工具，适用于复杂的企业内部应用开发。
- Appsmith**、**Budibase** 和 **NocoDB** 等平台凭借其快速构建 CRUD 应用和集成 API 的能力，被大量中小企业使用。
- Tooljet**、**Directus**、**Payload CMS** 等平台适用于需要高度自定义和数据管理的 enterprise 应用，广泛用于数据密集型场景和内容管理系统开发。

3.React低代码核心组件库

核心组件实现思路总结：

1. 组件化开发：每个核心组件均通过高度可配置的 `props` 实现用户自定义内容、交互和样式，确保组件具备灵活性和复用性。
2. 动态数据绑定与联动：表单、图表等组件需要支持动态数据绑定和联动逻辑，确保在复杂业务场景中能够根据用户输入和业务逻辑进行动态渲染。
3. 拖拽与布局灵活性：拖拽容器、布局组件通过使用拖拽库和布局系统，支持用户自由组合页面结构，提升页面构建效率和灵活性。
4. 响应式设计 with 权限控制：响应式设计组件确保页面在不同设备下有良好的显示效果，权限控制组件则保障了不同角色用户的访问限制。

核心组件名称	组件描述	实现思路（细化）
1. 表单组件	提供基础的表单元素，如输入框、选择框、日期选择器、单选框、复选框等，支持表单校验、联动逻辑和动态生成。	1. 封装基础表单元素，如 <code>Input</code>、<code>Select</code>、<code>DatePicker</code> 等，允许通过 <code>props</code> 动态配置属性（如默认值、校验规则、禁用状态）。 2. 使用 <code>onChange</code> 事件进行表单元素的实时状态更新，确保联动性。 3. 使用 <code>FormContext</code> 管理全局表单状态，支持表单字段联动、条件渲染等复杂逻辑。 4. 提供校验规则，支持同步校验和异步校验，并提供全局错误提示信息。 5. 支持表单的动态生成，允许用户通过拖拽添加字段。 6. 实现表单的提交、重置操作，通过统一 <code>API</code> 调用处理数据。
2. 布局组件	用于页面结构布局，支持多种布局方式，如网格布局、栅格布局、弹性布局，帮助用户快速搭建页面结构。	1. 封装 <code>Grid</code>、<code>Row</code>、<code>Col</code> 等布局组件，通过 <code>props</code> 控制列数、对齐方式、间距等属性。 2. 使用 <code>flexbox</code> 或 <code>CSS Grid</code> 实现响应式布局，支持通过 <code>media query</code> 自适应不同分辨率。 3. 组件支持动态调整布局，通过拖拽方式改变布局结构，记录每个组件位置及尺寸。 4. 提供网格间距、对齐方式、列数调整等属性，使布局组件适应多种页面布局需求。
3. 数据展示组件	用于展示动态数据的组件，如表格、图表、列表等，帮助用户构建可视化数据面板。	1. 封装 <code>Table</code>、<code>Chart</code>、<code>List</code> 等数据展示组件，支持动态数据绑定和显示。 2. 使用 <code>props</code> 传递数据源、列配置等属性，允许用户配置数据格式化、分页、筛选等功能。 3. 数据组件支持通过 <code>API</code> 获取数据，并动态刷新展示。 4. 对图表组件，如 <code>Echarts</code>，支持图表类型切换、数据源绑定、定时刷新等功能。
4. 按钮组件	提供基础的交互操作按钮，支持多种样式和功能，如普通按钮、提交按钮、重置按钮等，适用于表单或页面操作。	1. 封装基础按钮组件，如 <code>Button</code>，通过 <code>type</code> 属性控制按钮类型（如 <code>submit</code>、<code>reset</code>、<code>button</code>）。 2. 使用 <code>onClick</code> 事件绑定按钮的功能，支持异步操作。

		3. 支持按钮的样式自定义, 通过 <code>props</code> 配置不同的外观 (如颜色、大小、禁用状态等)。
5. 拖拽容器组件	提供拖拽功能的容器组件, 用户可以通过拖拽不同的组件到容器中, 动态生成页面布局。	1. 使用 <code>react-dnd</code> 或 <code>react-beautiful-dnd</code> 实现拖拽功能。 2. 封装 <code>DragContainer</code> 组件, 允许用户拖拽不同的组件到容器中, 并通过状态管理记录拖拽操作。 3. 提供组件位置记录、组件之间的交互处理, 并支持实时布局调整。 4. 拖拽容器支持嵌套布局, 允许用户拖拽表单、图表等复杂组件。
6. 动态表单生成器	允许用户根据需求自定义表单, 通过拖拽表单元素生成复杂的动态表单, 支持字段联动、校验、动态显示/隐藏等。	1. 基于表单组件封装表单生成器, 允许用户通过拖拽方式添加表单字段。 2. 每个表单元素的属性如 <code>label</code> 、 <code>value</code> 、 <code>required</code> 等通过 <code>props</code> 进行动态配置。 3. 实现表单字段之间的联动逻辑, 支持条件显示/隐藏、动态校验等。 4. 支持表单提交和校验, 自动生成表单配置 JSON。
7. 模态框/弹窗组件	用于弹出模态框或对话框, 常用于信息展示、表单提交、确认操作等, 支持自定义内容、按钮和回调事件。	1. 封装 <code>Modal</code> 组件, 通过 <code>visible</code> 属性控制模态框显示/隐藏状态。 2. 提供 <code>title</code> 、 <code>content</code> 、 <code>footer</code> 等自定义插槽, 用户可以在模态框中插入自定义内容。 3. 支持 <code>onOk</code> 和 <code>onCancel</code> 回调事件处理确认和取消操作。
8. 图表组件	提供数据可视化的图表组件, 如柱状图、折线图、饼图等, 用户可以通过配置数据源和图表类型生成图表。	1. 使用 <code>Echarts</code> 或 <code>D3.js</code> 等库封装图表组件, 支持通过 <code>props</code> 动态传入数据源和配置项。 2. 图表组件支持切换不同类型的图表, 如柱状图、折线图等。 3. 支持图表的动态刷新和交互操作, 如放大缩小、数据筛选等。
9. 页面导航组件	实现页面间导航功能的组件, 如面包屑导航、侧边栏导航、顶部导航栏等, 帮助用户在页面之间切换。	1. 封装 <code>Breadcrumb</code> 、 <code>Sidebar</code> 、 <code>Navbar</code> 等导航组件, 支持用户通过配置动态生成导航路径。 2. 结合 <code>React Router</code> 实现页面路由的切换。 3. 导航组件支持与权限控制结合, 限制不同用户对页面的访问权限。
10. 条件渲染组件	根据用户输入或业务逻辑条件渲染不同内容或组件, 适用于复杂业务场景下的动态渲染。	1. 封装 <code>ConditionalRender</code> 组件, 支持用户通过 <code>props</code> 传递条件表达式, 根据表达式结果动态渲染内容。 2. 支持多种条件渲染方式, 如 <code>if-else</code> 、 <code>switch</code> 等逻辑。 3. 结合表单和数据展示组件, 实现复杂场景下的条件渲染。
11. 富文本编辑器组件	提供所见即所得的富文本编辑功能, 允许用户插入图片、表格、超链接等, 支持文本样式编辑。	1. 使用 <code>Draft.js</code> 或 <code>Quill</code> 等库封装富文本编辑器, 提供 WYSIWYG 编辑功能。 2. 支持文本格式化、图片上传、超链接插入等操作。 3. 提供 <code>onChange</code> 事件, 用户可以实时获取编辑器中的内容并处理。
12.	用于动态获取和展示来自外部 API 的数	1. 封装 <code>DataSource</code> 组件, 用户通过 <code>props</code> 传递 API 地址, 组件自动发起请求并获取数据。 2. 支持 <code>loading</code> 状态, 处理数据加载过程中的交

API 数据源 组件	据，适合通过接口加载内容的场景，如数据表格、列表等。	互逻辑。 3. 数据源组件支持分页、搜索、筛选等功能，提升数据展示的灵活性。
13. 响应式 设计组 件	提供响应式设计功能，支持页面在不同分辨率下自适应布局，适用于 PC 端和移动端的统一 UI 开发。	1. 封装 Responsive 组件，使用 CSS media queries 或 window.resize 事件监听屏幕大小变化，动态调整组件样式和布局。 2. 提供布局断点配置，允许开发者灵活控制不同屏幕下的布局方式。
14. 权限控 制组件	通过用户角色或权限配置，控制不同用户对页面或组件的可见性和操作权限。	1. 封装 Permission 组件，通过 props 传递用户权限列表，条件渲染页面或组件。 2. 结合 React Router 实现页面级权限控制，未授权用户无法访问特定路由。 3. 支持不同权限角色的配置，提供权限管理的灵活性。

4.Vue低代码核心组件库

核心 组件 名称	组件描述	实现思路（细化）
1. 表 单组 件	提供基础的表单元素，如输入框、选择框、日期选择器、单选框、复选框等，支持表单校验、联动逻辑和动态生成。	1. 封装基础表单元素，如 Input 、 Select 、 DatePicker ，使用 props 传递属性（如默认值、校验规则、占位符、禁用状态）。 2. 使用 v-model 实现双向绑定，支持表单元素实时更新状态。 3. 通过 emit 提供表单状态更新和校验回调。 4. 通过 provide 和 inject 实现表单上下文，管理表单全局状态。 5. 支持表单字段联动和动态显示/隐藏。 6. 提供自定义校验规则，支持异步校验和多字段联动校验。 7. 支持表单的动态生成，用户可通过拖拽添加表单字段。
2. 布 局组 件	用于页面结构布局，支持多种布局方式，如栅格布局、弹性布局、网格布局，帮助用户快速搭建页面结构。	1. 封装 Row 、 Col 等布局组件，通过 props 控制列数、间距、对齐方式等属性。 2. 使用 CSS Flexbox 或 CSS Grid 实现响应式布局，支持不同屏幕尺寸下的自适应。 3. 通过拖拽方式调整布局结构，支持组件动态插入和位置调整。 4. 提供栅格间距和列宽自定义配置，用户可拖拽调整列宽。
3. 数 据展 示组 件	用于展示数据的组件，如表格、列表、图表等，帮助用户创建数据展示面板。	1. 封装 Table 、 List 、 Chart 等数据展示组件，通过 props 传递数据源、列配置等属性。 2. 提供分页、筛选、排序等功能，通过事件 emit 实现与父组件的交互。 3. 支持异步获取数据，结合 API 请求动态加载内容。

		4. 对图表组件（如 ECharts ）进行封装，支持动态数据绑定、切换图表类型。
4. 按钮组件	提供多种样式和功能的按钮组件，如提交按钮、重置按钮、普通操作按钮，适用于表单和页面交互。	1. 封装基础按钮组件，如 Button，通过 props 控制按钮类型（如 submit、reset），支持自定义外观（如颜色、大小、禁用状态）。 2. 通过 @click 事件绑定不同的操作逻辑，支持异步请求和表单提交。 3. 通过插槽（slot）支持自定义按钮内容和图标。
5. 拖拽容器组件	提供拖拽功能的容器，用户可以通过拖拽将组件插入到容器中，动态生成页面结构。	1. 使用 Vue.Draggable 实现拖拽功能，封装 DragContainer 容器组件，允许用户拖拽并放置其他组件到容器中。 2. 使用 v-model 绑定容器的内容和布局，通过拖拽调整组件位置。 3. 通过 emit 事件监听拖拽操作，更新组件位置和层级。 4. 提供拖拽调整的动画效果，提升用户体验。
6. 动态表单生成器	允许用户通过拖拽方式自定义表单字段，动态生成复杂表单，支持字段联动、校验、显示/隐藏等逻辑。	1. 基于表单组件封装表单生成器，用户可通过拖拽生成表单字段。 2. 每个表单字段的 props 可通过配置动态设置，如 label、value、required 等。 3. 实现字段之间的联动逻辑，支持动态显示/隐藏、字段值联动、条件校验等。 4. 最终表单生成后可输出 JSON 配置。
7. 模态框/弹窗组件	用于弹出对话框或模态框，适用于确认操作、信息提示、表单填写等场景，支持自定义内容和交互逻辑。	1. 封装 Modal 组件，通过 v-model 控制显示/隐藏状态，使用 props 传递标题、内容、按钮等配置。 2. 提供 @ok 和 @cancel 事件，处理确认和取消操作。 3. 提供插槽（slot）支持自定义内容区域，允许用户在模态框中插入表单或数据展示组件。
8. 图表组件	提供数据可视化的图表组件，如柱状图、饼图、折线图等，帮助用户创建动态数据展示面板。	1. 使用 ECharts 封装图表组件，通过 props 动态传递数据源、图表类型、配置项等。 2. 支持不同图表类型切换，如折线图、柱状图、饼图等，用户可通过配置选择图表类型。 3. 提供图表的交互功能，如放大、缩小、筛选等，支持数据的动态更新。
9. 页面导航组件	实现页面导航功能的组件，如面包屑导航、侧边栏导航、顶部导航等，帮助用户在多个页面之间进行跳转。	1. 封装 Breadcrumb、Sidebar、Navbar 等组件，支持动态生成导航路径，通过 props 传递导航数据。 2. 结合 Vue Router 实现路由跳转，支持路由激活状态的样式。 3. 导航组件支持权限控制，限制不同用户访问不同的页面。
10. 条件渲染组件	根据用户输入或业务逻辑动态渲染不同的内容或组件，适用于复杂业务场景。	1. 封装 ConditionalRender 组件，通过 props 传递条件表达式，根据表达式结果渲染不同的内容。 2. 支持 v-if、v-show 等条件渲染方式，满足不同业务需求。 3. 支持条件表达式的动态变化，确保用户输入或业务逻辑变化时及时更新显示内容。
11.		1. 使用 Quill 或 TinyMCE 封装富文本编辑器组件，支持文本样式编辑、图片插入、超链接等功能。

富文本编辑器组件	提供 WYSIWYG 富文本编辑功能，允许用户编辑和格式化文本内容，支持图片、表格、超链接等插入。	2. 提供 <code>@input</code> 事件，实时获取编辑器内容，支持内容格式化处理。 3. 支持自定义工具栏，通过 <code>props</code> 配置工具栏按钮。
12. API 数据源组件	用于通过外部 API 获取数据并展示，适用于动态内容加载，如数据表格、列表等。	1. 封装 <code>DataSource</code> 组件，允许用户通过 <code>props</code> 传入 API 地址和请求参数，自动获取数据。 2. 支持 <code>loading</code> 状态展示，处理数据加载中的交互逻辑。 3. 数据源组件支持分页、筛选、搜索等功能，提升数据展示的灵活性和实用性。
13. 响应式设计组件	提供响应式设计支持，确保页面在不同分辨率下自适应布局，适用于多端开发场景，如 PC 端和移动端。	1. 封装 <code>Responsive</code> 组件，使用 <code>CSS media queries</code> 或 <code>window.resize</code> 事件监听屏幕大小变化，动态调整组件的布局和样式。 2. 提供断点配置选项，允许用户自定义不同屏幕尺寸下的样式和布局变化
14. 权限控制组件	通过用户权限或角色配置，控制不同用户对页面或组件的访问权限，确保不同角色用户拥有不同的操作权限。	1. 封装 <code>Permission</code> 组件，通过 <code>props</code> 传入权限数据，判断当前用户是否具有访问权限。 2. 结合 <code>Vue Router</code> 实现页面级权限控制，未授权的用户无法访问指定路由。 3. 支持权限管理的灵活配置，确保不同角色用户的权限差异化。

11. 非技术面试

面试问题	最优方案	口语化话术示例
离职原因	清晰、专业地说明离职原因，重点强调对职业发展的追求，不涉及负面评价。	组织架构调整，个人原因，项目增量少，维护多，在那家公司，我已经实现了主要的项目目标，并且在技术和管理方面都得到了很好的锻炼。现在，我希望找到一个能够提供更大技术挑战和领导机会的角色，以便继续提升自己。
你有什么问题要问我吗	提出有针对性的问题，显示对公司的深入了解和兴趣，关注公司文化、技术方向和团队合作。	面试反馈，几轮面试，技术栈，公司业务，如果我能入职，我主要负责哪一块，我对贵公司非常感兴趣，特别想了解贵公司在技术创新方面的具体策略是什么？团队合作的流程和文化是怎样的？还有，公司未来 1-2 年内在技术领域有怎样的发展计划？这些信息将帮助我更好地了解我能如何融入贵公司的团队和目标。
为什么考虑来XX城市	明确城市的职业和生活优势，突出与个人职业规划的契合度。	家人，朋友，同学都在这里，打算长期发展，我选择 XX 城市主要是因为这里拥有一个活跃的技术社区和丰富的职业机会。此外，城市的生活质量和环境也非常适合我。这些因素都与我的职业目标和生活需求高度契合。
期望薪资	给出明确的薪资范围，并解释该范围如何基于市场行情和个人经验。	我的薪资期望在 X 万到 Y 万之间。这一范围是基于我对市场薪资水平的了解以及我在前端开发领域的丰富经验来设定的。我相信这个薪资水平能够与我为公司带来的价值和我的技能背景相匹配。
你手上有offer吗	诚实地回答是否有其他 offer，并简要说明这些 offer 的进展情况，强调自己对目标公司的兴趣。	目前，我确实在考虑几个其他的 offer，但我对贵公司的机会特别感兴趣，因为它符合我的职业发展目标和个人价值观。我希望能进一步了解这个职位的具体细节，以便做出最合适的决定。
你目前的级别	详细说明当前职位的级别和主要职责，并解释如何与目标职位相匹配。	我目前在 X 公司担任资深前端工程师，主要负责项目的技术架构设计、团队协作以及技术创新。我的职责包括从技术方案的制定到团队的日常管理。我希望在贵公司找到一个能够继续发挥我技术和管理能力的职位，以匹配我的经验和职业目标。
你的绩效考核依据	描述绩效考核的主要标准和如何与公司目标一致，解释自己如何达成这些标准。	在我目前的公司，绩效考核主要依据项目交付质量、Bug率，团队协作效果和技术创新能力。这些标准确保了我們不仅达成了项目目标，而且在团队和技术方面都有所提升。我通常会通过制定明确的目标和持续的自我评估来确保我能够满足这些考核标准。